Frontend Development

Dalla web app React alle app desktop, mobile e native

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Mappa del corso

1. Introduzione: il frontend nel 2026
2. React: un'applicazione completa
3. Dal web al desktop: Electron e Tauri
4. Dal desktop al mobile: responsive, PWA, Capacitor
5. SPA vs sito vetrina: rendering lato server
6. App native: Windows, Linux, macOS, Android, iOS
7. Qualità: testing, accessibilità, prestazioni, sicurezza
8. Consegna: monorepo, CI/CD, deploy, monitoraggio
9. Lavorare con gli agenti di programmazione

Il **progetto finale** e la griglia di valutazione sono in una dispensa a parte: `progetto-finale.pdf`.

Sommario del modulo

1. Introduzione: il frontend nel 2026
1.1 Il corso
1.2 Il progetto: minicommerce
1.3 Il panorama del frontend nel 2026
1.4 Ambiente di lavoro
1.5 Il package manager
1.6 Ripasso rapido di TypeScript
1.7 L'API di minicommerce
1.8 Metodo di lavoro

Benvenuti

- Corso: **Frontend Development** – secondo anno ITS.
- Il frontend non è più solo "il sito": è tutto ciò che l'utente vede e tocca.
- Obiettivo del corso: partire da **una** web app e portarla **ovunque**: browser, desktop, telefono, e capire quando invece conviene scrivere codice nativo.
- Non partiamo da zero: lavoriamo su **minicommerce**, un'applicazione vera e funzionante che vi consegno il primo giorno, con backend e frontend.
- Ogni lezione: teoria breve, poi laboratorio con il codice aperto.

Cosa sapete già (prerequisiti)

Dal primo anno

- HTML, CSS, JavaScript moderno
- Fondamenti di React (componenti, props, state, hook di base)
- Progettazione e sviluppo di una **API REST**
- Un primo contatto con **Capacitor / Ionic**
- Git di base

Cosa daremo per scontato

- Sapete leggere una risposta JSON
- Sapete cosa sono HTTP, verbi, status code
- Sapete usare il terminale
- Il backend di minicommerce è già scritto: in questo corso lo **consumiamo**, lo tocchiamo solo quando serve

Il programma in una slide

0. **Introduzione**: panorama, il progetto minicommerce, setup, ripasso TypeScript
1. **React**: da minicommerce a un'applicazione *completa*
2. **Desktop**: la stessa app diventa un eseguibile con Electron e Tauri
3. **Mobile**: responsive, PWA e app con Capacitor/Ionic
4. **SPA vs sito vetrina**: SSR/SSG con React, Astro, PHP e altri
5. **App native**: Windows, Linux, macOS, Android, iOS – incipit e vantaggi
6. **Qualità**: testing, accessibilità, performance, sicurezza
7. **Consegna**: CI/CD, deploy, monitoraggio
8. **Agenti di programmazione**: come lavorare (bene) con l'AI
9. **Progetto finale** e valutazione

Ripartizione indicativa delle ore

Modulo	Lezione (h)	Laboratorio (h)
0. Introduzione e setup	4	4
1. React: app completa	10	14
2. Desktop (Electron/Tauri)	4	6
3. Mobile (PWA/Capacitor)	6	8
4. Sito vetrina e SSR	6	6
5. App native	8	6
6. Qualità	4	6
7. Consegna e DevOps	3	3
8. Agenti AI	2	2
9. Progetto finale	2	16

Totale circa 120 ore. Se il monte ore è inferiore: tagliare prima i moduli 7 e 5 (parte Linux/macOS), poi il laboratorio del modulo 4.

minicommerce: il filo conduttore

- Un piccolo **e-commerce di articoli informatici**: PC, periferiche, smartphone, accessori.
- Due parti già scritte e funzionanti:
  - `backend/`: API REST con Express, SQLite e Drizzle, login JWT
  - `frontend/`: React 19 + Vite, Tailwind e shadcn/ui
- Tre schermate: **catalogo** pubblico, **login**, **area admin** per gestire gli articoli.
- Abbastanza semplice da tenere in testa, abbastanza ricca da toccare tutti i temi del corso.

Perché un'app sola

Cambiare piattaforma senza cambiare dominio permette di concentrarsi sulle *differenze* tra le piattaforme, non sul reinventare la logica ogni volta.

Alla fine del corso

La stessa minicommerce in 5 forme: web, desktop, PWA, app mobile, sito vetrina indicizzabile.

Le schermate, e i due tipi di utente

`/`	catalogo: ricerca, filtro per categoria, fascia di prezzo, ordinamento. Pubblico.
`/article/:id`	scheda di un articolo, con "aggiungi al carrello".
`/about`	pagina di presentazione.
`/login`	accesso; da qui si registra anche un nuovo cliente.
`/cart`	carrello del cliente e ordine (finto).
`/admin`	tabella articoli: crea, modifica, elimina.

Due ruoli, due mondi

Il campo `isAdmin` dell'utente divide l'applicazione in due:

- il **cliente** naviga il catalogo e riempie il carrello;
- l'**amministratore** gestisce gli articoli e *non* può usare il carrello (l'API gli risponde 403).

Tenetelo a mente: quasi ogni scelta del corso si spiega chiedendosi "per quale dei due?".

Com'è fatto il repository

```
minicommerce/
  README.md              # cosa fa, come si avvia, API, modello dati
  AGENTS.md              # convenzioni del progetto (lo useremo nel modulo 8)
  backend/
    src/server.ts        # Express 5: cors, json, rotte, gestore errori
    src/routes/          # auth.ts, categories.ts, products.ts
    src/middleware/      # auth.ts -> requireAdmin (verifica il JWT)
    src/db/schema.ts     # Drizzle: users, categories, products
    src/seed.ts          # dati demo + utente admin
  frontend/
    index.html
    vite.config.ts       # plugin react + tailwind, proxy /api -> :3001
    src/main.tsx         # BrowserRouter + AuthProvider
    src/App.tsx          # Header + <Routes> + <Toaster>
    src/pages/           # Catalog, Article, About, Login, Cart, Admin
    src/components/      # Header, ProductFormDialog, ui/ (shadcn)
    src/lib/             # api.ts (axios + tipi), auth.tsx (context)
```

Come si avvia

```
# 1. Backend (porta 3001)
cd backend
npm install
npm run db:push      # crea le tabelle SQLite (solo la prima volta)
npm run db:seed      # categorie, articoli demo e utente admin
npm run dev          # node --watch src/server.ts

# 2. Frontend (porta 5173), in un secondo terminale
cd frontend
npm install
npm run dev          # vite, con proxy /api -> http://localhost:3001

# App su http://localhost:5173
# Admin: admin@minicommerce.dev / admin123
```

- Serve **Node.js 24 o superiore**: il backend esegue TypeScript *nativamente*, senza compilazione.
- `npm run db:reset` ricrea il database da zero: comodo, ma cancella i vostri dati locali.

Lo stack di partenza

Parte	Tecnologia	Note
Linguaggio	TypeScript ovunque	backend e frontend
Backend	Node 24, Express 5	TypeScript eseguito nativamente
Database	SQLite + Drizzle ORM	un file, zero installazione
Auth	JWT + bcrypt	token di 8 ore, ruolo `isAdmin`
Build	Vite 8	dev server e bundle di produzione
UI	React 19, Tailwind v4	
Componenti	shadcn/ui su Radix	copiati dentro `src/components/ui/`
Icone, toast	lucide-react, sonner	
HTTP	Axios con interceptor	aggiunge il token a ogni richiesta
Routing	React Router 7	`BrowserRouter`
Lint	Oxlint	molto veloce, niente Prettier

Cosa c'è già e cosa manca

C'è, e funziona

- Catalogo con filtri, ricerca con debounce, ordinamento
- Login con JWT e sessione che sopravvive al refresh
- CRUD degli articoli con validazione del form
- Stato vuoto del catalogo, toast di errore
- Etichette, `aria-invalid`, `role="alert"` nel form

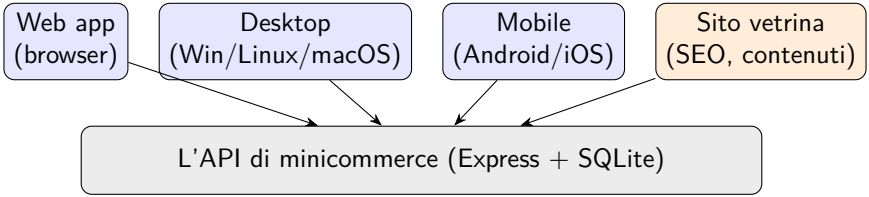
Manca, e lo aggiungeremo

- Nessuna cache: ogni pagina ricarica tutto da capo
- Filtri non nell'URL: non si condividono né si ricaricano
- Nessuno stato di errore in pagina, solo un toast
- Nessun test, nessuna CI, nessun deploy
- Niente offline, desktop o app mobile
- Il catalogo non è indicizzabile da Google

Il corso in una frase

Prendiamo un'app che funziona e la portiamo al livello di un prodotto vero, su cinque piattaforme.

Frontend: che cosa intendiamo



- Il frontend è ogni programma che parla con l'utente e con la vostra API.
- La domanda del corso: **quanto codice possiamo condividere** tra queste forme, e quando conviene **non** condividerlo?

Tre famiglie di tecnologie

Web-based
React, Vue, Svelte, Angular; impacchettate con Electron, Tauri, Capacitor, PWA.

- +Un solo codice
- +Team e strumenti web
- –Prestazioni e “feeling” non nativi

Cross-platform nativo
React Native/Expo, Flutter, .NET MAUI, Kotlin Multiplatform.

- +Widget nativi o quasi
- +Un solo linguaggio
- –Strato intermedio da mantenere

Nativo puro
SwiftUI, Jetpack Compose, WinUI/.NET, GTK/Qt.

- +Massime prestazioni e integrazione
- +Accesso a tutto l’hardware
- –Un codice per piattaforma

Tesi del corso

Nel 2026 il costo di “un codice per piattaforma” è crollato grazie agli agenti di programmazione. Vale la pena ripensare la scelta.

Il browser come piattaforma

Cosa sa fare oggi una web app (senza plugin):

- Lavorare **offline** (Service Worker, Cache API, IndexedDB)
- Ricevere **notifiche push**
- Accedere a **fotocamera, microfono, GPS**
- Leggere/scrivere file locali (File System Access API)
- Bluetooth, USB, seriale (con limiti e permessi)
- Grafica 3D e calcolo (WebGL, **WebGPU**)
- Codice compilato ad alte prestazioni (**WebAssembly**)
- Installazione "come app" (PWA)
- Condivisione nativa (Web Share), pagamenti (Payment Request)

Riferimento: `https://whatwebcando.today` e `https://caniuse.com`

Cosa NON sa fare (bene) il browser

- Girare in background quando la scheda è chiusa (a parte push e sync limitati).
- Accedere liberamente al file system, ai processi, alla rete a basso livello.
- Integrarsi con il sistema: barra di sistema, menu nativi, scorciatoie globali, widget della home.
- Garantire prestazioni costanti su hardware modesto (grafica complessa, liste enormi).
- Essere trovato nello store: su iOS la distribuzione fuori dallo store è di fatto limitata.

Da ricordare

Ognuno di questi limiti è il motivo per cui esistono Electron, Tauri, Capacitor e le app native. Li incontreremo uno per uno.

Come si sceglie: le domande giuste

1. Chi sono gli utenti e **dove** usano l'app? (negozio, magazzino, casa, in mobilità)
2. Serve funzionare **offline**? Quanto spesso?
3. Servono **capacità hardware** specifiche? (fotocamera, lettore di codici a barre, stampante etichette)
4. Serve essere **trovati** da Google? (allora è un sito, non un'app)
5. Quanto conta il **feeling nativo**? (vetrina per i clienti vs gestionale per i commessi)
6. Quante persone manterranno il codice tra due anni?

Esercizio da fare in aula su minicommerce: rispondete alle sei domande per il *catalogo* (clienti) e per l'*area admin* (negozio). Vengono fuori due risposte diverse: è il punto di partenza di tutto il corso.

Strumenti che useremo

- **Node.js** 24 LTS (o superiore): obbligatorio per il backend
- **npm**, incluso in Node: nessuna installazione in più
- **VS Code** con estensioni: Oxc, ESLint, Error Lens, GitLens, Tailwind CSS IntelliSense
- **Git** + account GitHub
- Browser: Chrome/Edge (DevTools) e Firefox
- Rust toolchain (per Tauri), Android Studio, Xcode (solo su Mac)

```
# Verifica installazione
node --version      # v24.x o superiore
npm --version       # 11.x
git --version

# Il progetto
git clone <url>/its2026_frontend_dev
cd its2026_frontend_dev
```

La toolchain moderna in breve

Vite	Dev server istantaneo e bundler di produzione. È quello che fa partire il frontend di minicommerce.
TypeScript	JavaScript con i tipi. In questo progetto **tutto** è TypeScript, anche il backend.
Oxlint	Il linter del progetto: scritto in Rust, trova errori e cattive pratiche in millisecondi. ESLint fa le stesse cose, più lentamente e con più plugin disponibili.
Prettier	Formatta il codice in modo uniforme. Non è configurato nel progetto: nel modulo 6 decideremo se aggiungerlo.
Vitest	Test runner compatibile con Vite. Il progetto non ha test: li scriviamo noi nel modulo 6.
Drizzle	L'ORM del backend. Lo sfioriamo soltanto: a noi interessa l'API che espone.

Configurazione del frontend

```
// vite.config.ts del progetto
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: { '@': path.resolve(import.meta.dirname, './src') },   // import '@/lib/api'
  },
  server: {
    proxy: { '/api': 'http://localhost:3001' },   // niente CORS in sviluppo
  },
})
```

- Il **proxy** fa sì che il browser veda una sola origine (`localhost:5173`): le chiamate ad `/api` vengono inoltrate al backend.
- In produzione il proxy non esiste: sarà il nostro primo problema di deploy (modulo 1) e tornerà su desktop e mobile.
- L'alias `@` evita i `../../..` negli import.

Che cos'è un package manager

- Il programma che **scarica e organizza le librerie** di cui il progetto ha bisogno, con tutte le loro dipendenze.
- Due file, una cartella e un registro:
  - `package.json`: cosa volete e in che versione
  - `package-lock.json`: cosa avete ottenuto *esattamente*
  - `node_modules/`: il codice scaricato
  - il **registro** (`registry.npmjs.org`): da dove si scarica
- Serve anche a **eseguire gli script** del progetto (`dev`, `build`, `lint`).

In questo corso: npm

È incluso in Node, quindi c'è già ovunque, anche sui PC del laboratorio dove non possiamo installare altro. minicommerce arriva con il suo `package-lock.json`: lo usiamo così com'è.

Da sapere

Esistono **pnpm**, **Yarn** e **Bun**. Installano gli *stessi* pacchetti dallo *stesso* registro: cambia come li mettono su disco. Ne parliamo fra due slide.

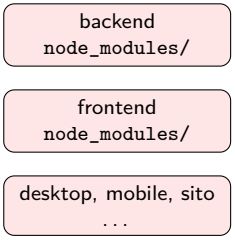
I comandi che useremo

Comando	Cosa fa
`npm install`	installa tutto quello che dice `package.json`
`npm install axios`	aggiunge una dipendenza e la scrive nel file
`npm install -D vitest`	dipendenza di sviluppo (non finisce in produzione)
`npm uninstall axios`	rimuove una dipendenza
`npm run dev`	esegue uno script di `package.json`
`npm run`	elenca gli script disponibili: usatelo in un progetto nuovo
`npx playwright test`	esegue un binario installato, o lo scarica al volo
`npm ci`	in CI: installa *esattamente* il lockfile, da zero
`npm audit`	vulnerabilità note nelle dipendenze

Il **lockfile va sempre committato**: è ciò che fa sì che voi, il compagno di banco e il server di build installiate le stesse identiche versioni.

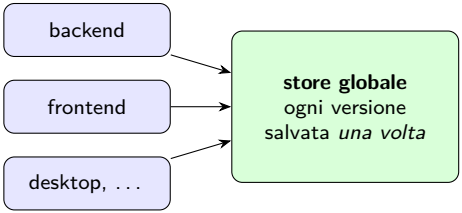
Il prezzo di npm: `node_modules`

- Ogni progetto riscarica e ricopia tutto: React, Vite e compagnia stanno **una volta per progetto**.
- minicommerce ha già **due** `node_modules`; nel corso diventeranno cinque o sei, con desktop, mobile e sito.
- npm appiattisce l'albero: in `node_modules/` finiscono anche le dipendenze *delle* dipendenze, e voi potete importarle per sbaglio.
- Funziona da voi, si rompe in CI o sul PC del compagno: sono le **phantom dependencies**.



Convivere con questi difetti si può: basta conoscerli. Importate solo ciò che avete dichiarato, e in CI usate `npm ci`.

L'alternativa che troverete: pnpm



- Ogni versione di ogni pacchetto è scaricata **una volta per macchina**; nei progetti arriva come **hard link**, non come copia.
- `node_modules/` contiene **solo** ciò che avete dichiarato: le phantom dependencies non esistono proprio.
- Non esegue gli script di installazione dei pacchetti senza permesso: una difesa in più (modulo 6).
- Yarn è storico e oggi di nicchia; Bun è un runtime oltre che un package manager, velocissimo, ancora poco diffuso in azienda.

Perché non lo usiamo: sui PC del laboratorio non possiamo installare nulla. Non è una bocciatura di pnpm, è un vincolo dell'aula.

Se un giorno volete passare a pnpm

```
# sul vostro computer, dove potete installare: una volta sola
npm install -g pnpm           # oppure: corepack enable
# senza installare nulla, per provarlo: npx pnpm --version

# conversione di un progetto esistente, dentro la cartella del progetto
npx pnpm import               # legge package-lock.json -> pnpm-lock.yaml
rm -rf node_modules package-lock.json
pnpm install

# da li' in poi
pnpm dev                      # al posto di npm run dev
pnpm add axios                # al posto di npm install axios
```

- Il campo `"packageManager"` in `package.json` fissa lo strumento per chiunque apra il repository.
- Nel mondo del lavoro troverete tutti e quattro. Saper leggere un `package.json` e un lockfile conta più dello strumento.

I tipi del progetto

```ts
// frontend/src/lib/api.ts -- questi tipi li useremo per tutto il corso
export interface Category {
  id: number;
  name: string;
  description: string;
  productCount: number;      // calcolato dal backend
}

export interface Product {
  id: number;
  name: string;
  description: string;
  categoryId: number;
  category: string;          // nome della categoria, dalla join lato backend
  brand: string;
  price: number;
  stock: number;
  imageUrl: string | null;  // puo' mancare: il tipo lo dice
  createdAt: string;
}
```

Tipi derivati e union

```ts
// In scrittura la categoria si indica con l'id, e non servono id/createdAt
export interface ProductInput {
  name: string; description: string; categoryId: number;
  brand: string; price: number; stock: number; imageUrl: string | null;
}
// equivalente, senza riscrivere i campi:
type ProductInput2 = Omit<Product, 'id' | 'category' | 'createdAt'>;

// Union di stringhe: i valori ammessi sono solo questi
export interface ProductFilters {
  search?: string;                                  // ? = opzionale
  category?: string;
  minPrice?: string;
  maxPrice?: string;
  sort?: 'name' | 'price' | 'stock' | 'createdAt';
  order?: 'asc' | 'desc';
}
type SortField = NonNullable<ProductFilters['sort']>;   // 'name' | 'price' | ...
```

`Omit`, `Pick`, `Partial`, `NonNullable`: derivate un tipo da un altro invece di duplicarlo.

Narrowing e generics

```
function etichettaOrdinamento(s: SortField): string {
  switch (s) {
    case 'name':      return 'Nome';
    case 'price':     return 'Prezzo';
    case 'stock':     return 'Disponibilita';
    case 'createdAt': return 'Piu recenti';
    // se aggiungo un campo ordinabile e dimentico un case, TS mi avvisa
  }
}

// Generics: una funzione che vale per qualsiasi tipo
function groupBy<T, K extends string>(items: T[], key: (t: T) => K) {
  const out = {} as Record<K, T[]>;
  for (const it of items) (out[key(it)] ??= []).push(it);
  return out;
}
const perCategoria = groupBy(products, p => p.category);   // Record<string, Product[]>
```

Il client HTTP del progetto

```ts
import axios from 'axios';

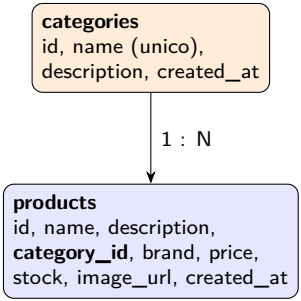
export const api = axios.create({ baseURL: '/api' });

// A ogni richiesta allega il token, se c'e'
api.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export async function fetchProducts(filters: ProductFilters = {}) {
  const { data } = await api.get<Product[]>('/products', { params: filters });
  return data;
}
```

- **Axios** lancia un'eccezione sugli status 4xx e 5xx; `fetch` no: lì si controlla `res.ok`.
- `params` costruisce la query string: `/products?search=mouse&sort=price`.
- Un unico punto di ingresso per tutte le chiamate: base URL, token, errori.

Il modello dati



- Una categoria raggruppa molti articoli; ogni articolo appartiene a **una** categoria.
- Chiave esterna con `ON DELETE RESTRICT`: una categoria con articoli **non** si può cancellare.
- C'è anche `users` (email, password_hash, name, is_admin) per il login.
- Le risposte sugli articoli contengono sia `categoryId` sia `category` (il nome): niente seconda chiamata per l'etichetta.

Gli endpoint

Metodo	Rotta	Auth	Note
POST	`/api/auth/login`	–	`{email, password}` → `{token, user}`
POST	`/api/auth/register`	–	crea un cliente
GET	`/api/categories`	–	categorie con `productCount`
GET	`/api/products[/:id]`	–	lista con filtri, oppure il dettaglio
POST / PUT / DELETE	`/api/products[/:id]`	admin	crea, aggiorna, elimina
GET	`/api/cart`	cliente	`{ items, total }`
POST / PATCH / DELETE	`/api/cart/items[/:id]`	cliente	aggiunge, cambia quantità, rimuove
POST	`/api/orders`	cliente	ordine finto: accetta e svuota il carrello

Filtri del catalogo: `search`, `category`, `categoryId`, `brand`, `minPrice`, `maxPrice`, `sort`, `order`. Header `Authorization: Bearer <jwt>`: senza token 401, con il ruolo sbagliato 403. Il **carrello vive sul server**, legato all'utente: non è uno stato del browser.

Esempio di richiesta e risposta

```
GET /api/products?search=mouse&sort=price&order=asc
200 OK
[
  {
    "id": 7,
    "name": "Mouse wireless Pro",
    "description": "Mouse ergonomico 2.4 GHz con ricevitore USB-C",
    "categoryId": 2,
    "category": "Periferiche",
    "brand": "Logitech",
    "price": 39.9,
    "stock": 18,
    "imageUrl": null,
    "createdAt": "2026-09-20 09:12:00"
  }
]
```

- `price` è un numero (`real` in SQLite): la formattazione in euro è compito del frontend.
- Gli id sono numeri interi assegnati dal database.

Provare l'API senza il frontend

```
# elenco articoli
curl http://localhost:3001/api/products | head

# login: prendiamo il token
curl -X POST http://localhost:3001/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@minicommerce.dev","password":"admin123"}'

# creare un articolo (serve il token)
curl -X POST http://localhost:3001/api/products \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"name":"Tastiera 60%","categoryId":2,"brand":"Keychron","price":89.9,"stock":5}'
```

- Imparate a leggere l'API **prima** di scrivere il frontend: metà dei bug sono malintesi sul contratto.
- Alternative a curl: l'estensione REST Client di VS Code, Bruno, Postman, o la scheda Rete del browser.

Git durante il corso

- Un repository per studente, partendo dal codice di minicommerce che vi consegno.
- Un branch per laboratorio: `lab/01-query`, `lab/02-tauri`, ...
- Commit piccoli con messaggi che spiegano il *perché*.
- Pull request verso `main` a fine laboratorio: la userò per la valutazione continua.
- Mai committare segreti: il `JWT_SECRET` di produzione non entra nel repository.
- Il file del database (`minicommerce.db`) è locale: non si committa, si rigenera con `db:seed`.

Convenzione per i messaggi

`feat(catalogo): filtro per marca` – `fix(auth): gestione del 401` – `chore: aggiorna dipendenze`

Agenti di programmazione: le regole della classe

- Potete usare Copilot, Claude Code, Cursor, Codex e simili: nel 2026 fa parte del mestiere.
- Nel repository c'è già un `AGENTS.md` con le convenzioni del progetto: leggetelo, è anche il vostro manuale di stile.
- Regola 1: **dovete saper spiegare** ogni riga che consegnate. Se non la capite, non è vostra.
- Regola 2: l'agente scrive, **voi rivedete**. Il diff si legge prima di committare.
- Regola 3: nei laboratori marcati "a mano" l'agente resta chiuso: servono a costruire le basi.
- Regola 4: nel progetto finale documentate *come* avete usato l'agente (prompt, cosa ha sbagliato).

Laboratorio 0

Laboratorio: Avviare e capire minicommerce

1. Clonare il repository, avviare backend (`db:push`, `db:seed`, `dev`) e frontend; entrare come admin.
2. Provare l'API con curl o REST Client: lista articoli, login, creazione di un articolo.
3. Disegnare su carta il percorso di un dato: dal click sul filtro "Periferiche" alla card sullo schermo, passando per i file toccati.
4. Prima modifica: aggiungere al catalogo il filtro per **marca** (il backend lo supporta già, il frontend no), poi aprire una PR.

Domande di verifica

Perché in sviluppo non serve configurare CORS? Che differenza c'è tra `categoryId` e `category` nella risposta? Dove viene aggiunto il token alle richieste? Cosa succede se cancellate una categoria che ha articoli?

Sommario del modulo

2. React: un'applicazione completa
2.1 Da dove partiamo
2.2 Ripasso di React sul codice del progetto
2.3 Routing
2.4 Autenticazione
2.5 I dati dal server
2.6 Form e validazione
2.7 Stato dell'applicazione
2.8 Interfaccia: Tailwind e shadcn/ui
2.9 Funzionalità da prodotto vero
2.10 Robustezza e prestazioni
2.11 Build e pubblicazione

Obiettivo del modulo

- minicommerce *funziona*. Non è ancora un **prodotto**.
- Differenza fra le due cose: cache, URL condivisibili, errori gestiti in pagina, form robusti, prestazioni, build pubblicata.
- In questo modulo tocchiamo quasi solo `frontend/src`: il backend resta com'è.
- Metodo: per ogni argomento guardiamo **il codice che c'è**, diciamo cosa non va, e lo miglioriamo.

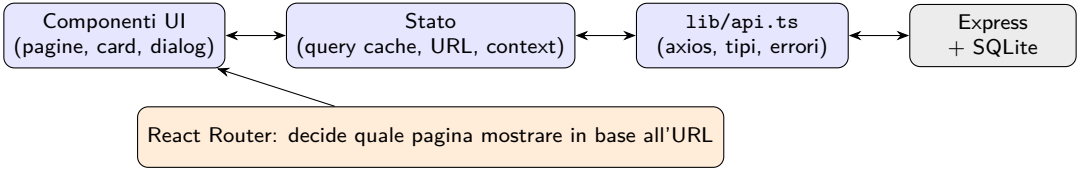
Cosa aggiungiamo allo stack

Un hook per i dati scritto da noi, react-hook-form + zod (form), filtri nell'URL con React Router, Error Boundary, code splitting.

La lista della spesa del modulo

Com'è oggi	Dove lo portiamo
`useEffect` + `useState` copiati in ogni pagina	un hook `useProducts` solo, con annullamento e *reload*
Filtri in `useState`	filtri nell'URL: link condivisibili, tasto Indietro
Errori solo in un toast che sparisce	stato di errore in pagina con pulsante Riprova
`/admin` e `/cart` si proteggono da sole	route protette da un layout, con il ruolo
401 non gestito	interceptor di risposta: sessione scaduta $\rightarrow$ login
Validazione del form scritta a mano	schema zod riusabile, meno codice
`window.confirm` per eliminare	dialog accessibile e coerente con il resto
Tutto il JS in un solo file	una route, un file: code splitting

Architettura di una SPA



- **Single Page Application**: un solo `index.html`, il JavaScript disegna tutto e parla con l'API in JSON.
- Il browser non ricarica mai la pagina: la navigazione è gestita dal router.
- Conseguenza: in produzione il server deve rispondere `index.html` a *qualsiasi* percorso. Ci torniamo a fine modulo.

Componenti e props

```tsx
// una card del catalogo, estratta da CatalogPage in un componente suo
interface ProductCardProps {
  product: Product;
  onEdit?: (product: Product) => void;    // opzionale: nel catalogo non serve
}

export function ProductCard({ product, onEdit }: ProductCardProps) {
  return (
    <Card className="flex flex-col">
      <CardTitle className="text-base">{product.name}</CardTitle>
      <Badge variant="secondary">{product.category}</Badge>
      <span className="text-lg font-bold">{formatPrice(product.price)}</span>
      {product.stock > 0
        ? <Badge variant="outline">Disponibili: {product.stock}</Badge>
        : <Badge variant="destructive">Esaurito</Badge>}
    </Card>
  );
}
```

Un componente è una **funzione pura**: stesse props, stesso risultato. I tipi delle props sono la sua documentazione.

State ed eventi: la ricerca del catalogo

```
const [search, setSearch] = useState('');

<Input
  placeholder="Cerca per nome, marca o descrizione..."
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
```

- Un input con `value` + `onChange` è **controllato**: React è la fonte di verità, non il DOM.
- Lo stato vive nel componente più in basso che ne ha bisogno. Se serve a due fratelli, sale nel padre (*lifting*).
- In `CatalogPage` ci sono sei stati di filtro (`search`, `category`, `minPrice`, `maxPrice`, `sort`, `order`): fra poco li spostiamo tutti nell'URL, e il componente ne esce più semplice.

Liste, chiavi, rendering condizionale

```
{loading ? (
  <p className="py-12 text-center text-muted-foreground">Caricamento...</p>
) : products.length === 0 ? (
  <div className="flex flex-col items-center gap-2 py-12">
    <PackageX className="size-8" />
    <p>Nessun articolo trovato con i filtri selezionati.</p>
  </div>
) : (
  <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
    {products.map((product) => <ProductCard key={product.id} product={product} />)}
  </div>
)}
```

- La `key` serve a React per capire *quale* elemento è cambiato: stabile e unica. Qui è `product.id`: corretto.
- Errore classico: `key={index}` con una lista che si riordina. Il catalogo si riordina a ogni cambio di `sort`.
- Il ternario annidato regge tre casi. Ne mancano due: **errore** e **aggiornamento in corso**. Li aggiungiamo nel modulo.

useEffect: quando sì e quando no

```
// Nel progetto: carica le categorie una volta sola, al montaggio
useEffect(() => {
  fetchCategories().then(setCategories).catch(err => toast.error(getErrorMessage(err)));
}, []);

// Nel progetto: ricarica gli articoli quando cambia un filtro, con debounce
useEffect(() => {
  const timer = setTimeout(() => { /* fetchProducts(...) */ }, 250);
  return () => clearTimeout(timer);     // pulizia: annulla il timer precedente
}, [search, category, minPrice, maxPrice, sort, order]);
```

- La funzione restituita è la **pulizia**: qui annulla il timer, ed è ciò che rende il debounce corretto.
- **Sì** agli effetti per sincronizzare con qualcosa fuori da React (timer, titolo della pagina, sottoscrizioni).
- **No** per calcolare dati derivati, e (fra poco vedremo perché) **no** per caricare dati dal server.

React 19: cosa usa già il progetto

```
// 1. Il Context si usa direttamente come provider: niente <AuthContext.Provider>
return <AuthContext value={{ user, login, logout }}>{children}</AuthContext>;

// 2. startTransition: logout e navigazione insieme, senza render intermedi
const handleLogout = () => {
  startTransition(() => { logout(); navigate('/'); });
};
```

- Senza la transition, fra `logout()` e `navigate('/')` React farebbe un render con `user === null` mentre siamo ancora su `/admin`: la pagina reindirizzerebbe a `/login`. Il commento nel codice lo dice.
- Altre novità di React 19 che incontreremo: `ref` come prop normale (niente `forwardRef`), `useActionState` per i form, `<title>` e `<meta>` scrivibili dentro i componenti.

Composizione: children e slot

- Non fate componenti con quindici prop booleane: passate **children**.
- È il pattern di shadcn/ui, che avete già nel progetto: `<Card>` contiene `<CardHeader>`, `<CardContent>`, `<CardFooter>`; voi decidete cosa ci va dentro.
- `asChild` di Radix: `<Button asChild><Link to="/">Catalogo</Link></Button>` dà l'aspetto del bottone a un link vero. Lo usa già `Header.tsx`.
- Custom hook: quando la stessa *logica* (non la stessa UI) serve in due posti, estraetela in `useQualcosa()`.

Esercizio in aula

`CatalogPage` è lunga più di 200 righe e fa tre cose. Dividetela in `ProductFilters`, `ProductGrid`, `ProductCard` e un hook `useProductFilters()`. È il refactoring più formativo del modulo.

Le route di oggi

```
// App.tsx, cosi' com'e': sei route, dentro <div className="min-h-screen">
<Header />
<Routes>
  <Route path="/" element={<CatalogPage />} />
  <Route path="/article/:articleid" element={<ArticlePage />} />
  <Route path="/cart" element={<CartPage />} />
  <Route path="/about" element={<AboutPage />} />
  <Route path="/login" element={<LoginPage />} />
  <Route path="/admin" element={<AdminPage />} />
</Routes>
<Toaster richColors position="top-right" />
```

- `<Header />` e `<Toaster />` stanno fuori da `<Routes>`: restano su tutte le pagine. È corretto.
- Manca una route `*`: un URL sbagliato mostra una pagina vuota con solo l'header.
- Le protezioni stanno dentro le pagine: `AdminPage` rimanda a `/login` chi non è amministratore, `CartPage` rimanda a `/admin` l'admin. Il controllo è già **duplicato**.

Route protette con un layout

```tsx
// components/ProtectedLayout.tsx -- con due ruoli serve anche isAdmin
export function ProtectedLayout({ adminOnly = false }: { adminOnly?: boolean }) {
  const { user } = useAuth();
  const location = useLocation();
  if (!user) return <Navigate to="/login" replace state={{ from: location }} />;
  if (adminOnly && !user.isAdmin) return <Navigate to="/" replace />;
  return <Outlet />;       // qui viene renderizzata la route figlia
}

// App.tsx
<Route element={<ProtectedLayout />}>
  <Route path="/cart" element={<CartPage />} />
</Route>
<Route element={<ProtectedLayout adminOnly />}>
  <Route path="/admin" element={<AdminPage />} />
</Route>
<Route path="*" element={<NotFoundPage />} />
```

Con `state: { from }` dopo il login si torna dove si voleva andare. Il campo `isAdmin` dell'utente divide clienti e amministratori: il carrello è dei clienti, l'area admin degli amministratori.

I filtri appartengono all'URL

```tsx
import { useSearchParams } from 'react-router-dom';

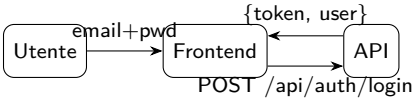
const [params, setParams] = useSearchParams();
const search = params.get('search') ?? '';
const sort   = (params.get('sort') ?? 'name') as SortField;

function setFilter(key: string, value: string) {
  setParams(prev => {
    if (value) prev.set(key, value); else prev.delete(key);
    return prev;
  }, { replace: true });    // replace: non riempie la cronologia a ogni tasto
}

<Input value={search} onChange={(e) => setFilter('search', e.target.value)} />
```

- Risultato: `/?search=mouse&sort=price` si può **inviare a un collega**, mettere nei preferiti, ricaricare.
- Il tasto Indietro torna ai filtri precedenti: è ciò che l'utente si aspetta.
- I parametri dell'URL hanno già la forma di quelli dell'API: meno conversioni.

Il flusso di login



1. Il backend confronta la password con `bcrypt` e firma un **JWT** che dura **8 ore**.
2. Il frontend salva token e utente in `localStorage` e li rimette allo **stato** React.
3. L'interceptor di axios allega il token a *ogni* richiesta successiva.
4. Al ricaricamento della pagina l'utente resta dentro, perché lo stato iniziale si legge da `localStorage`.
5. Stessa pagina, secondo percorso: `POST /api/auth/register` crea un **cliente** (`isAdmin: false`) e risponde con la stessa coppia `{token, user}`. Dopo l'accesso, l'amministratore va su `/admin`, il cliente sul catalogo.

L'AuthProvider del progetto

```tsx
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(loadStoredUser);   // lazy init

  const login = async (email: string, password: string) => {
    const { token, user } = await apiLogin(email, password);
    localStorage.setItem(TOKEN_KEY, token);
    localStorage.setItem(USER_KEY, JSON.stringify(user));
    setUser(user);
  };
  const logout = () => { /* rimuove le due chiavi e azzera lo stato */ };

  return <AuthContext value={{ user, login, logout }}>{children}</AuthContext>;
}
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth deve essere usato dentro AuthProvider');
  return ctx;
}
```

`useState(loadStoredUser)`: la funzione è passata *senza* parentesi, così viene eseguita una volta sola al primo render e non a ogni render.

Il buco: il token scaduto

Dopo 8 ore il token non vale più. Oggi l'utente vede una tabella vuota e un toast generico, ma resta "loggato".

```
// lib/api.ts: un interceptor di RISPOSTA accanto a quello di richiesta
api.interceptors.response.use((response) => response, (error) => {
  if (axios.isAxiosError(error) && error.response?.status === 401) {
    localStorage.removeItem(TOKEN_KEY);
    localStorage.removeItem(USER_KEY);
    window.dispatchEvent(new Event('auth:expired'));   // l'AuthProvider ascolta
  }
  return Promise.reject(error);
});

// AuthProvider: azzera l'utente quando arriva l'evento
useEffect(() => {
  const onExpired = () => setUser(null);
  window.addEventListener('auth:expired', onExpired);
  return () => window.removeEventListener('auth:expired', onExpired);
}, []);
```

Con la route protetta del layout, azzerare `user` basta: l'utente finisce su `/login` da solo.

Dove salvare il token

Opzione	Pro	Contro
Memoria (variabile JS)	immune al furto via storage	persa a ogni reload
`localStorage` (il progetto)	semplice, persiste	leggibile da qualsiasi script (XSS)
Cookie `httpOnly`	non leggibile da JS	va cambiato il backend, CSRF da gestire

- minicommerce usa `localStorage`: è anche la scelta che funziona identica in Electron, Tauri e Capacitor, dove i cookie sono più complicati.
- Nel modulo 6 vedremo perché per il *solo* web il cookie `httpOnly` è preferibile, e cosa cambierebbe nel backend.
- In ogni caso: il token è una **chiave**, non un segreto da nascondere nel codice. Chi ce l'ha, è l'utente.

Laboratorio 1a

Laboratorio: Routing e sessione (a mano, senza agenti)

1. Estrarre `ProtectedLayout` (con l'opzione `adminOnly`) e spostarci i controlli che oggi stanno dentro `AdminPage` e `CartPage`.
2. Aggiungere la route `*` con una pagina "404" che riporta al catalogo.
3. Interceptor di risposta per il 401 + evento `auth:expired` gestito dall'`AuthProvider`.
4. Dopo il login tornare alla pagina richiesta (`state.from`), non sempre a `/admin`.
5. Provare: cancellate a mano il token da DevTools, poi cliccate "Elimina" su un articolo.

Domande di verifica

Perché il controllo di autenticazione sta meglio nel layout? Cosa fa `replace` in `<Navigate>`?
Perché `useState(loadStoredUser)` e non `useState(loadStoredUser())`?

Il codice che c'è, e cosa gli manca

```
const [products, setProducts] = useState<Product[]>([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  const timer = setTimeout(() => {
    setLoading(true);
    fetchProducts({ search, category, minPrice, maxPrice, sort, order })
      .then(setProducts)
      .catch(err => toast.error(getErrorMessage(err)))
      .finally(() => setLoading(false));
  }, 250);
  return () => clearTimeout(timer);
}, [search, category, minPrice, maxPrice, sort, order]);
```

- Funziona. Ma: nessuno stato di **errore** in pagina, nessun **annullamento** della richiesta precedente, e la stessa logica è ricopiata in `AdminPage`, `ArticlePage`, `CartPage` e `ProductFormDialog`.
- Il difetto peggiore è il terzo: cinque copie che si comportano in modo leggermente diverso.

Un hook, una volta sola

```tsx
// lib/useProducts.ts
export function useProducts(filters: ProductFilters) {
  const [data, setData] = useState<Product[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [tick, setTick] = useState(0);            // reload(): fa ripartire l'effetto

  useEffect(() => {
    const ctrl = new AbortController();
    setLoading(true); setError(null);
    fetchProducts(filters, ctrl.signal)
      .then(setData)
      .catch(e => { if (!ctrl.signal.aborted) setError(getErrorMessage(e)); })
      .finally(() => { if (!ctrl.signal.aborted) setLoading(false); });
    return () => ctrl.abort();                    // annulla la richiesta precedente
  }, [JSON.stringify(filters), tick]);

  return { data, loading, error, reload: () => setTick(t => t + 1) };
}
```

Le due righe che contano

```ts
// lib/api.ts: basta accettare il signal e passarlo ad axios
export async function fetchProducts(filters: ProductFilters = {}, signal?: AbortSignal) {
  const { data } = await api.get<Product[]>('/products', { params: filters, signal });
  return data;
}
```

- `AbortController` risolve un bug che non avete ancora visto: scrivendo in fretta nella ricerca partono più richieste, e **non arrivano in ordine**. Senza annullamento, una risposta vecchia può sovrascrivere quella giusta.
- `JSON.stringify(filters)` nelle dipendenze è un trucco: l'oggetto dei filtri è nuovo a ogni render, quindi va confrontato per *valore*. L'alternativa pulita è costruirlo con `useMemo`.
- Lo stesso schema vale per `useCategories()`: due righe, nessun filtro.

La pagina diventa più corta

```
export function CatalogPage() {
  const [params] = useSearchParams();
  const filters = useDebounced(filtersFromParams(params), 250);
  const { data: products, loading, error, reload } = useProducts(filters);

  if (loading && products.length === 0) return <CatalogSkeleton />;
  if (error)   return <ErrorPanel message={error} onRetry={reload} />;
  if (products.length === 0) return <EmptyState />;

  return <ProductGrid products={products} busy={loading} />;
}
```

- La pagina non sa più *come* si caricano i dati: sa solo in quale stato si trova.
- `loading && products.length === 0`: scheletro solo al primo caricamento. Ai successivi restano i dati vecchi, leggermente sbiaditi.
- `AdminPage` usa lo stesso hook con filtri diversi, e la duplicazione sparisce.

Scrivere: creare, aggiornare, eliminare

```ts
// lib/useSaveProduct.ts
export function useSaveProduct(onSaved: () => void) {
  const [saving, setSaving] = useState(false);

  async function save(id: number | undefined, input: ProductInput) {
    setSaving(true);
    try {
      const saved = id ? await updateProduct(id, input) : await createProduct(input);
      toast.success(id ? 'Articolo aggiornato' : 'Articolo creato');
      onSaved();                          // chi possiede la lista la ricarica
      return saved;
    } finally {
      setSaving(false);                   // anche se l'API fallisce: il bottone si riattiva
    }
  }
  return { save, saving };
}
```

`AdminPage` possiede la lista e passa il suo `reload` come `onSaved`: è già quello che fa oggi il progetto, e a questa scala va benissimo.

Il carrello: lo stesso schema, su dati del server

```
// CartPage: il carrello non sta nel client, si legge e si riscrive
const loadCart = useCallback(() => {
  fetchCart().then(setCart).catch(e => toast.error(getErrorMessage(e)));
}, []);

async function cambiaQuantita(productId: number, quantity: number) {
  await updateCartItem(productId, quantity);   // PATCH /api/cart/items/:productId
  loadCart();                                  // il totale lo ricalcola il server
}
```

- Il carrello è di chi è loggato **come cliente**: a un token da amministratore l'API risponde 403.

L'ordine è finto

`placeMockOrder` chiama `POST /api/orders`: l'ordine è registrato con stato `accepted` e il carrello svuotato. Nessun pagamento, nessun controllo della carta, nessuno scarico delle scorte. Si salvano solo intestatario e ultime quattro cifre: numero completo, scadenza e CVV non vengono nemmeno inviati.

Aggiornamento ottimistico, a mano

```
function StockCell({ product, onChanged }: StockCellProps) {
  const [stock, setStock] = useState(product.stock);
  const [busy, setBusy] = useState(false);
  useEffect(() => setStock(product.stock), [product.stock]);   // la lista ricaricata vince

  async function cambia(delta: number) {
    const precedente = stock;
    setStock(stock + delta);                                   // 1. la UI cambia subito
    setBusy(true);
    try { await updateProduct(product.id, { ...toInput(product), stock: stock + delta });
     onChanged(); }
    catch (e) { setStock(precedente); toast.error(getErrorMessage(e)); }  // 2. si torna indietro
    finally { setBusy(false); }
  }
  ...
}
```

Nessuna libreria: due righe di stato locale. La UI risponde subito e, se il server rifiuta, il numero torna com'era. Non esiste una rotta per il solo stock: si riusa `updateProduct`, cioè `PUT /api/products/:id` con tutti i campi.

I quattro stati di ogni schermata

1. **Caricamento**: scheletro della griglia, non uno spinner al centro dello schermo.
2. **Errore**: messaggio comprensibile e pulsante *Riprova*. Oggi minicommerce mostra solo un toast che sparisce dopo 4 secondi.
3. **Vuoto**: c'è già, con icona e testo. Buon esempio da imitare.
4. **Dati**: la griglia.

Il quinto stato

Dati vecchi mentre ne arrivano di nuovi: con `busy` si abbassa un po' l'opacità della griglia. L'utente capisce che sta succedendo qualcosa senza perdere ciò che sta leggendo.

Regola

Un toast è un *avviso*, non uno stato. Se l'informazione serve ancora dopo cinque secondi, deve stare in pagina.

Quando il progetto cresce: le librerie

Quello che avete scritto sta in trenta righe e basta per minicommerce. Da una certa dimensione in poi servono cose che a mano costano care:

- **Cache condivisa** fra componenti e fra pagine: due schermate che chiedono le categorie fanno *una* richiesta.
- **Ricarica in background** quando si torna sulla scheda, e dopo un tempo configurabile.
- **Retry** automatici, deduplica delle richieste uguali, cache che sopravvive alla chiusura.

Da conoscere di nome

TanStack Query e **SWR** sono le due risposte standard nel mondo React, e le troverete in azienda. In questo corso non le usiamo: prima bisogna sapere *cosa* fanno al posto vostro, altrimenti diventano magia.

Laboratorio 1b

Laboratorio: Catalogo e admin su un hook solo

1. Spostare i filtri nell'URL con `useSearchParams`.
2. Scrivere `useProducts` e `useCategories` in `lib/`, con `AbortController` e `reload`; stessa forma per un `useCart()`.
3. `CatalogPage` e `AdminPage` usano gli hook: tutti e quattro gli stati presenti, `ErrorPanel` con Riprova.
4. `useSaveProduct` per creazione e modifica; l'eliminazione ricarica la lista.
5. Provare il bug delle risposte fuori ordine: rallentate `/api/products` con un `setTimeout` nel backend, digitate in fretta, e verificate che l'annullamento lo risolva.

Domande di verifica

Perché una risposta vecchia può sovrascrivere quella nuova? Cosa fa `AbortController`? Perché i filtri vanno confrontati per valore e non per riferimento? Chi possiede la lista degli articoli?

La validazione scritta a mano

```
// ProductFormDialog.tsx, com'e' oggi: 7 campi come stringhe + una funzione
function validateForm(form: typeof EMPTY_FORM): FieldErrors {
  const errors: FieldErrors = {};
  if (!form.name.trim()) errors.name = 'Il nome è obbligatorio.';
  else if (form.name.trim().length > 32) errors.name = 'Il nome non può superare 32 caratteri.';
  if (!form.categoryId) errors.categoryId = 'Seleziona una categoria.';
  if (form.price.trim() === '') errors.price = 'Il prezzo è obbligatorio.';
  else if (!Number.isFinite(Number(form.price)) || Number(form.price) < 0) { ... }
  // ... e cosi' via, piu' lo stato 'touched' per ogni campo
}
```

- Non è codice sbagliato: è codice **ripetitivo**, e ce ne sarà uno per ogni form che aggiungete.
- I valori sono tutti stringhe e vanno riconvertiti a mano in numeri prima dell'invio.
- Le regole stanno solo qui: non si possono riusare nei test né condividere con il backend.

Lo stesso form con zod

```
npm install react-hook-form zod @hookform/resolvers
```

```
export const productSchema = z.object({
  name: z.string().trim().min(1, 'Il nome è obbligatorio').max(32, 'Massimo 32 caratteri'),
  categoryId: z.coerce.number().int().positive('Seleziona una categoria'),
  brand: z.string().trim().max(50, 'Massimo 50 caratteri').default(''),
  description: z.string().trim().max(500, 'Massimo 500 caratteri').default(''),
  price: z.coerce.number().min(0, 'Il prezzo deve essere maggiore o uguale a 0'),
  stock: z.coerce.number().int().min(0, 'Deve essere un intero maggiore o uguale a 0').default(0),
  imageUrl: z.string().nullable().default(null),
});
export type ProductFormValues = z.infer<typeof productSchema>;   // il tipo nasce dallo schema
```

- `z.coerce` converte la stringa dell'`<input>` in numero *prima* di validare: sparisce la conversione manuale.
- Uno schema, tre usi: validazione del form, tipo TypeScript, e (se volete) controllo delle risposte dell'API.

react-hook-form nel dialog

```
const form = useForm<ProductFormValues>({
  resolver: zodResolver(productSchema),
  defaultValues: product ? toFormValues(product) : EMPTY_VALUES,
});
const { register, handleSubmit, reset, setError, formState: { errors, isSubmitting } } = form;

useEffect(() => { reset(product ? toFormValues(product) : EMPTY_VALUES); }, [product, reset]);

<form onSubmit={handleSubmit(onSubmit)} noValidate className="space-y-4">
  <Label htmlFor="p-name">Nome *</Label>
  <Input id="p-name" {...register('name')} aria-invalid={!!errors.name}
         aria-describedby={errors.name ? 'p-name-error' : undefined} />
  {errors.name && <p id="p-name-error" role="alert" className="text-sm text-destructive">
    {errors.name.message}</p>}
  <Button type="submit" disabled={isSubmitting}>Salva</Button>
</form>
```

Gli attributi di accessibilità (`aria-invalid`, `aria-describedby`, `role="alert"`) ci sono già nel progetto: li teniamo, cambia solo chi produce l'errore.

Gli errori del server

```
async function onSubmit(values: ProductFormValues) {
  try {
    await save(product?.id, values);        // il save di useSaveProduct
    onOpenChange(false);
  } catch (err) {
    // il backend risponde 400 { error: "Nessuna categoria con id 99" }
    if (axios.isAxiosError(err) && err.response?.status === 400) {
      const message = err.response.data?.error ?? 'Dati non validi';
      setError(message.toLowerCase().includes('categoria') ? 'categoryId' : 'root', { message });
    } else toast.error(getErrorMessage(err));
  }
}
```

- La validazione lato client migliora l'esperienza; quella lato server è l'unica che conta per la sicurezza. Entrambe servono.
- Il backend oggi restituisce un solo messaggio: se volete errori per campo, è una piccola modifica a `routes/products.ts`.

Il campo immagine: un caso da discutere

```
// Oggi: il file viene letto e trasformato in data URL base64, salvato nel DB
const reader = new FileReader();
reader.onload = () => setForm(c => ({ ...c, imageUrl: reader.result as string }));
reader.readAsDataURL(file);        // "..."
```

- Funziona senza toccare il backend, ed è il motivo per cui è stato fatto così.
- Però: un'immagine da 5 MB diventa circa **6,7 MB** di testo, entra nel database, e viaggia *dentro* ogni risposta di `/api/products`. Il catalogo con venti articoli diventa pesantissimo.
- La via corretta: `POST /api/products/:id/image` in `multipart/form-data`, il file su disco o su un bucket, nel database solo l'URL.
- Lo rifaremo nel modulo 3, quando la foto arriverà dalla fotocamera del telefono.

Che tipo di stato è?

Tipo	Esempi in minicommerce	Dove vive
Server state	articoli, categorie, **carrello**, ordine	hook + `lib/api.ts`
URL state	ricerca, categoria, prezzi, ordinamento	React Router
Form state	campi del dialog, errori	react-hook-form
UI locale	dialog aperto, articolo in modifica	`useState`
UI globale	tema chiaro/scuro, toast	Context o Zustand
Sessione	utente loggato e suo ruolo (`isAdmin`)	Context (`AuthProvider`)

L'errore più comune

Mettere i dati del server in uno store globale e reinventare a mano cache, invalidazione e sincronizzazione. Il carrello di minicommerce è il caso da ricordare: vive nel database, legato all'utente, e si legge con `GET /api/cart`.

Tema chiaro/scuro: il progetto è già pronto

```
// index.css del progetto: la variante dark c'e' gia'
@custom-variant dark (&:is(.dark *));
```

```
// next-themes e' gia' fra le dipendenze: manca solo usarlo
import { ThemeProvider, useTheme } from 'next-themes';

<ThemeProvider attribute="class" defaultTheme="system" enableSystem>
  <App />
</ThemeProvider>

function ThemeToggle() {
  const { theme, setTheme } = useTheme();
  return (
    <Button variant="ghost" size="icon" aria-label="Cambia tema"
            onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>
      {theme === 'dark' ? <Sun /> : <Moon />}
    </Button>);
}
```

`attribute="class"` mette `class="dark"` sulla radice: è ciò che la variante CSS si aspetta.

Context, Zustand, Redux

- **Context** (quello che usa il progetto): perfetto per valori che cambiano raramente, come l'utente loggato. Ogni consumatore si ri-renderizza a ogni cambiamento.
- **Zustand**: store minimale con selettori, utile per UI globale che cambia spesso. In minicommerce non serve: il candidato naturale sarebbe il carrello, ma il carrello **non è stato del client**.
- Vive sul server, legato all'utente: si legge con `fetchCart` e si scrive con `addCartItem`, `updateCartItem`, `removeCartItem`, `clearCart`. Uno store client sarebbe una seconda copia da tenere allineata.
- **Redux Toolkit**: ancora molto diffuso in azienda, utile per flussi complessi. Non lo useremo, ma dovete riconoscerlo.

Come fa l'header a sapere quanti articoli ci sono

Dopo ogni scrittura, `lib/api.ts` emette l'evento `minicommerce:cart-updated`; `Header.tsx` lo ascolta e richiama `fetchCart`. Nessuno store globale: una rilettura dalla fonte di verità, che è il server.

Il problema che Tailwind risolve

```
/* CSS "classico": tre domande a ogni componente nuovo */
.product-card__footer--highlighted { ... }   /* 1. come lo chiamo? */
.card .footer span { color: red; }           /* 2. chi vince? specificita' */
/* 3. questa regola serve ancora a qualcuno? (nessuno lo sa: resta li' per anni) */
```

```
<!-- Tailwind: le stesse decisioni, prese dove servono -->
<div className="flex items-center justify-between gap-2 border-t pt-3">
```

- Ogni classe fa **una** cosa sola e non ha specificità da contendere.
- Niente nomi da inventare, niente file da cercare, niente CSS morto: il build include solo le classi che compaiono davvero nel codice.
- Il prezzo: righe di markup lunghe. Si paga estraendo *componenti*, non scrivendo CSS.

Le classi che userete davvero

Gruppo	Esempi	A cosa serve
Layout	`flex`, `grid`, `grid-cols-3`, `hidden`	disporre gli elementi
Allineamento	`items-center`, `justify-between`	nel contenitore flex o grid
Spaziatura	`gap-4`, `p-4`, `px-3`, `mt-2`	scala a passi di 0,25 rem: $4 = 1$ rem
Dimensioni	`w-full`, `max-w-6xl`, `size-4`	`size-4` è larghezza + altezza
Tipografia	`text-sm`, `font-medium`, `truncate`	`line-clamp-3` taglia a tre righe
Colori	`bg-background`, `text-muted-foreground`	nomi *semantici*, non `gray-500`
Bordi	`rounded-lg`, `border`, `divide-y`	`divide-y` riga fra i figli

Sono le stesse che trovate in `CatalogPage`: `mx-auto max-w-6xl space-y-6 px-4 py-8`. Leggetele come una frase: centrata, larga al massimo 72 rem, con spazio fra i blocchi.

Prefissi: responsive, stati, tema

```
<!-- schermo: senza prefisso = telefono, poi si allarga -->
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3">

<!-- stato dell'elemento -->
<button className="bg-primary hover:bg-primary/80 focus-visible:ring-3 disabled:opacity-50">

<!-- tema scuro (nel progetto: classe "dark" sulla radice) -->
<div className="bg-white dark:bg-zinc-900">

<!-- stato del genitore o di un fratello -->
<div className="group"> <span className="opacity-0 group-hover:opacity-100">Modifica</span> </div>
<input className="peer" /> <p className="hidden peer-invalid:block">Valore non valido</p>
```

- I prefissi si combinano: `lg:hover:bg-muted`, `dark:aria-invalid:border-destructive`.
- `bg-primary/80` è l'opacità dello sfondo: niente `rgba` da calcolare a mano.

Tailwind v4: la configurazione sta nel CSS

```
/* src/index.css del progetto, in ordine */
@import "tailwindcss";
@import "shadcn/tailwind.css";
@custom-variant dark (&:is(.dark *));   /* da qui nasce il prefisso dark: */

@theme inline {                          /* i nomi delle classi */
  --color-background: var(--background);
  --radius-lg: var(--radius);
}
:root {                                  /* i valori, in tema chiaro */
  --background: oklch(1 0 0);
  --destructive: oklch(0.577 0.245 27.325);
}
```

- Niente `tailwind.config.js`: in v4 il tema è CSS. `@theme` crea le classi, `:root` dà i valori.
- `-color-primary` genera `bg-primary`, `text-primary`, `border-primary`.
- Colori in **oklch**: cambiare `-primary` ricolora tutta l'applicazione.

Riuso: componenti, cn, varianti

```
// 1. cn(): unisce le classi e risolve i conflitti (l'ultima vince)
<div className={cn('rounded-lg border p-4', isLate && 'border-destructive', className)} />

// 2. cva(): un componente, piu' varianti. E' cosi' che e' fatto Button nel progetto
const buttonVariants = cva(
  'inline-flex items-center justify-center rounded-lg text-sm font-medium disabled:opacity-50',
  { variants: {
      variant: { default: 'bg-primary text-primary-foreground hover:bg-primary/80',
                 outline: 'border-border bg-background hover:bg-muted',
                 destructive: 'bg-destructive text-white' },
      size:    { default: 'h-9 px-4', sm: 'h-8 px-3', icon: 'size-9' } },
    defaultVariants: { variant: 'default', size: 'default' } },
);
<Button variant="destructive" size="icon" />
```

Senza `cn` una prop `className` passata da fuori finirebbe *accanto* a quella interna, e vincerebbe la più specifica a caso.

Tailwind: errori comuni

Da evitare

- Costruire nomi di classe a pezzi: `` `text-${colore}-500` ``. Il build non li trova e spariscono.
- Usare `gray-500` invece dei colori semantici: in tema scuro resta grigio sbagliato.
- Valori arbitrari ovunque (`w-[237px]`): se capita spesso, è un token che manca.
- Reinventare `@apply` per tornare al CSS di prima.

Da fare

- Estrarre un componente quando la stessa stringa compare tre volte.
- Installare l'estensione **Tailwind CSS IntelliSense**: completamento, anteprima del colore, ordinamento.
- Leggere `components/ui/`: è Tailwind scritto bene, ed è già nel vostro repository.
- Nel dubbio sul nome di una classe: `https://tailwindcss.com/docs`, la ricerca è ottima.

shadcn/ui: componenti copiati, non installati

```
npx shadcn@latest add checkbox pagination skeleton alert
# scrive i file in src/components/ui/: da quel momento sono VOSTRI
```

- Non è una libreria di componenti: è un **generatore**. Il codice finisce nel vostro repository e lo potete modificare.
- Sotto c'è **Radix UI**: dialog, select e menu accessibili (focus, tastiera, Esc, ARIA) già fatti bene.
- `components.json` dice dove mettere i file e quale stile usare: nel progetto, stile `radix-nova` e icone `lucide`.
- Vantaggio: zero attesa per una patch upstream. Svantaggio: gli aggiornamenti non arrivano da soli.

Da provare in laboratorio

Aggiungete `skeleton` e usatelo per lo stato di caricamento del catalogo, al posto della scritta "Caricamento...".

Il dialog: perché non va fatto a mano

```
<Dialog open={open} onOpenChange={onOpenChange}>
  <DialogContent className="sm:max-w-lg">
    <DialogHeader>
      <DialogTitle>{product ? 'Modifica articolo' : 'Nuovo articolo'}</DialogTitle>
      <DialogDescription>Compila i campi e salva.</DialogDescription>
    </DialogHeader>
    ...
  </DialogContent>
</Dialog>
```

- Radix vi dà gratis: focus spostato dentro all'apertura e restituito alla chiusura, chiusura con Esc, sfondo che blocca il resto, `aria-modal`, lettura del titolo da parte dello screen reader.
- Un `<div>` con `position: fixed` non fa nulla di tutto questo.
- Alternativa senza librerie: l'elemento HTML `<dialog>` con `showModal()`, che fa quasi altrettanto.

Da sostituire: window.confirm

```
// AdminPage oggi
const handleDelete = async (product: Product) => {
  if (!window.confirm(‘Eliminare "${product.name}" dal catalogo?‘)) return;
  ...
};
```

- `window.confirm` **blocca** il thread, non si può stilizzare, ha un aspetto diverso su ogni browser, e in Electron o in una webview mobile stona del tutto.
- Sostituzione: `AlertDialog` di shadcn/ui, con il nome dell'articolo nel testo e il pulsante distruttivo in rosso.
- Regola generale: `alert`, `confirm` e `prompt` non entrano in un'applicazione finita.

Laboratorio 1c

Laboratorio: Form, dialog, tema

1. Riscrivere `ProductFormDialog` con react-hook-form + zod, mantenendo tutti gli attributi ARIA esistenti.
2. Mappare l'errore 400 della categoria sul campo `categoryId`.
3. Sostituire `window.confirm` con `AlertDialog`; aggiungere `Skeleton` al caricamento del catalogo.
4. Attivare il tema chiaro/scuro con `next-themes` e un pulsante nell'header.

Domande di verifica

Perché lo schema zod genera anche il tipo? Cosa fa `z.coerce`? Quando un errore del server va su un campo e quando su `root`? Cosa vi dà Radix che un `<div>` non vi dà?

Debounce riutilizzabile

```
// hooks/useDebounced.ts  -- la stessa idea del timer in CatalogPage, ma riusabile
export function useDebounced<T>(value: T, ms = 250): T {
  const [v, setV] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setV(value), ms);
    return () => clearTimeout(id);
  }, [value, ms]);
  return v;
}

// uso: i filtri cambiano a ogni tasto, la richiesta parte dopo 250 ms di quiete
const filters = filtersFromParams(params);
const { data } = useProducts(useDebounced(filters));
```

- Senza debounce: una richiesta per ogni tasto premuto. Con 50 articoli non si nota; con 50.000 sì, e il server se ne accorge prima di voi.
- Attenzione a *cosa* si fa il debounce: l'input resta immediato, è la *query* a essere ritardata.

Paginazione: il catalogo non può crescere all'infinito

```
// Il backend oggi restituisce TUTTI gli articoli che passano i filtri.
// Primo passo (lato server): aggiungere limit e offset in routes/products.ts
//   .limit(Number(limit) || 24).offset(Number(offset) || 0)
// e restituire { items, total } invece di un array.

const [page, setPage] = useState(0);
const { data, loading } = useProducts({ ...filters, limit: 24, offset: page * 24 });
<Pagination page={page} total={data.total} pageSize={24} onChange={setPage} />

// la pagina cambia -> cambiano i filtri -> l'hook rifa' la richiesta, da solo
```

- In alternativa, **scorrimento infinito**: più adatto a un catalogo da telefono (modulo 3). Si accumulano le pagine invece di sostituirle.
- Per liste molto lunghe in area admin: **virtualizzazione** (per esempio TanStack Virtual), che disegna solo le righe visibili.

Modifica rapida della disponibilità

```
// In tabella admin: +/- sullo stock senza aprire il dialog
function StockCell({ product, onChanged }: StockCellProps) {
  const { stock, busy, cambia } = useStockOttimistico(product, onChanged);
  return (
    <div className="flex items-center justify-end gap-1">
      <Button variant="ghost" size="icon" aria-label={`Togli una unità da ${product.name}`}
              disabled={busy || stock === 0} onClick={() => cambia(-1)}>
        <Minus className="size-3" />
      </Button>
      <span className="w-8 text-center tabular-nums">{stock}</span>
      <Button variant="ghost" size="icon" aria-label={`Aggiungi una unità a ${product.name}`}
              disabled={busy} onClick={() => cambia(+1)}>
        <Plus className="size-3" />
      </Button>
    </div>
  );
}
```

Con l'aggiornamento ottimistico il numero cambia istantaneamente: è la funzione che il commesso userà cento volte al giorno.

Error Boundary

```
npm install react-error-boundary
```

```
<ErrorBoundary
  FallbackComponent={({ error, resetErrorBoundary }) => (
    <section role="alert" className="mx-auto max-w-md py-16 text-center">
      <h2 className="text-lg font-semibold">Qualcosa è andato storto</h2>
      <p className="text-muted-foreground">{error.message}</p>
      <Button onClick={resetErrorBoundary}>Riprova</Button>
    </section>
  )}
>
  <Routes>...</Routes>
</ErrorBoundary>
```

- Senza boundary, un errore durante il rendering **smonta tutta l'app**: schermo bianco, header compreso.
- Provate: in `ProductCard` scrivete `product.category.toUpperCase()` e fate restituire al backend una categoria nulla.

Code splitting

```
import { lazy, Suspense } from 'react';
const AdminPage = lazy(() => import('@/pages/AdminPage'));

<Route element={<ProtectedLayout />}>
  <Route path="/admin" element={
    <Suspense fallback={<PageSkeleton />}><AdminPage /></Suspense>
  } />
</Route>
```

- Il cliente che apre il catalogo non ha motivo di scaricare l'area admin, il dialog, la tabella e la validazione.
- Vite crea un file JS per ogni `import()` dinamico. `npm run build` stampa le dimensioni di ciascuno.
- Regola: le pagine si dividono sempre; i componenti solo se pesano davvero.

Memoizzazione: quando serve davvero

- React ri-renderizza quando cambiano props o stato, o quando si ri-renderizza il padre. Nel 99% dei casi è **abbastanza veloce**.
- Misurate prima di ottimizzare: React DevTools, scheda Profiler, registrate mentre digitate nella ricerca.
- `useMemo` per calcoli costosi veri (raggruppare o ordinare migliaia di righe lato client).
- `useCallback` + `React.memo` quando passate funzioni a liste lunghe di figli memoizzati.
- **React Compiler**: memoizza da solo, si attiva con un plugin di Babel in `vite.config.ts`. Provatelo nel progetto finale e confrontate il profilo.

Il proxy non esiste in produzione

```
# .env.development
VITE_API_URL=/api                                    # il proxy di Vite fa il resto
# .env.production
VITE_API_URL=https://api.minicommerce.example/api
```

```
// lib/api.ts
export const api = axios.create({ baseURL: import.meta.env.VITE_API_URL ?? '/api' });
```

- Solo le variabili con prefisso `VITE_` finiscono nel bundle. Tutto il resto resta fuori: **nel client non ci sono segreti**.
- Con un'origine diversa dal backend servono le intestazioni **CORS**: il progetto ha già `app.use(cors())`, che in produzione va ristretto al vostro dominio (modulo 6).
- Questa stessa riga tornerà identica in Electron, Tauri e Capacitor: lì il frontend non è *mai* sulla stessa origine dell'API.

Build e deploy di una SPA

```
npm run build      # tsc -b && vite build  ->  dist/
npm run preview    # serve dist/ in locale, senza proxy: come in produzione
```

```
# nginx: qualsiasi percorso -> index.html, altrimenti /admin ricaricata da' 404
location / {
  try_files $uri $uri/ /index.html;
}
location /assets/ { add_header Cache-Control "public, max-age=31536000, immutable"; }
```

- `dist/` è una cartella statica: Netlify, Vercel, Cloudflare Pages, GitHub Pages o un'immagine `nginx:alpine` da 20 MB.
- Il backend va ospitato a parte (un piccolo server Node con il file SQLite accanto).
- Nel modulo 7 automatizziamo tutto con GitHub Actions.

Laboratorio 1d

Laboratorio: Rifinitura, build, pubblicazione

1. `useDebounced` riusabile; paginazione del catalogo (anche lato backend: `limit` e `offset`).
2. Pulsanti +/- sullo stock in area admin, con aggiornamento ottimistico.
3. Error boundary attorno alle route; `AdminPage` in lazy loading; controllare le dimensioni del bundle.
4. `VITE_API_URL`, `npm run build`, `npm run preview`; deploy del frontend su Netlify o Cloudflare Pages con la regola di fallback.

Domande di verifica

Perché `npm run preview` trova errori che `npm run dev` non mostra? Cosa succede aprendo direttamente `/admin` senza la regola di nginx? Cosa finisce nel bundle se scrivo `VITE_JWT_SECRET`?

Checklist: minicommerce Web è pronta?

- ☐ Login, logout, sessione persistente, 401 gestito
- ☐ Catalogo: filtri nell'URL, debounce, paginazione
- ☐ Admin: CRUD completo, stock rapido, conferme accessibili
- ☐ I quattro stati in ogni schermata
- ☐ Carrello e ordine finto provati da cliente
- ☐ Route protette per ruolo e pagina 404
- ☐ Tema chiaro/scuro
- ☐ Error boundary e code splitting
- ☐ Build pubblicata e raggiungibile
- ☐ README aggiornato con le vostre aggiunte
- ☐ Nessun segreto nel repository

Da qui in poi

Questa applicazione, *così com'è*, diventerà un programma per il desktop del negozio nel prossimo modulo. Tenete il codice pulito: lo riusiamo tutto.

Sommario del modulo

3. Dal web al desktop: Electron e Tauri
3.1 Perché un'app desktop
3.2 Electron
3.3 Tauri
3.4 Confronto e conclusioni

Il caso d'uso: il PC del negozio

Chi usa l'area admin non è un cliente: è chi sta dietro il bancone, tutto il giorno, sullo stesso computer.

- Icona sul desktop e avvio all'accensione.
- **Icona nella barra di sistema** con il numero di articoli esauriti.
- Notifiche native quando una scorta scende sotto la soglia.
- Scorciatoia globale (`Ctrl+Shift+N`: nuovo articolo) anche a finestra ridotta.
- **Esportare il catalogo** in CSV, con una vera finestra di salvataggio.
- Stampare le etichette dei prezzi.
- Lettore di codici a barre USB: si comporta da tastiera, ma serve una finestra con il fuoco.
- Nessuna barra degli indirizzi: non si naviga altrove e non si chiude la scheda per sbaglio.

Il patto

Riusiamo **tutto** il codice React del modulo 1: aggiungiamo un "guscio" nativo e un canale per parlarci.

Le due strade

Electron (2013, GitHub)

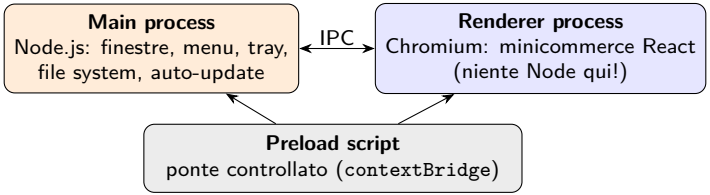
- Impacchetta **Chromium + Node.js** con l'app.
- Stesso browser ovunque: ciò che funziona su Windows funziona su Linux.
- VS Code, Slack, Discord, Figma, Obsidian, 1Password.
- Costo: 150–250 MB di installazione, memoria da browser.

Tauri (2020, v2 dal 2024)

- Usa la **webview di sistema** + un backend in **Rust**.
- Eseguibili da 5–15 MB, meno memoria, avvio rapido.
- Webview diverse per OS: piccole differenze da testare.
- Modello di sicurezza a permessi espliciti; v2 gira anche su Android/iOS.

Le facciamo entrambe: Electron perché lo troverete nelle aziende, Tauri perché è la direzione attuale. Il laboratorio principale è su Tauri.

Architettura di Electron



- **Un** main process, **un** renderer per finestra. Parlano solo via **IPC** (messaggi).
- Il renderer è una pagina web: se avesse Node, un XSS nella descrizione di un articolo diventerebbe un accesso completo al PC del negozio.
- Il preload espone al renderer *solo* le funzioni che decidiamo noi.

Setup: una cartella, nessun generatore

```
# dalla radice del repository, una volta sola
node scripts/setup-electron.mjs
```

```
desktop/                  # il guscio nativo, accanto a frontend/ e backend/
  package.json            # electron, electron-builder, esbuild
  src/main.ts             # processo principale
  src/preload.ts          # il ponte
  scripts/dev.mjs         # esbuild -> Vite -> Electron, in quest'ordine
  electron-builder.yml
frontend/                 # NON duplicato: resta uno solo
```

- Esistono generatori (`electron-vite`, Electron Forge) che creano un progetto nuovo in cui poi *copiare* il frontend. Qui no: il frontend resta uno solo e il guscio gli sta accanto.
- Due file di TypeScript compilati da **esbuild** in un comando: niente da configurare, e main e preload si leggono per intero.
- Lo script fa tutto: dipendenze, file del guscio, patch al frontend. Lo vediamo passo per passo nelle prossime slide.

Il main process: creare la finestra

```
import { app, BrowserWindow, shell } from 'electron';
import { join } from 'node:path';

function createWindow() {
  const win = new BrowserWindow({
    width: 1280, height: 800, minWidth: 1024,
    show: false,
    webPreferences: {
      preload: join(__dirname, '../preload/index.js'),
      contextIsolation: true, nodeIntegration: false, sandbox: true,  // il renderer NON vede Node
    },
  });
  win.once('ready-to-show', () => win.show());   // niente flash bianco
  win.webContents.setWindowOpenHandler(({ url }) => { shell.openExternal(url); return { action:
    'deny' }; });
  if (process.env.ELECTRON_RENDERER_URL) win.loadURL(process.env.ELECTRON_RENDERER_URL);  // dev
  else win.loadFile(join(__dirname, '../renderer/index.html'));                           // prod
}
app.whenReady().then(createWindow);
app.on('window-all-closed', () => { if (process.platform !== 'darwin') app.quit(); });
```

Le tre sorprese

1. **Il routing**: in produzione la pagina è caricata da `file://.../index.html`. Un `BrowserRouter` che naviga su `/admin` cerca *un file* che non esiste.
2. **L'origine**: con `file://` il browser invia `Origin: null` e il proxy di Vite non c'è più. L'API va chiamata al suo indirizzo assoluto, e il backend deve accettare quell'origine.
3. **Le immagini**: i data URL base64 degli articoli funzionano anche qui; percorsi relativi come `/uploads/mouse.png` no.

```
// nella SPA: scegliere il router in base all'ambiente
const createRouter = import.meta.env.VITE_ROUTER === 'hash' ? createHashRouter :
    createBrowserRouter;
// e nel .env del progetto desktop:
// VITE_ROUTER=hash
// VITE_API_URL=http://localhost:3001/api
```

In alternativa all'hash router: registrare un protocollo personalizzato (`app://`) che serve sempre `index.html` e dà un'origine stabile.

Preload e contextBridge: il ponte sicuro

```ts
// src/preload/index.ts
import { contextBridge, ipcRenderer } from 'electron';

const api = {
  notify: (title: string, body: string) => ipcRenderer.invoke('notify', { title, body }),
  exportCatalogo: (csv: string) => ipcRenderer.invoke('export-csv', csv),
  setEsauriti: (count: number) => ipcRenderer.send('badge', count),
  onNuovoArticolo: (cb: () => void) => { ipcRenderer.on('shortcut:nuovo-articolo', cb); },
  platform: process.platform,
};
contextBridge.exposeInMainWorld('minicommerce', api);
export type MinicommerceApi = typeof api;
```

```ts
// src/renderer/src/electron.d.ts
declare global { interface Window { minicommerce?: import('../../preload').MinicommerceApi } }
```

`window.minicommerce` è *opzionale*: lo stesso codice React gira anche nel browser, dove non esiste.

Main: gestire le richieste IPC

```
import { ipcMain, Notification, dialog, app, BrowserWindow } from 'electron';
import { writeFile } from 'node:fs/promises';

ipcMain.handle('notify', (_e, { title, body }) => { new Notification({ title, body }).show(); });

ipcMain.handle('export-csv', async (e, csv: string) => {
  const win = BrowserWindow.fromWebContents(e.sender)!;
  const { filePath, canceled } = await dialog.showSaveDialog(win, {
    defaultPath: `catalogo-${new Date().toISOString().slice(0, 10)}.csv`,
    filters: [{ name: 'CSV', extensions: ['csv'] }],
  });
  if (canceled || !filePath) return false;
  const BOM = String.fromCharCode(0xFEFF);          // Excel apre bene gli accenti
  await writeFile(filePath, BOM + csv, 'utf8');
  return true;
});

ipcMain.on('badge', (_e, count: number) => app.setBadgeCount(count));
```

Il main process è Node: tutto ciò che arriva dal renderer è **input non fidato** e va validato qui.

Dal lato React: usare il ponte se c'è

```
// hooks/usePiattaforma.ts
export function usePiattaforma() {
  const bridge = window.minicommerce;
  return {
    isDesktop: !!bridge,
    notify: (t: string, b: string) => (bridge ? bridge.notify(t, b) : notificaWeb(t, b)),
    exportCatalogo: (csv: string) =>
      bridge ? bridge.exportCatalogo(csv) : scaricaBlob(csv, 'catalogo.csv'),
  };
}

// AdminPage.tsx
const { exportCatalogo } = usePiattaforma();
const { data: products = [] } = useProducts({});
<Button variant="outline" onClick={() => exportCatalogo(toCsv(products))}>
  <Download className="size-4" /> Esporta catalogo
</Button>
```

Il fallback web (`scaricaBlob` crea un `<a download>`) mantiene **un solo codice** per browser e desktop.

Tray con il numero di esauriti

```
import { Tray, Menu, nativeImage } from 'electron';

tray = new Tray(nativeImage.createFromPath(join(__dirname, '../../resources/tray.png')));
tray.setContextMenu(Menu.buildFromTemplate([
  { label: 'Apri minicommerce', click: () => win.show() },
  { label: 'Nuovo articolo', accelerator: 'CmdOrCtrl+Shift+N',
    click: () => win.webContents.send('shortcut:nuovo-articolo') },
  { type: 'separator' },
  { role: 'quit' },
]));
tray.setToolTip('minicommerce');

// chiudere = nascondere nella tray, non uscire
win.on('close', (e) => { if (!app.isQuitting) { e.preventDefault(); win.hide(); } });
```

```
// nel renderer: a ogni ricarica dell'elenco articoli
useEffect(() => {
  window.minicommerce?.setEsauriti(products.filter(p => p.stock === 0).length);
}, [products]);
```

Il token: dove finisce su desktop

```
import { safeStorage } from 'electron';
import { readFile, writeFile } from 'node:fs/promises';
const file = join(app.getPath('userData'), 'session.bin');

ipcMain.handle('session:save', async (_e, token: string) => {
  await writeFile(file, safeStorage.encryptString(token));   // cifrato con il keychain dell'OS
});
ipcMain.handle('session:load', async () => {
  try { return safeStorage.decryptString(await readFile(file)); } catch { return null; }
});
```

- `localStorage` funziona anche in Electron, ma è un file in chiaro dentro il profilo utente. Sul PC condiviso del negozio non è il massimo.
- `safeStorage` usa DPAPI (Windows), Keychain (macOS), libsecret (Linux).
- Conseguenza sul codice: la lettura del token diventa **asincrona**, quindi l'interceptor di axios va reso `async` e l'`AuthProvider` parte da uno stato "sto caricando la sessione".

Pacchettizzare e distribuire

```
cd desktop
npm run package      # build del frontend + esbuild + electron-builder
                     # -> desktop/release/ (su Windows: NSIS, ~110 MB)
```

```
// electron-builder.yml (estratto)
appId: it.its.minicommerce
productName: minicommerce
win:   { target: ["nsis"] }
mac:   { target: ["dmg"], hardenedRuntime: true, notarize: true }
linux: { target: ["AppImage", "deb"], category: "Office" }
publish: { provider: "github" }
```

- **Firma del codice**: senza certificato Windows mostra "editore sconosciuto"; senza notarizzazione macOS blocca l'app.
- **Auto-update**: `electron-updater` scarica dalle release GitHub e aggiorna al riavvio. Per il negozio è essenziale: nessuno andrà a reinstallare a mano.

Checklist di sicurezza per Electron

- `contextIsolation: true`, `nodeIntegration: false`, `sandbox: true`.
- Esporre via `contextBridge` **funzioni specifiche**, mai `ipcRenderer` intero.
- Validare *nel main* i dati che arrivano dal renderer: il contenuto del CSV, il percorso, le dimensioni.
- `setWindowOpenHandler`: i link esterni si aprono nel browser, non in una finestra Electron senza barra.
- Content Security Policy nell'`index.html`, con attenzione ai data URL delle immagini degli articoli.
- Aggiornare Electron: ogni versione porta le patch di sicurezza di Chromium.

Riferimento: `https://www.electronjs.org/docs/latest/tutorial/security`

Laboratorio 2a

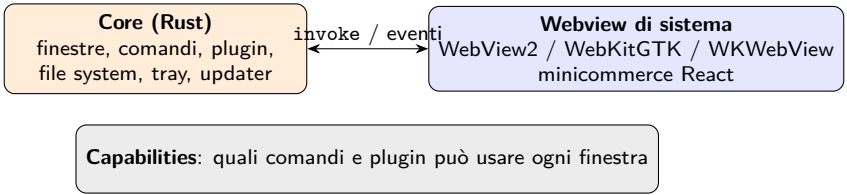
Laboratorio: minicommerce Desktop con Electron

1. Lanciare `node scripts/setup-electron.mjs` e far partire l'app: `cd desktop && npm run dev`.
2. Leggere `desktop/src/main.ts` e `preload.ts` e spiegare a voce ogni opzione di sicurezza.
3. Aggiungere al ponte una funzione vostra: scorciatoia globale "nuovo articolo" oppure contatore degli esauriti nella tray.
4. `npm run package` e installare il risultato. Confrontare il vostro lavoro con il branch `desktop/electron`.

Domande di verifica

Perché il renderer non deve avere accesso a Node? Che differenza c'è fra `ipcRenderer.invoke` e `send`? Perché `BrowserRouter` non funziona con `file://`?

Architettura di Tauri 2



- Il backend del guscio è un programma Rust: sicuro per costruzione, veloce, senza Node.
- Non conoscete Rust? Per il 90% dei casi bastano i **plugin ufficiali** e comandi di cinque righe.
- Le webview non sono Chromium: su Linux (WebKitGTK) qualche funzione CSS recente può mancare. Tailwind v4 usa CSS moderno: provate presto su Linux.

Setup sul frontend esistente

```
# prerequisiti: Rust (rustup); su Windows i Build Tools C++ e WebView2 (gia' in Win 11)
# su Linux: libwebkit2gtk-4.1-dev, libssl-dev, ...

# dalla radice del repository, una volta sola
node scripts/setup-tauri.mjs

# dentro, in sostanza, fa questo nel frontend:
npm install -D @tauri-apps/cli@^2 && npm install @tauri-apps/api@^2
npx tauri init --ci          # dev URL :5173, dist ../dist
npx tauri add dialog fs notification
npx tauri icon src/assets/hero.png
```

```
frontend/src-tauri/   Cargo.toml, tauri.conf.json, capabilities/, src/lib.rs, icons/
```

Notate la differenza con Electron: qui **non si crea un nuovo progetto**, si aggiunge una cartella a quello che avete già.

tauri.conf.json e capabilities

```
{
  "productName": "minicommerce",
  "identifier": "it.its.minicommerce",
  "build": { "devUrl": "http://localhost:5173", "frontendDist": "../dist",
             "beforeDevCommand": "npm run dev", "beforeBuildCommand": "npm run build:desktop" },
  "app": { "windows": [{ "title": "minicommerce", "width": 1280, "height": 800 }],
           "security": { "csp": "default-src 'self'; img-src 'self' data:; connect-src http://localhost:3001" } },
  "bundle": { "active": true, "targets": "all", "icon": ["icons/icon.png"] }
}
```

```
// capabilities/default.json: cosa puo' fare la finestra principale
{ "identifier": "default", "windows": ["main"],
  "permissions": ["core:default", "notification:default", "dialog:default",
                  "fs:allow-write-text-file", "store:default", "global-shortcut:default"] }
```

In Tauri l'app **non può** fare nulla che non sia elencato qui. È il contrario di Electron, dove tutto è permesso finché non lo chiudete voi.

Comandi Rust e invoke

```rust
// src-tauri/src/lib.rs
#[tauri::command]
fn esporta_csv(path: String, csv: String) -> Result<(), String> {
    std::fs::write(&path, csv).map_err(|e| e.to_string())
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_notification::init())
        .plugin(tauri_plugin_dialog::init())
        .invoke_handler(tauri::generate_handler![esporta_csv])
        .run(tauri::generate_context!())
        .expect("errore avvio minicommerce");
}
```

```typescript
import { invoke } from '@tauri-apps/api/core';
await invoke('esporta_csv', { path, csv });   // Promise<void>, errori come eccezioni
```

I plugin: quasi tutto senza scrivere Rust

```
npx tauri add dialog fs notification        # crate Rust + pacchetto npm + permesso
```

```
import { sendNotification, isPermissionGranted, requestPermission } from '@tauri-apps/plugin-notification';
import { save } from '@tauri-apps/plugin-dialog';
import { writeTextFile } from '@tauri-apps/plugin-fs';
import { register } from '@tauri-apps/plugin-global-shortcut';
import { Store } from '@tauri-apps/plugin-store';

export async function exportCatalogo(csv: string) {
  const path = await save({ defaultPath: 'catalogo.csv', filters: [{ name: 'CSV', extensions: ['csv'] }] });
  if (path) await writeTextFile(path, csv);
}
export async function avvisaScorte(nomi: string[]) {
  if (!(await isPermissionGranted())) await requestPermission();
  sendNotification({ title: 'Scorte in esaurimento', body: nomi.join(', ') });
}
await register('CommandOrControl+Shift+N', () => apriNuovoArticolo());
const store = await Store.load('session.json');    // per cifrare: plugin stronghold
```

Una sola interfaccia, tre implementazioni

```ts
// lib/piattaforma.ts
export const piattaforma =
  '__TAURI_INTERNALS__' in window ? 'tauri' :
  window.minicommerce              ? 'electron' :
  'web';

export interface Piattaforma {
  notify(title: string, body: string): Promise<void>;
  exportCatalogo(csv: string): Promise<void>;
  onNuovoArticolo(cb: () => void): () => void;
}

const impl: Piattaforma =
  piattaforma === 'tauri' ? tauriImpl : piattaforma === 'electron' ? electronImpl : webImpl;
export const usePiattaforma = () => impl;
```

- `CatalogPage`, `AdminPage` e il dialog non sanno dove stanno girando.
- È lo stesso schema che useremo per il mobile, dove si aggiungerà un quarto caso: Capacitor.

Tray, eventi, build

```rust
use tauri::{tray::TrayIconBuilder, menu::{Menu, MenuItem}, Manager};
// dentro .setup(|app| { ... })
let quit = MenuItem::with_id(app, "quit", "Esci", true, None::<&str>)?;
let menu = Menu::with_items(app, &[&quit])?;
TrayIconBuilder::new().menu(&menu).icon(app.default_window_icon().unwrap().clone())
    .on_menu_event(|app, e| if e.id() == "quit" { app.exit(0) })
    .build(app)?;
```

```
npm run desktop:build   # -> src-tauri/target/release/bundle/{msi,nsis,deb,appimage,dmg}
```

- Eventi dal core alla webview: `app.emit("scorte", 3)` e `listen('scorte', ...)` in TypeScript.
- Aggiornamenti: plugin `updater` con un manifest JSON firmato (chiave generata da `tauri signer`).
- Anche qui: firma su Windows e notarizzazione su macOS per una distribuzione seria.

Laboratorio 2b

Laboratorio: minicommerce Desktop con Tauri

1. Lanciare `node scripts/setup-tauri.mjs`; `cd frontend && npm run desktop:dev`.
2. Leggere `capabilities/default.json` e provare a usare un plugin *non* elencato: guardare l'errore.
3. Aggiungere un comando Rust vostro accanto a `conta_esauriti` e chiamarlo con `invoke`.
4. `npm run desktop:build` e confronto delle dimensioni con il pacchetto Electron.
5. Completare `lib/piattaforma.ts` con l'implementazione che manca.

Domande di verifica

Cosa succede se chiamo un plugin non elencato nelle capabilities? Perché l'app Tauri pesa 10 MB e quella Electron 200? Cosa non posso dare per scontato sulla webview di Linux?

Electron vs Tauri

	Electron	Tauri 2
Motore web	Chromium incluso	Webview dell'OS
Backend del guscio	Node.js	Rust (+ plugin)
Dimensione	150–250 MB	5–15 MB
Memoria	alta	media/bassa
Coerenza del rendering	identica ovunque	da testare per OS
Sicurezza	opt-in (checklist)	permessi espliciti
Ecosistema	enorme, maturo	in crescita rapida
Mobile	no	sì (Android/iOS)
Curva di apprendimento	solo JS	JS + un po' di Rust

- Nuovo progetto nel 2026: **Tauri**, salvo bisogno di moduli nativi Node o di un rendering identico garantito.
- Progetto esistente con team solo JavaScript e molte dipendenze Node: Electron resta ragionevole.

Cosa abbiamo cambiato nel frontend (e cosa no)

Cambiato

- Router: hash sul desktop
- `VITE_API_URL` assoluto
- Token in storage cifrato, lettura asincrona
- Un modulo `piattaforma.ts` con tre implementazioni
- Link esterni aperti fuori dall'app

Non toccato

- Tutte le pagine e i componenti
- Gli hook dei dati, il form, lo schema zod
- Tailwind, shadcn/ui, il tema
- `lib/api.ts` (a parte la base URL)
- Il backend

Lezione

Progettare **confini** (il client HTTP, lo storage, le capacità native) dietro interfacce è ciò che rende un'app portabile. Non è "roba da architetti": lo avete appena fatto in due laboratori.

I due branch di riferimento

```
git switch desktop/electron    # master, un commit di merge, i PDF delle slide
git switch desktop/tauri       # idem: il ramo aggiunge una cosa sola

git diff master..desktop/electron -- frontend/   # le patch condivise
git diff desktop/electron..desktop/tauri -- frontend/src/lib/piattaforma.ts
```

- Ogni branch aggiunge **una cosa sola**, in un commit di merge: il suo messaggio racconta tutto ciò che è stato fatto. Leggetelo, è la sintesi della lezione.
- Il secondo `diff` mette una accanto all'altra le due implementazioni della stessa interfaccia: stessa UI, due mondi nativi diversi.
- I due script sono idempotenti: potete rilanciarli senza rompere nulla.

Alternative da conoscere

- **Neutralinojs**, **Wails** (Go): stessa idea di Tauri con altri linguaggi nel guscio.
- **PWA installata** (prossimo modulo): per il PC del negozio può bastare, senza installer né store.
- **.NET MAUI Blazor Hybrid**, **Flutter desktop**: interfaccia non web, stesso codice su desktop e mobile.
- **Nativo puro** (WinUI, SwiftUI, GTK): modulo 5. Quando il feeling e le prestazioni contano più del riuso.

Sommario del modulo

4. Dal desktop al mobile: responsive, PWA, Capacitor
4.1 Il mobile è un altro mondo
4.2 Responsive design
4.3 Progressive Web App
4.4 Capacitor (e Ionic)
4.5 Confronto

Due utenti diversi, sullo stesso telefono

Il cliente: catalogo e carrello

- Arriva da un link o da una ricerca, apre la scheda articolo, aggiunge al carrello, invia l'ordine.
- Rete mobile, fretta, una mano sola.
- Deve caricare in fretta, scorrere bene e avere pulsanti che si prendono al primo colpo.

Il magazziniere: l'area admin

- Gira fra gli scaffali, aggiorna le scorte, fotografa gli articoli nuovi.
- Wi-Fi che cade in magazzino, telefono in una mano e scatola nell'altra.
- Ha bisogno di **fotocamera**, di funzionare **offline** e di bersagli grandi.

Tre livelli, in ordine crescente di "nativo"

1. Web responsive $\rightarrow$ 2. PWA installabile $\rightarrow$ 3. App Capacitor negli store. Li facciamo tutti e tre sulla stessa minicommerce. Il campo `isAdmin` separa i due mondi: il carrello è solo del cliente, a un admin l'API risponde 403.

Cosa cambia su un telefono

- **Schermo piccolo e verticale**: la tabella admin a sei colonne non entra.
- **Touch**: niente hover, dita larghe (bersaglio minimo 44 px), gesti.
- **Rete instabile**: 4G in strada, Wi-Fi che cade in magazzino.
- **Batteria e CPU**: il JavaScript pesante si sente.
- **Interruzioni**: telefonata, notifica, app in background.

- Tastiera virtuale che copre metà schermo, proprio sopra il form dell'articolo o i campi del pagamento.
- Aree "non sicure": notch, barra home, angoli arrotondati.
- Aspettative da app nativa: barra in basso, swipe indietro, splash.
- Due piattaforme con regole diverse (Android, iOS).

Le basi che non si possono sbagliare

```
<!-- index.html del progetto: manca viewport-fit, serve per notch e barra home -->
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
```

```
/* altezza reale dello schermo, barre del browser incluse */
.shell { min-height: 100dvh; padding-bottom: env(safe-area-inset-bottom); }

/* hover solo dove esiste un mouse: su touch l'effetto "resta attaccato" */
@media (hover: hover) { .card:hover { box-shadow: var(--shadow-2); } }

/* bersagli touch */
button, a { min-height: 44px; }
```

- In Tailwind: `min-h-dvh`, `pb-[env(safe-area-inset-bottom)]`; la variante `hover:` si limita con `@media (hover: hover)` nel CSS base.
- Gli `<input>` sotto i 16 px fanno zoomare Safari su iOS: i campi del form articolo e quelli del pagamento devono restare almeno a `text-base`.

Il percorso del cliente è già quasi a posto

```
<!-- CatalogPage, cosi' com'e': una colonna, due da 640px, tre da 1024px -->
<div className="grid grid-cols-1 gap-4 sm:grid-cols-2 lg:grid-cols-3">
<!-- CartPage: la riga diventa orizzontale solo da 640px in su -->
<Card className="sm:flex-row sm:items-center">
```

- Tailwind è **mobile-first**: le classi senza prefisso valgono sul telefono, `sm:` e `lg:` aggiungono da lì in su. Catalogo, scheda articolo (`/article/:id`) e carrello (`/cart`) sono nati così.
- Quello che resta da sistemare sul telefono:
  - i filtri del catalogo occupano quattro righe: vanno chiusi in un pannello "Filtri" apribile, e `min-w-56` sulla ricerca stringe tutto il resto sotto i 360 px;
  - nel carrello i pulsanti "+" e "−" sono `size="icon"`, cioè 32 px: sotto la soglia del dito;
  - il dialog dell'articolo è alto quanto lo schermo e la tastiera lo copre.
- **Il vero problema è la tabella dell'area admin**: prossima slide.

Una tabella non si mette in un telefono

```
// AdminPage: sei colonne (nome, categoria, marca, prezzo, stock, azioni).
// Sotto i 640 px diventano illeggibili. Due strade:

// A. Scorrimento orizzontale: onesto, veloce, brutto
<div className="overflow-x-auto"><Table>...</Table></div>

// B. Lo stesso dato, due presentazioni
export function ProductAdminList({ products }: { products: Product[] }) {
  const isMobile = useMediaQuery('(max-width: 639px)');
  return isMobile
    ? <div className="divide-y">{products.map(p => <ProductRowCard key={p.id} product={p} />)}</div>
    : <ProductTable products={products} />;
}
```

- La card mobile mostra nome, prezzo e scorte; categoria e marca sotto, più piccole; le azioni diventano uno **swipe** o un menu con tre puntini.
- Stessi dati, stesse mutation, due layout: cambia solo il componente di presentazione.

Container queries: il componente decide da solo

```
<!-- Tailwind v4: @container sul contenitore, @sm: sui figli -->
<div className="@container">
  <article className="flex flex-col @sm:flex-row @sm:items-center gap-3">
    ...
  </article>
</div>
```

- Con le media query un componente sa solo quanto è larga la *finestra*; con le container query sa quanto spazio *ha*.
- Serve esattamente alla card dell'articolo, che vive in tre posti diversi: griglia del catalogo, lista admin, riga del carrello.
- Altre armi moderne: `clamp()` per la tipografia fluida, `grid` con `auto-fit`/`minmax`, il selettore `:has()`.

Pattern di navigazione mobile

- **Barra in basso** con le destinazioni principali: per il cliente Catalogo, Carrello e Profilo, per l'admin Catalogo e Admin. Oggi stanno tutte nell'header (contatore del carrello compreso), raggiungibile solo con il pollice teso.
- **Sheet** dal basso al posto del dialog: il form dell'articolo diventa un pannello che sale, e la tastiera non lo copre.
- Pulsante flottante per l'azione principale: nuovo articolo.
- Filtri in uno sheet, con il numero di filtri attivi sul pulsante.
- Pull-to-refresh: è la `reload()` del vostro hook, attaccata a un gesto.
- Tastiere giuste: `inputMode="decimal"` sul prezzo, `inputMode="numeric"` sulle scorte, `type="email"` su login e registrazione. Nel progetto prezzo e campi della carta lo fanno già.
- Il tasto Indietro di Android deve chiudere lo sheet, non uscire dall'app.

Immagini e prestazioni su rete mobile

- Il problema numero uno di minicommerce su mobile è l'**immagine in base64**: il form admin legge il file con `FileReader.readAsDataURL` e salva la stringa `data:image/...` nella colonna `image_url`. Ogni articolo porta così il suo peso dentro la risposta JSON del catalogo.
- Si vede anche in salita: `express.json()` accetta 100 KB per richiesta, quindi una foto scattata dal telefono fa fallire il salvataggio.
- Soluzione (aperta nel modulo 1, ancora da scrivere): file su disco o su bucket, nel database solo l'URL, poi `<img srcset>` con due o tre dimensioni.
- Formati moderni: **AVIF** e WebP; `loading="lazy"` sotto la piega; `width` e `height` sempre, per non far saltare il layout.
- JavaScript: il code splitting del modulo 1 conta il doppio qui. Obiettivo: meno di 200 KB compressi al primo caricamento del catalogo.
- Misurate su un telefono *vero* di fascia media, non sul vostro portatile.

Provare sul telefono

```
# espone il dev server in rete locale (con npm i flag di Vite vanno dopo --)
npm run dev -- --host       # Vite stampa anche l'indirizzo http://192.168.x.x:5173

# il backend va solo tenuto acceso: app.listen(PORT) senza host ascolta
# gia' su tutte le interfacce, e il proxy di Vite inoltra /api dal PC
```

- Il proxy di Vite inoltra `/api` a `localhost:3001` **dal PC**, quindi funziona anche quando la pagina è aperta dal telefono. Se invece usate `VITE_API_URL` assoluto, lì va messo l'IP del PC.
- **Telefono reale via USB**: `chrome://inspect` su Android, Safari → Sviluppo su iOS con un Mac. DevTools complete, sul dispositivo vero.
- Device Mode di Chrome: buono per il layout, **inaffidabile** per prestazioni, gesti e tastiera.
- Emulatori: Android Studio gira anche su Windows; il simulatore iOS solo su macOS.

Laboratorio 3a

Laboratorio: minicommerce responsive

1. Viewport con `viewport-fit=cover`, `min-h-dvh`, safe area, bersagli da 44 px.
2. Filtri del catalogo dentro uno sheet apribile, con il contatore dei filtri attivi.
3. Tabella admin $\rightarrow$ lista di card sotto i 640 px, con le stesse azioni.
4. Carrello con una mano: pulsanti quantità da 44 px e form di pagamento che la tastiera non copre.
5. Barra di navigazione in basso sotto i 768 px (Catalogo, Carrello, Profilo); il form articolo diventa uno sheet.
6. Provare dal telefono (`npm run dev - -host`) e correggere i tre problemi più evidenti.

Domande di verifica

Perché `100vh` è sbagliato su mobile? Differenza fra media query e container query? Perché iOS zooma sugli input? Cosa rende il catalogo già quasi responsive?

Che cos'è una PWA

- Una web app che il browser può **installare**: icona in home, finestra propria, avvio offline.
- Tre ingredienti: **HTTPS**, un **manifest** (nome, icone, colori), un **Service Worker** (script che intercetta la rete).
- Nessuno store, nessuna approvazione, aggiornamento istantaneo: è "solo" un sito, ma si comporta da app.
- Per minicommerce significa due cose molto concrete: il cliente ha l'icona in home e **rivede il catalogo anche senza rete**; il magazziniere ha la sua app senza passare dallo store.

Manifest e installazione

```
// public/manifest.webmanifest
{
  "name": "minicommerce", "short_name": "minicommerce",
  "start_url": "/", "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff", "theme_color": "#0a0a0a",
  "icons": [
    { "src": "/icons/192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ],
  "shortcuts": [{ "name": "Carrello", "url": "/cart" }]
}
```

- `display: standalone` toglie la barra del browser; `maskable` dà l'icona adattiva su Android.
- Chrome ed Edge propongono l'installazione da soli; su iOS l'utente usa "Aggiungi a Home" da Safari.

vite-plugin-pwa

```
npm install -D vite-plugin-pwa
```

```
// vite.config.ts  (accanto a react() e tailwindcss())
VitePWA({
  registerType: 'prompt',
  manifest: { /* come nella slide precedente */ },
  workbox: {
    globPatterns: ['**/*.{js,css,html,svg,png,woff2}'],       // app shell precachata
    runtimeCaching: [
      { urlPattern: ({ url }) => url.pathname.startsWith('/api/products'),
        handler: 'NetworkFirst',
        options: { cacheName: 'articoli', networkTimeoutSeconds: 3,
                   expiration: { maxEntries: 200, maxAgeSeconds: 86400 } } },
      { urlPattern: ({ url }) => url.pathname.startsWith('/api/categories'),
        handler: 'StaleWhileRevalidate', options: { cacheName: 'categorie' } },
    ],
  },
})
```

Le categorie cambiano una volta al mese: si servono dalla cache e si aggiornano dietro le quinte. Gli articoli cambiano spesso: prima la rete, cache come rete di sicurezza.

Strategie di cache

Strategia	Comportamento	In minicommerce
Precache	scaricato all'installazione, servito dalla cache	JS, CSS, HTML, icone
CacheFirst	cache se c'è, altrimenti rete	immagini degli articoli, font
NetworkFirst	rete, con fallback alla cache	`GET /api/products`
StaleWhileRevalidate	subito la cache, aggiorna dietro	`GET /api/categories`
NetworkOnly	mai cache	login, `/api/cart`, `/api/orders`, scritture admin

- Le scritture non si cachano: si mettono in coda (prossima slide). Nemmeno `GET /api/cart` si cacha: è legato al token di quell'utente.
- Il service worker vive fuori dalla pagina: DevTools → Application per vederlo, aggiornarlo, cancellarlo.

Offline: leggere dalla cache, scrivere in coda

```
// 1. LEGGERE: ci pensa il service worker. GET /api/products in NetworkFirst
//    significa che senza rete il catalogo arriva comunque, dalla cache.

// 2. SCRIVERE: una coda nostra, in localStorage
export async function inviaOppureAccoda(op: Operazione) {
  try { await esegui(op); }
  catch { coda.push(op); salvaCoda(); }      // niente rete: in coda
}
window.addEventListener('online', async () => {
  for (const op of [...coda]) { await esegui(op); togliDallaCoda(op); }
  toast.success('Modifiche inviate');
});

// 3. La UI lo dice
const online = useOnlineStatus();
{!online && <Banner>Sei offline: {coda.length} modifiche in attesa</Banner>}
```

- È lo scenario del magazziniere: conta le scatole senza campo, e quando esce le modifiche partono da sole.
- Vale anche per il cliente: il carrello è **lato server**, quindi offline un `POST /api/cart/items` va in coda. L'ordine invece si fa online: deve vedere subito se è accettato.
- Conflitti: decide l'API. Per le scorte un endpoint `+1/-1` batte il valore assoluto.

Aggiornamenti e prompt di installazione

```
import { useRegisterSW } from 'virtual:pwa-register/react';

export function UpdateToast() {
  const { needRefresh: [needRefresh], updateServiceWorker } = useRegisterSW();
  if (!needRefresh) return null;
  return <Toast action={() => updateServiceWorker(true)}>Nuova versione disponibile</Toast>;
}

// prompt di installazione (Chromium)
window.addEventListener('beforeinstallprompt', (e) => { e.preventDefault(); setInstallEvent(e); });
<Button onClick={() => installEvent.prompt()}>Installa minicommerce</Button>
```

- Un service worker nuovo aspetta che tutte le schede siano chiuse: senza prompt, il negozio resta su una versione vecchia per giorni.
- Mostrate il pulsante “Installa” quando serve davvero: dopo il login, o dopo il primo articolo messo nel carrello. Al visitatore alla prima pagina non interessa.

Limiti delle PWA (soprattutto su iOS)

- Niente accesso a contatti, Bluetooth e NFC su iOS, niente esecuzione continua in background, niente widget.
- iOS può cancellare lo storage di una PWA non usata per qualche settimana: la cache del catalogo sparisce.
- Nessuna presenza negli store, a meno di impacchettarla.
- Push web su iOS solo per PWA *installate* in home; su Android ovunque.
- **PWABuilder** (Microsoft) genera pacchetti per Microsoft Store e Google Play (Trusted Web Activity) partendo dall'URL. Utile se vi serve solo la presenza nello store.

Quindi

PWA per il negozio del cliente (catalogo, scheda articolo, carrello) e per il PC del punto vendita. Per il magazziniere con fotocamera e lettore di codici: Capacitor.

Laboratorio 3b

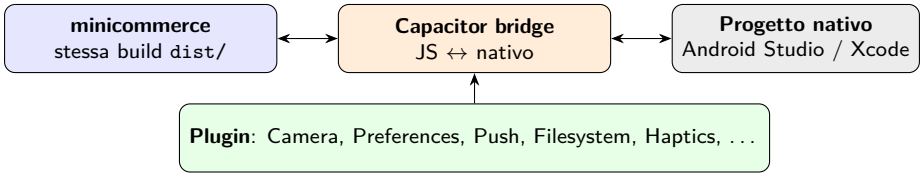
Laboratorio: minicommerce come PWA

1. Manifest, icone (anche maskable), `vite-plugin-pwa` con precache e le regole di runtime per `/api`.
2. Installare l'app dal telefono; in modalità aereo il catalogo si vede ancora, mentre carrello e ordine dicono chiaramente che serve la rete.
3. Coda delle scritture in `localStorage`, invio al ritorno online, banner con il numero di modifiche in attesa.
4. Prompt di aggiornamento; audit Lighthouse: PWA installabile e prestazioni sopra 90.

Domande di verifica

Che differenza c'è fra precache e runtime cache? Perché una nuova versione del service worker non parte subito? Perché le categorie usano StaleWhileRevalidate e gli articoli no?

Capacitor: la webview con i superpoteri



- È l'erede di Cordova, fatto dal team di Ionic. Genera **veri progetti** Android e iOS che potete aprire e modificare.
- Ionic (i componenti UI) è **opzionale**: minicommerce ha già shadcn/ui e Tailwind, e li teniamo.

Aggiungere Capacitor al frontend

```
cd frontend
npm install @capacitor/core && npm install -D @capacitor/cli
npx cap init minicommerce it.its.minicommerce --web-dir dist
npm install @capacitor/android @capacitor/ios
npx cap add android          # crea android/ (progetto Gradle)
npx cap add ios              # crea ios/ (solo su macOS)

npm run build && npx cap sync   # copia dist/ nei progetti nativi e installa i plugin
npx cap open android         # Android Studio -> Run sul telefono o sull'emulatore
```

```
// capacitor.config.ts
const config: CapacitorConfig = {
  appId: 'it.its.minicommerce', appName: 'minicommerce', webDir: 'dist',
  server: { androidScheme: 'https' },   // origine https://localhost
};
```

Le solite due modifiche, più una

```
npm run dev -- --host
npx cap run android --livereload --external     # l'app nativa carica il dev server del PC
```

- **Router**: la pagina è `https://localhost/index.html`, quindi hash router (o `server.url` in sviluppo).
- **API**: oggi `api.ts` usa `baseURL: '/api'` con il proxy di Vite, che nell'app nativa non c'è. Servono `VITE_API_URL` assoluto e **CORS** sul backend per `https://localhost` (Android) e `capacitor://localhost` (iOS). Il `cors()` aperto del progetto li accetta già: in produzione andrà ristretto a questi due valori più il dominio del sito.
- **Piattaforma**: a `lib/piattaforma.ts` si aggiunge il quarto caso.

```
import { Capacitor } from '@capacitor/core';
export const piattaforma = Capacitor.isNativePlatform() ? 'capacitor' : /* ...come prima... */;
```

La fotocamera: finalmente le foto degli articoli

```ts
// npm install @capacitor/camera && npx cap sync
import { Camera, CameraResultType, CameraSource } from '@capacitor/camera';

export async function scattaFotoArticolo(): Promise<File> {
  const photo = await Camera.getPhoto({
    quality: 80, width: 1200,          // ridimensiona PRIMA di inviare
    resultType: CameraResultType.Uri,
    source: CameraSource.Prompt,       // fotocamera o galleria, sceglie l'utente
  });
  const blob = await (await fetch(photo.webPath!)).blob();
  return new File([blob], `articolo.${photo.format}`, { type: blob.type });
}
```

- Sul web lo stesso plugin usa `<input type="file">`: **un solo codice** per le tre piattaforme.
- Va inviata in `multipart` a un endpoint di upload che **oggi non esiste** (lo scrivete nel laboratorio): la base64 del form attuale sfonda i 100 KB di `express.json()`.
- iOS: senza `NSCameraUsageDescription` in `Info.plist` l'app si chiude. Android: permessi nel manifest.

Sessione, tasto Indietro, deep link

```
import { Preferences } from '@capacitor/preferences';   // storage nativo chiave-valore
export const tokenStore = {
  get: async () => (await Preferences.get({ key: 'minicommerce_token' })).value,
  set: (t: string) => Preferences.set({ key: 'minicommerce_token', value: t }),
  clear: () => Preferences.remove({ key: 'minicommerce_token' }),
};

import { App } from '@capacitor/app';
App.addListener('backButton', ({ canGoBack }) => {
  if (sheetAperto) chiudiSheet();
  else if (canGoBack) window.history.back();
  else App.exitApp();
});
App.addListener('appUrlOpen', ({ url }) => {
  // minicommerce://article/7  oppure https://minicommerce.example/article/7
  router.navigate(new URL(url).pathname);
});
```

`Preferences` usa SharedPreferences su Android e UserDefaults su iOS, ma è **asincrono**: l'interceptor di axios legge il token da `localStorage` in modo sincrono, quindi va isolato dietro un `tokenStore`.

Due funzioni che solo l'app nativa può dare

Lettore di codici a barre

- Plugin `@capacitor-mlkit/barcode-scanning`.
- Il magazziniere inquadra il codice EAN e l'articolo si apre.
- Richiede un campo `barcode` nella tabella `products`: una riga nello schema Drizzle e un filtro in più nell'API.

Notifiche push

- `@capacitor/push-notifications` con Firebase Cloud Messaging (e APNs su iOS).
- Al magazziniere: "Tastiera meccanica: scorte esaurite". Al cliente: "Ordine accettato".
- Il backend invia a un token di dispositivo salvato al login.

Rifinitura nativa

`@capacitor/splash-screen`, `status-bar` (colore coerente con il tema), `haptics` (vibrazione quando una scorta va a zero), `keyboard` (ridimensiona la vista quando appare la tastiera).

Build, firma, store

Android

- Android Studio → Generate Signed Bundle (`.aab`), oppure `./gradlew bundleRelease`.
- Keystore: **non perdetelo mai**, senza non si aggiorna più l'app.
- Google Play Console: 25 $ una tantum; test interno → chiuso → produzione.

iOS

- Serve un Mac con Xcode e un account Apple Developer (99 $/anno).
- TestFlight per i tester, poi revisione (1–3 giorni).
- Attenzione: un'app che è solo un catalogo può essere rifiutata come "sito impacchettato". Fotocamera e scanner la rendono un'app vera; il nostro ordine finto, invece, andrebbe completato o tolto.

Laboratorio 3c

Laboratorio: minicommerce su Android (e iOS se avete un Mac)

1. `cap init`, `cap add android`, live reload sul telefono o sull'emulatore.
2. Fotocamera per la foto dell'articolo + endpoint di upload nel backend (multipart, file su disco, URL nel database).
3. `Preferences` per il token; tasto Indietro gestito; deep link `minicommerce://article/:id`.
4. Splash, status bar, haptics; APK di debug installato sul telefono di un compagno.
5. Aggiornare `lib/piattaforma.ts` con il caso Capacitor.

Domande di verifica

Cosa fa `cap sync`? Perché serve l'hash router? Cosa succede su iOS senza la descrizione d'uso della fotocamera? Perché il keystore va custodito?

Responsive vs PWA vs Capacitor vs nativo

	Web responsive	PWA	Capacitor	Nativo
Codice condiviso	100%	100%	95%	0–30%
Store	no	via wrapper	sì	sì
Offline	no	sì	sì	sì
Push	no	sì (limiti iOS)	sì	sì
Fotocamera, scanner	parziale	parziale	plugin	tutto
Prestazioni UI	browser	browser	webview	massime
Feeling nativo	basso	medio	medio/alto	alto
Costo di manutenzione	basso	basso	medio	alto

- Per minicommerce la combinazione sensata è: **PWA per catalogo e carrello** dei clienti, **Capacitor per l'app del magazzino**. Stessa build, due canali.
- Il modulo 5 mostra quando la colonna "Nativo" vince, e perché costa meno di quanto costava.

Dove siamo con minicommerce

Forma	Tecnologia	Stato
Web app	React + Vite	completata (mod. 1)
Desktop	Electron / Tauri	completata (mod. 2)
Mobile web	Responsive + PWA	completata (mod. 3)
Mobile store	Capacitor	completata (mod. 3)
Sito vetrina	SSR/SSG	prossimo modulo
Native	Swift / Kotlin / ...	modulo 5

Un solo repository, una sola logica, quattro modi di installarla. Ora cambiamo completamente problema: **farsi trovare da chi non sa che esistete**.

Sommario del modulo

5. SPA vs sito vetrina: rendering lato server
5.1 Due problemi diversi
5.2 Modalità di rendering
5.3 SEO e prestazioni percepite
5.4 La vetrina con Next.js
5.5 Il blog con Astro
5.6 PHP e le altre strade
5.7 I due rami, e il laboratorio

App e sito non sono la stessa cosa

App (admin e carrello di minicommerce)

- Utenti **loggati**: admin e clienti
- Molta interazione, poca "lettura"
- Non deve essere trovata da Google
- Si usa per ore: il primo caricamento conta poco
- SPA: scelta giusta

Vetrina (il negozio online)

- Visitatori **anonimi**, contenuti pubblici
- Poca interazione, molta lettura
- **Deve** essere trovata: SEO, anteprime social
- Visita di 30 secondi: il primo caricamento è tutto
- SPA: scelta sbagliata

Il problema di minicommerce oggi

Il catalogo è una SPA React: HTML vuoto, tutto il JavaScript prima di vedere un prezzo. Per un negozio che vive di ricerche su Google è il difetto più costoso di tutti.

Cosa vede un crawler sul catalogo

```
<!doctype html>
<html lang="it"><head><title>minicommerce</title></head>
<body>
  <div id="root"></div>
  <script type="module" src="/assets/index-8f2c.js"></script>
</body></html>
```

- Googlebot *esegue* JavaScript, ma in una seconda fase, con ritardo e budget limitato.
- Bing, i crawler di WhatsApp/LinkedIn/Slack per le anteprime, i comparatori di prezzo: **non lo eseguono**.
- Nessun prezzo, nessuna marca, nessuna disponibilità nell'HTML: niente risultati ricchi, niente anteprima della scheda.
- La scheda articolo un URL ce l'ha (`/article/:articleid`), la categoria no: i filtri del catalogo vivono nello stato React e nella query string.
- In ogni caso titolo e descrizione restano quelli di `index.html`, uguali per tutte le rotte: `<title>minicommerce</title>` e basta.

La prova, in una riga di comando

```
node scripts/confronto-seo.mjs --id 1
```

Stessa scheda, stesso articolo	SPA (Vite)	Vetrina (Next)
byte di HTML	632	39 111
parole di testo	1	125
`<title>` specifico	−	+
`description`	−	+
`canonical`	−	+
`<h1>`	−	+
blocchi JSON-LD	0	4
link `<a href>`	0	22
nome dell'articolo	assente	presente
prezzo	assente	presente

Da leggere ad alta voce

La SPA non sta sbagliando niente: fa il suo mestiere, che è far *usare* l'applicazione. Solo che il contenuto arriva dopo, in JavaScript, e in questo HTML non c'è. La vetrina fa il mestiere opposto: farsi *leggere*.

La divisione dei ruoli che adottiamo

Vetrina pubblica (ramo `web/nextjs`)

- `/`, `/categorie/monitor`, `/chi-siamo`
- `/prodotti/1-thinkpad-x1-...`: una per articolo
- Pregenerata al build, rigenerata quando il catalogo cambia
- Indicizzabile, veloce, quasi senza JavaScript

Area applicativa (modulo 1)

- `/login`, `/cart`, `/admin`: SPA React + Vite
- Protetta, dati sempre freschi a ogni apertura
- Nessun bisogno di SEO: un carrello non si indicizza

Due applicazioni, una sola API

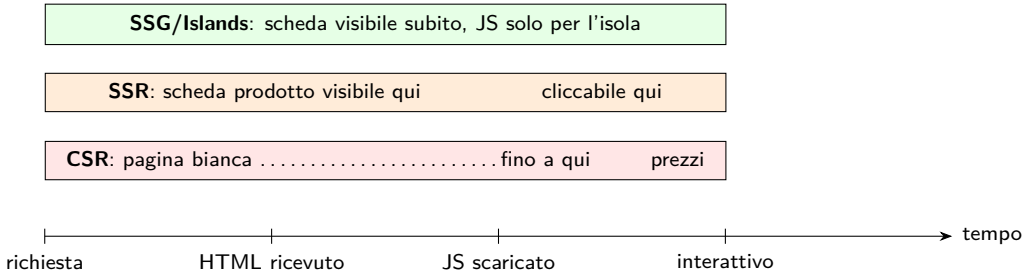
Entrambe parlano con lo stesso backend Express su `:3001`. La vetrina usa solo le rotte pubbliche in lettura (`GET /api/categories`, `GET /api/products`); l'app usa anche quelle protette: `/api/cart` e `POST /api/orders` per i clienti, le scritture sui prodotti per gli admin.

Il vocabolario

Sigla	Cosa succede	Esempio
CSR	Il browser riceve HTML vuoto e costruisce tutto con JS	la SPA di minicommerce
SSR	Il server genera l'HTML *a ogni richiesta*, poi il JS "idrata"	Next.js, PHP
SSG	L'HTML è generato *al build*, servito come file statico	Astro, Hugo
ISR	Statico, ma rigenerato dopo una scadenza o su richiesta	Next.js
Streaming	Il server invia l'HTML a pezzi man mano che è pronto	React 19, Next
PPR	Guscio statico spedito subito, i pezzi dinamici in streaming	Next 16
Islands	Pagina statica con "isole" interattive idratate a parte	Astro
RSC	Componenti React eseguiti *solo* sul server, zero JS al client	Next.js App Router

- Nessuna è "la migliore": un sito le **mescola** pagina per pagina. Nella vetrina Next ce ne sono cinque (SSR, SSG, ISR, streaming, PPR, tutte su RSC); nel blog Astro ci sono SSG e islands. La CSR è la SPA del modulo 1.

CSR vs SSR vs SSG: la linea del tempo



- **Idratazione**: React riprende in mano l'HTML già disegnato dal server e ci attacca gli eventi. Nel frattempo la pagina si vede ma non risponde.
- Meno JavaScript = idratazione più breve = pagina utilizzabile prima.

Come scegliere, pagina per pagina

Pagina	Perché	Modalità
`/chi-siamo`	non ha dati, cambia una volta all'anno	statica
`/categorie`	sette categorie, cambiano raramente	statica
`/categorie/:slug`	una per categoria, dal catalogo	SSG + ISR
`/prodotti/:slug`	una per articolo: è il contenuto che deve finire su Google	SSG + ISR
`/` con i filtri	dipende dalla query string: infinite combinazioni	dinamica
`sitemap.xml`, `robots.txt`	nascono dai dati	statiche
`/api/revalidate`	è un'azione, non una pagina	dinamica
`/login`, `/cart`, `/admin`	hanno un utente e un token	la SPA del modulo 1

Regola pratica

Statico finché possibile, server quando i dati cambiano a ogni richiesta, client solo per ciò che è davvero interattivo: è il confine fra pagina indicizzabile e area applicativa.

SEO tecnica: il minimo indispensabile

- `<title>` unico per pagina (50–60 caratteri): articolo, marca, negozio.
- `<meta name="description">` (150–160 caratteri): è il testo dello snippet.
- Un solo `<h1>`, gerarchia `h2`/`h3` sensata, HTML semantico.
- URL leggibili: `/categorie/monitor` e non `/?categoryId=3`.
- `<link rel="canonical">`: un elenco filtrato per prezzo non è una pagina nuova.
- `sitemap.xml` e `robots.txt`.
- Open Graph: titolo, descrizione, immagine 1200×630.
- Dati strutturati **JSON-LD**: per un negozio, `Product` e `BreadcrumbList`.
- Niente `noindex` dimenticato in produzione.
- Google Search Console: come Google vi vede davvero.

La testa di una scheda prodotto (quella vera)

```
<title>ThinkPad X1 Carbon Gen 12 | minicommerce</title>
<meta name="description" content="Ultrabook 14&amp;quot; con Intel Core Ultra 7,
      32GB RAM, 1TB SSD. Leggero e robusto, ideale per il lavoro in mobilità.">
<link rel="canonical"
      href="http://localhost:3000/prodotti/1-thinkpad-x1-carbon-gen-12">
<meta property="og:type" content="website">
<meta property="og:title" content="ThinkPad X1 Carbon Gen 12">
<meta property="og:image"
      content=".../prodotti/1-thinkpad-x1-carbon-gen-12/opengraph-image">
<meta property="og:image:width" content="1200">
<meta property="og:image:height" content="630">
<meta name="twitter:card" content="summary_large_image">
```

- Nessuno di questi tag è scritto a mano: li produce `generateMetadata()` partendo da `name`, `brand`, `category` e `description`.
- `title.template` nel layout fa il resto: la pagina dichiara solo “ThinkPad X1 Carbon Gen 12”.
- `metadataBase` trasforma i percorsi relativi in indirizzi assoluti: un `canonical` relativo non vuol dire niente per un crawler.

JSON-LD: il cuore della SEO di un negozio

```
{ "@context": "https://schema.org", "@type": "Product",
  "name": "ThinkPad X1 Carbon Gen 12", "sku": "MC-1",
  "category": "Portatili",
  "description": "Ultrabook 14\" con Intel Core Ultra 7, 32GB RAM, 1TB SSD...",
  "url": "http://localhost:3000/prodotti/1-thinkpad-x1-carbon-gen-12",
  "brand": { "@type": "Brand", "name": "Lenovo" },
  "offers": { "@type": "Offer",
    "price": "1899.00", "priceCurrency": "EUR",
    "itemCondition": "https://schema.org/NewCondition",
    "availability": "https://schema.org/InStock",
    "seller": { "@type": "Organization", "name": "minicommerce" } } }
```

- `availability`: `InStock` se `stock > 0`, altrimenti `OutOfStock`. Lo stesso dato del badge "Esaurito" del catalogo.
- `aggregateRating` darebbe le stelline nei risultati, ma **le recensioni non esistono** nel modello dati: aggiungerle è un esercizio, non si inventano.

Il JSON-LD generato dai dati veri

```
// web/lib/seo.ts
export function productJsonLd(prodotto: Product) {
  return {
    '@context': 'https://schema.org',
    '@type': 'Product',
    name: prodotto.name,
    sku: `MC-${prodotto.id}`,
    category: prodotto.category,
    url: absoluteUrl(productPath(prodotto)),
    ...(prodotto.brand ? { brand: { '@type': 'Brand', name: prodotto.brand } } : {}),
    offers: {
      '@type': 'Offer',
      price: prodotto.price.toFixed(2), priceCurrency: 'EUR',
      availability: prodotto.stock > 0
        ? 'https://schema.org/InStock'
        : 'https://schema.org/OutOfStock',
    },
  }
}
```

Quattro blocchi per pagina: `OnlineStore` e `WebSite` nel layout, `BreadcrumbList` e `Product` nella scheda. Da validare con il Rich Results Test di Google.

Il JSON-LD si stampa, non si costruisce a mano

```tsx
// web/components/StructuredData.tsx
export function StructuredData({ data }: { data: Record<string, unknown> }) {
  return (
    <script
      type="application/ld+json"
      dangerouslySetInnerHTML={{
        __html: JSON.stringify(data).replace(/</g, '\\u003c'),
      }}
    />
  )
}
```

- `dangerouslySetInnerHTML` qui è obbligato: React non sa mettere testo dentro uno `<script>`.
- Il rischio è reale, quindi lo chiudiamo a mano: se una descrizione contenesse `</script>` il browser chiuderebbe il blocco e il resto diventerebbe HTML eseguibile. Sostituendo `<` con `\u003c` il JSON resta valido e il tag non si può chiudere per sbaglio.
- È la stessa classe di problemi dell'XSS del modulo 6, in un posto dove nessuno la cerca.

Le immagini del catalogo: il nodo del base64

- Nel seed `imageUrl` è sempre `null`: il caso "nessuna immagine" non è un dettaglio, è il caso normale, e va disegnato bene.
- Quando l'admin carica una foto, il form la legge con `FileReader.readAsDataURL`: finisce nel database come **data URL base64**, fino a 5 MB.
- `next/image` non può ottimizzare un data URL: lo riconosce e imposta `unoptimized`, quindi lo lascia passare *così com'è*. Niente errori, ma niente WebP, niente ridimensionamento e 5 MB dentro l'HTML.
- Per le anteprime social ce la caviamo: l'`og:image` non è la foto dell'articolo, è un'immagine generata da noi (fra poche slide).
- **La soluzione vera** è l'upload su disco o su storage, con l'API che restituisce un URL. È l'esercizio che collega questo modulo al modulo 3.

Core Web Vitals su una pagina prodotto

Metrica	Misura	Buono	Cosa la rovina qui
LCP	quando appare l'elemento più grande	$< 2{,}5$ s	la foto dell'articolo: fino a 5 MB di base64 non ottimizzabili
INP	reattività ai click/tap	< 200 ms	idratare tutta la pagina invece della sola isola
CLS	quanto "salta" il layout	$< 0{,}1$	immagine senza `width`/`height`, prezzo che arriva dopo

- Sono un **fattore di ranking** e, soprattutto, misurano ciò che l'utente sente.
- Strumenti: Lighthouse (laboratorio), PageSpeed Insights e Search Console (utenti Chrome reali).
- Una SPA fatica su LCP; una pagina prodotto statica passa senza sforzo.

Next.js in due minuti

- Il framework React "full-stack" più diffuso: routing su file system, SSR/SSG/ISR, immagini ottimizzate, API route.
- **App Router**: **React Server Components** per default. I componenti girano sul server e mandano HTML; solo quelli marcati `'use client'` arrivano al browser.
- Sviluppato da Vercel; si distribuisce su Vercel con zero configurazione, o come server Node/Docker ovunque.
- Noi usiamo la versione **16**, quella che esce oggi da `create-next-app`: cache espliciti e guscio statico spedito subito.
- Per la vetrina di minicommerce è *molto*: lo usiamo perché conoscete React e perché lo troverete nelle aziende.

Il comando, e la struttura che ne esce

```
npx create-next-app@latest web --ts --app --tailwind \
    --no-src-dir --import-alias "@/*" --use-npm
```

```
web/
  app/
    layout.tsx                intestazione, pie' di pagina, metadati comuni
    page.tsx                  /                CATALOGO: dipende dai filtri
    not-found.tsx             la pagina 404
    robots.ts  sitemap.ts     generano /robots.txt e /sitemap.xml
    chi-siamo/page.tsx        pagina statica pura, nessun dato
    categorie/page.tsx        /categorie
    categorie/[slug]/page.tsx  /categorie/monitor
    prodotti/[slug]/page.tsx   /prodotti/1-thinkpad-x1-carbon-gen-12
    prodotti/[slug]/opengraph-image.tsx   l'anteprima social, una per articolo
    api/revalidate/route.ts    butta via la cache quando i dati cambiano
  components/                 Header, Footer, ProductCard, FilterForm, ...
  lib/api.ts                  il client dell'API, versione server
  lib/seo.ts  lib/slug.ts  lib/format.ts
```

Ogni cartella con `page.tsx` è una route; `layout.tsx` avvolge le figlie; `[slug]` è un parametro.

Server Component: dati senza useEffect

```tsx
// web/app/categorie/[slug]/page.tsx  -- gira SOLO sul server
export default async function CategoryPage({
  params,
}: PageProps<'/categorie/[slug]'>) {
  const { slug } = await params;
  const categoria = await trovaCategoria(slug);
  if (!categoria) notFound();

  const prodotti = await getProductsByCategory(categoria.name);

  return (
    <main>
      <h1>{categoria.name}</h1>
      <p>{categoria.description}</p>
      <ProductGrid prodotti={prodotti} />
    </main>
  );
}
```

In Next 16 `params` è una `Promise`: si aspetta con `await`. Il tipo `PageProps<'/categorie/[slug]'>` lo genera Next dalle cartelle che esistono davvero.

Cosa è cambiato rispetto al catalogo SPA

- Niente `useState`, niente `useEffect`, niente spinner: è una funzione `async` che restituisce JSX.
- La chiamata a `GET /api/products?category=Monitor` avviene **sul server**, al build: l'utente non la aspetta.
- Il client riceve HTML finito e **zero JavaScript** per questo componente.
- L'API non ha bisogno di CORS per queste chiamate: partono da Node, non dal browser.
- Niente `axios` e niente token: `lib/api.ts` usa `fetch` e legge solo rotte pubbliche. L'indirizzo sta in `API_URL`, *senza* prefisso pubblico, quindi resta sul server.

Una pagina HTML per articolo: generateStaticParams

```tsx
// web/app/prodotti/[slug]/page.tsx

/** L'elenco delle pagine da pregenerare: una per articolo. */
export async function generateStaticParams() {
  const prodotti = await getCatalog();                    // GET /api/products
  return prodotti.map((p) => ({ slug: productSlug(p) }));
}

export default async function ProductPage({
  params,
}: PageProps<'/prodotti/[slug]'>) {
  const { slug } = await params;
  const id = idFromSlug(slug);                // "1-thinkpad-..." -> 1
  if (id === null) notFound();

  const prodotto = await getProduct(id);
  if (!prodotto) notFound();

  // Slug vecchio (nome cambiato)? 308 verso l'indirizzo canonico.
  const slugCanonico = productSlug(prodotto);
  if (slug !== slugCanonico) permanentRedirect(`/prodotti/${slugCanonico}`);
```

`generateStaticParams` gira durante `next build`: se il backend non è avviato, la build si ferma. È il patto del rendering statico, i dati devono esistere prima dei visitatori.

Lo slug: perché l'id sta davanti

- `/prodotti/1` non dice niente a chi legge il link, né a un motore di ricerca. `/prodotti/1-thinkpad-x1-carbon-gen-12` sì.
- Il nome di un articolo **cambia**; l'id no. Con l'id in testa la pagina si ritrova sempre, anche dopo una rinomina.
- Se lo slug non è quello canonico, la pagina risponde **308** verso quello giusto: l'utente arriva dove voleva e Google aggiorna il suo indice.
- Lo slug non esiste nel database: lo calcoliamo dal nome con `slugify()` (`normalize('NFD')` per togliere gli accenti, tutto il resto diventa un trattino).
- Aggiungere una colonna `slug` al database, per poterlo scegliere a mano, è un esercizio.

Next 16: cosa è in cache lo decidete voi

```ts
// web/lib/api.ts
import { cacheLife, cacheTag } from 'next/cache'

/** Catalogo completo: alimenta pagine statiche, sitemap e staticParams. */
export async function getCatalog(): Promise<Product[]> {
  'use cache'
  cacheTag('prodotti')
  cacheLife('hours')
  return chiamaApi<Product[]>('/products?sort=name&order=asc')
}

/** La ricerca con i filtri NON e' in cache: cambia a ogni visita. */
export async function searchProducts(filtri: ProductFilters) {
  const query = new URLSearchParams(/* ... */)
  return chiamaApi<Product[]>(`/products?${query}`)   // cache: 'no-store'
}
```

- Con `cacheComponents` acceso la regola è una: **niente è in cache se non lo chiedete voi**.
- `'use cache'` rende la funzione memorizzabile, `cacheTag` le dà un'etichetta per invalidarla, `cacheLife` dice quanto può restare.
- "Statico" e "dinamico" diventano una scelta scritta nel codice, invece di un effetto collaterale di quale funzione si è chiamata.

L'elenco di `next build`: il riassunto più onesto

Route	Modalità	
`/`	PPR	guscio statico, risultati in streaming
`/categorie`	statica	
`/categorie/[slug]`	SSG $\times$ 7	una per categoria
`/chi-siamo`	statica	nessun dato
`/prodotti/[slug]`	SSG $\times$ 16	una per articolo
`/prodotti/[slug]/opengraph-image`	SSG $\times$ 16	anche le immagini
`/robots.txt`, `/sitemap.xml`	statiche	
`/api/revalidate`	dinamica	è un'azione

- Nell'output di `next build` le trovate marcate con un cerchio vuoto (statica), pieno (SSG), mezzo pieno (PPR) e una *f* corsiva (dinamica).
- 16 articoli, 16 pagine, 16 immagini di anteprima: tutto pronto **prima** del primo visitatore. Tempo totale della generazione: circa due secondi.
- Il catalogo con i filtri è l'unica pagina che non si può pregenerare, e si vede.

I filtri senza una riga di JavaScript

```
<form method="get" action="/">
  <input name="search" type="search" value="cuffie">
  <select name="category">...</select>
  <input name="minPrice" type="number"> <input name="maxPrice" type="number">
  <select name="sort">...</select> <select name="order">...</select>
  <button type="submit">Applica i filtri</button>
</form>
```

- Nella SPA questo pannello è uno stato React tenuto in sincronia con `useSearchParams`. Qui è un `<form method="get">`: il browser compone la query string da solo e il server risponde con l'HTML giusto.
- Funziona con JavaScript disattivato, funziona dentro un crawler, e il risultato si può copiare e incollare come link.
- Il prezzo da pagare: i campi vuoti finiscono comunque nell'URL (`?search=&brand=`). Il backend li ignora, ma per un motore di ricerca sono indirizzi diversi con lo stesso contenuto.

La pagina filtrata non va indicizzata

```tsx
// web/app/page.tsx
export async function generateMetadata({ searchParams }: PageProps<'/'>) {
  const filtri = leggiFiltri(await searchParams)

  if (contaFiltri(filtri) === 0) {
    return { title: { absolute: 'minicommerce - ...' },
             alternates: { canonical: '/' } }
  }
  return {
    title: `Ricerca: ${pezzi.join(', ')}`,
    alternates: { canonical: '/' },
    robots: { index: false, follow: true },   // <-- noindex, follow
  }
}
```

- `?search=mouse&minPrice=10&order=desc` e le sue infinite varianti mostrano pezzi della stessa lista. Lasciarle indicizzare significa far spendere al crawler il suo budget su migliaia di doppioni invece che sulle schede prodotto.
- `follow` resta attivo: i link dentro la pagina vanno seguiti, la pagina no.
- `canonical` punta al catalogo pulito: a Google diciamo qual è l'indirizzo che conta.

“E quando cambia un prezzo?”

```
// web/app/api/revalidate/route.ts
export async function POST(richiesta: NextRequest) {
  const segreto = process.env.REVALIDATE_SECRET
  const fornito = richiesta.headers.get('x-revalidate-secret')
  if (fornito !== segreto) {
    return Response.json({ error: 'Segreto mancante o sbagliato' },
                         { status: 401 })
  }
  const { tag } = await richiesta.json()        // 'prodotti', 'prodotto-7', ...
  revalidateTag(tag, { expire: 0 })             // butta via la copia in cache
  return Response.json({ ok: true, tag })
}
```

- È la domanda che arriva sempre, ed è quella giusta. Risposta: si butta via la copia in cache, e la prima visita successiva rigenera l’HTML. **Nessuna build, nessun deploy**.
- Il secondo argomento dice quanto si può ancora servire la copia vecchia: con `'max'` l’aggiornamento avviene di nascosto, con `{ expire: 0 }` la copia vecchia va subito in pattumiera. In produzione si preferisce il primo; a lezione serve vedere il cambiamento subito.
- Serve un segreto in variabile d’ambiente: un indirizzo pubblico che costa lavoro al server è un invito a farsi male.

La prova del nove, in tre comandi

```
# 1. l'admin cambia il prezzo (dalla SPA, oppure con l'API)
#    prezzo nel database: 1555      pagina: ancora 1899,00 EUR  (in cache)

# 2. si butta via la copia in cache
curl -X POST "http://localhost:3000/api/revalidate" ‘
     -H "x-revalidate-secret: cambiami" ‘
     -H "content-type: application/json" ‘
     -d '{"tag":"prodotti"}'
# -> {"ok":true,"tag":"prodotti"}

# 3. si ricarica la pagina:  1.555,00 EUR
```

- Senza il segreto: `401 Segreto mancante o sbagliato`.
- In fondo alla scheda c'è scritto "Dati in cache dal . . . ": quella data è il momento in cui l'HTML è stato generato, non quello della visita. Ricaricate dieci volte e non cambia; invalidate l'etichetta e cambia.
- **L'esercizio naturale**: chiamare questa rotta dal backend dentro `PUT /api/products/:id`, così il sito si aggiorna da solo appena si salva un articolo.

404 e 308: lo stato HTTP conta

```
curl -I http://localhost:3000/prodotti/999-non-esiste    # 404
curl -I http://localhost:3000/prodotti/1-nome-sbagliato  # 308 -> slug giusto
curl -I http://localhost:5173/pagina/che/non/esiste      # 200 (!) nella SPA
```

- Un motore di ricerca che riceve 404 toglie l'indirizzo dall'indice. Nella SPA *qualunque* indirizzo sbagliato risponde 200 con la pagina vuota: per un crawler sono tutte pagine valide e identiche.
- Una finezza da provare: alla **primissima** richiesta di un indirizzo mai visto, Next spedisce subito il guscio statico (stato 200) e completa la pagina in streaming; dalla seconda in poi l'esito è in cache e lo stato è quello giusto. Lanciate il comando due volte e si vede.
- Nel frattempo la pagina dichiara comunque `noindex`, quindi nell'indice non ci finisce.

L'anteprima social, una per articolo

```tsx
// web/app/prodotti/[slug]/opengraph-image.tsx
export const size = { width: 1200, height: 630 }
export const contentType = 'image/png'

export default async function OpenGraphImage({ params }) {
  const prodotto = await getProduct(idFromSlug((await params).slug))
  return new ImageResponse(
    <div style={{ display: 'flex', flexDirection: 'column', padding: 72,
                  backgroundColor: '#0a0a0a', color: '#fafafa' }}>
      <div style={{ fontSize: 68, fontWeight: 700 }}>{prodotto.name}</div>
      <div style={{ fontSize: 56 }}>{formatPrice(prodotto.price)}</div>
    </div>,
    size,
  )
}
```

- Il file si chiama `opengraph-image` e sta nella cartella della pagina: Next aggiunge da solo i `<meta property="og:image">` con indirizzo e dimensioni.
- Dentro `ImageResponse` non c'è un browser: c'è **satori**, che sa disegnare flexbox e testo e nient'altro. Stili inline, niente Tailwind.
- Sono 16 PNG generati al build. È anche il modo di avere un'anteprima decente *senza* avere le foto degli articoli.

sitemap.xml e robots.txt nascono dai dati

```ts
// web/app/sitemap.ts
export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const [prodotti, categorie] = await Promise.all([getCatalog(), getCategories()])
  return [
    { url: absoluteUrl('/'), changeFrequency: 'daily', priority: 1 },
    ...categorie.map((c) => ({ url: absoluteUrl(`/categorie/${categorySlug(c)}`) })),
    ...prodotti.map((p) => ({ url: absoluteUrl(`/prodotti/${productSlug(p)}`),
                              lastModified: new Date(p.createdAt) })),
  ]
}
```

- 26 indirizzi: tre pagine fisse, sette categorie, sedici articoli. Nessuno li ha scritti a mano, e un articolo nuovo ci finisce dentro alla prima rigenerazione.
- `lastModified` non è decorativo: dice al crawler cosa vale la pena riscaricare. Non metteteci "adesso", o l'informazione non vale più niente.
- `robots.ts` dichiara dov'è la sitemap e cosa non vale la pena leggere (`/api/`). Attenzione all'equivoco: `disallow` non nasconde e non protegge niente.

Due cose che la vetrina *non* usa

Nessun `'use client'`

- Nella vetrina non c'è un solo componente client: i filtri sono un `<form>`, il tema è CSS, il carrello sta nella SPA.
- `'use client'` segna il confine: da lì in giù il codice va anche al browser.
- Serve quando c'è davvero interazione locale. Qui non c'era.

Nessuna Server Action

- `'use server'` esegue sul server la funzione che gestisce un form, senza scrivere una API.
- La vetrina non ha form da inviare: l'unica scrittura è `POST /api/revalidate`, che è una rotta.
- Nel repository però c'è un progetto intero costruito su di esse, ed è la prossima slide.

Il tema scuro, per dire

Niente `next-themes` e nessuna classe sul `<html>`: solo `prefers-color-scheme`. Nessun JavaScript, nessuna idratazione, nessun lampeggio.

La stessa applicazione in un progetto solo: `singleton/`

Sul ramo `web/nextjs` c'è una seconda risposta, in una cartella a parte: non la vetrina *accanto* alla SPA, ma **tutto il negozio dentro Next**. Catalogo, login, carrello e admin, senza API REST in mezzo.

	SPA + Express (modulo 1)	`singleton/`
Lettura del catalogo	il browser chiama `GET /api/products`	un Server Component chiama `listProducts()`
Scrittura	`POST /api/products` con token Bearer	una Server Action scrive su SQLite
Sessione	JWT in `localStorage`	JWT in un cookie `httpOnly`
Processi da avviare	2 (Vite e Express)	1

L'API non scompare: cambia natura

Le stesse quattro operazioni ci sono tutte, ma da *contratto HTTP* diventano **chiamate di funzione tipizzate**. Il confine fra client e server resta, e resta sorvegliato: cambia solo che non è più una rete.

singleton/: leggere e scrivere senza una rotta

```tsx
// singleton/app/page.tsx -- il catalogo, filtri compresi, e' un Server Component
export default async function CatalogPage({ searchParams }: PageProps<'/'>) {
  const params = await searchParams          // i filtri stanno nella query string
  const session = await getSession()         // dal cookie firmato, non da un header
  const products = listProducts({ search, category, sort, order })   // SQLite, qui
  return <div>{products.map((p) => <ProductCard key={p.id} product={p} />)}</div>
}
```

```tsx
// singleton/server/actions/cart.ts -- la scrittura, senza una rotta
'use server'
export async function addToCartAction(productId: number, quantity = 1) {
  const session = await requireCustomer()    // non c'e' un 401: c'e' un'eccezione
  db.insert(cartItems).values({ userId: session.userId, productId, quantity }).run()
  revalidatePath('/carrello')                // la pagina si rifa' da sola
}
```

- `lib/db/index.ts` è marcato `server-only`: se un componente client provasse a importarlo, la build *fallisce* invece di spedire il driver SQLite al browser.
- `serverExternalPackages: ['better-sqlite3']` in `next.config.ts`: è un modulo nativo, deve restare un `require()` di Node.

Tutto in uno: cosa si guadagna, cosa si perde

+ Si guadagna

- Un processo invece di due, un deploy invece di due.
- I tipi e la validazione si scrivono **una volta**.
- Niente token in `localStorage`, niente CORS, niente proxy di Vite.

– Si perde

- **L'API pubblica**: Electron, Capacitor e le app native dei moduli 2, 3 e 5 non hanno più niente da chiamare.
- Il server diventa obbligatorio: serve un hosting con Node.
- Il contratto (`docs/API.md`) non ha più un posto dove vivere.

Perché nel corso restano entrambi

Rifare l'applicazione senza API è costato circa **3.400 righe**, e il backend Express non si è toccato: ci parlano altri quattro client. Con `DATABASE_URL=../backend/minicommerce.db` le due versioni lavorano sugli **stessi dati**: l'unico stato vero è il database.

Astro: nato per i siti di contenuto

- Genera **HTML statico con zero JavaScript** per default. Ogni componente interattivo è un'**isola** che dichiarate esplicitamente.
- Le isole possono essere in **React**, Vue, Svelte, Solid: riusate i componenti che sapete già scrivere.
- **Content collections**: Markdown/MDX con schema validato (zod), perfetto per tutto ciò che scrive una persona.
- `astro build` produce una cartella di file: hosting statico ovunque, anche dove Node non esiste. Punteggi Lighthouse da 100 senza sforzo.
- Nel 2026 è la scelta più comune per landing, blog, documentazione, vetrine di contenuto.

Si può usare Astro per minicommerce?

Pezzo del progetto	Astro è adatto?
Guide, recensioni, pagine informative	+ **Sì**: è esattamente il suo mestiere
Catalogo con filtri e ricerca	– No: dipende dalla query string, cambia a ogni richiesta
Scheda prodotto con prezzo e stock	+ Sa pregenerarla, – ma non ha una risposta pronta a "e quando cambia un prezzo?"
Carrello, login, area admin	– No: hanno utente, token e stato

La domanda giusta non è "si può", è "su che pezzo"

Astro genera le pagine in anticipo: va a nozze con ciò che cambia *quando lo decide chi scrive*, va male con ciò che cambia da solo o che dipende da chi guarda. Per le schede prodotto la risposta è Next con `revalidateTag`; per il blog è Astro, e non c'è gara.

Il comando, e la struttura che ne esce

```
npm create astro@latest blog -- --template minimal
cd blog && npx astro add react sitemap tailwind
npm install @astrojs/rss
```

```
blog/
  astro.config.mjs           site, output: 'static', le tre integrazioni
  src/
    content.config.ts        la forma del frontmatter: un errore ferma la build
    content/blog/*.md        i sei articoli
    layouts/BaseLayout.astro head, metadati, Open Graph, JSON-LD
    components/
      PrezzoLive.tsx         LA sola isola React del sito
      ProductCard.astro      la scheda di un articolo del catalogo
    lib/shop.ts              legge il catalogo dall'API, solo in fase di build
    pages/
      index.astro            l'elenco degli articoli
      blog/[...slug].astro   una pagina per post (getStaticPaths)
      tag/[tag].astro        una pagina per argomento
      rss.xml.ts  robots.txt.ts  404.astro
```

Template `minimal`: partiamo da una pagina vuota, così si vede nascere ogni pezzo. Gli altri template arrivano con un blog già fatto, che a lezione è come leggere la soluzione prima dell'esercizio.

I contenuti hanno uno schema

```
// blog/src/content.config.ts
const blog = defineCollection({
  loader: glob({ pattern: '**/*.md', base: './src/content/blog' }),
  schema: z.object({
    title: z.string().min(10).max(70),
    description: z.string().min(50).max(160),
    pubDate: z.coerce.date(),
    author: z.string().default('Redazione minicommerce'),
    tags: z.array(z.string()).min(1),
    /** Gli id degli articoli del catalogo di cui parla il post. */
    prodotti: z.array(z.number().int().positive()).default([]),
    bozza: z.boolean().default(false),
  }),
})
export const collections = { blog }
```

- Da qui nascono **i tipi**: nelle pagine `post.data.pubDate` è una `Date` vera, con l'autocompletamento.
- I limiti sulla `description` non sono pignoleria: sotto i 50 caratteri non dice niente, sopra i 160 Google la taglia. La regola SEO diventa un controllo, una volta per tutte.

Lo schema ferma la build, non il visitatore

```
# scrivete "pubdate:" invece di "pubDate:", oppure accorciate la description
npm run build

[InvalidContentEntryDataError] blog -> portatile-per-studiare-informatica
  data does not match collection schema.
  description: Too small: expected string to have >=50 characters
```

- Trenta secondi di dimostrazione, e si capisce a cosa serve un file di configurazione dei contenuti.
- Un contenuto sbagliato non arriva in produzione: si ferma sul computer di chi scrive, con il nome del file e il nome del campo.
- È la stessa idea della validazione zod sul form del modulo 1, spostata un passo più indietro: non “l’utente ha sbagliato”, ma “l’autore ha sbagliato”.

Una pagina per post

```
---
// blog/src/pages/blog/[...slug].astro  -- questa parte gira al BUILD
export async function getStaticPaths() {
  const post = await getCollection('blog', ({ data }) => !data.bozza);
  return post.map((articolo) => ({
    params: { slug: articolo.id },
    props: { articolo },
  }));
}

const { articolo } = Astro.props;
const { Content } = await render(articolo);              // Markdown -> HTML
const prodotti = await getProducts(articolo.data.prodotti);  // dall'API
---
<BaseLayout title={articolo.data.title} description={articolo.data.description}>
  <h1>{articolo.data.title}</h1>
  <div class="prosa"><Content /></div>
  {prodotti.map((prodotto) => <ProductCard prodotto={prodotto} />)}
</BaseLayout>
```

`getStaticPaths` è il cuore di un generatore statico: dice "ecco l'elenco delle pagine che devono esistere". Astro le costruisce tutte e il risultato è una cartella di file `.html`.

Il filo con il negozio, e il suo difetto

- Ogni post dichiara nel frontmatter gli `prodotti` di cui parla. `lib/shop.ts` li legge dall'API **una volta per build** (sei post, due articoli ciascuno, una sola chiamata).
- Quindi il prezzo scritto in pagina è il prezzo **del giorno della pubblicazione**. Se domani cambia nel catalogo, qui resta quello vecchio finché il blog non viene rigenerato.
- Il difetto è strutturale: invece di nasconderlo, lo raccontiamo ai lettori in un post e lo correggiamo dove serve.
- `lib/shop.ts` **non** fa fallire la build se l'API è spenta: qui il contenuto sono gli articoli, i prezzi sono un extra. Nella vetrina Next la scelta è opposta, e per il motivo opposto: là il contenuto *è* il prodotto.

L'isola: rendere dinamico il numero, non il sito

```
// blog/src/components/PrezzoLive.tsx     (React, idratato nel browser)
export default function PrezzoLive({ productId, prezzoAlBuild }) {
  const [prezzo, setPrezzo] = useState(prezzoAlBuild)
  const [stato, setStato] = useState('attesa')

  useEffect(() => {
    const controllo = new AbortController()
    fetch(`${API_URL}/products/${productId}`, { signal: controllo.signal })
      .then((r) => r.json())
      .then((p) => { setPrezzo(p.price)
                     setStato(p.price === prezzoAlBuild ? 'confermato' : 'cambiato') })
      .catch(() => { if (!controllo.signal.aborted) setStato('errore') })
    return () => controllo.abort()
  }, [productId, prezzoAlBuild])
```

```
<PrezzoLive productId={prodotto.id} prezzoAlBuild={prodotto.price} client:visible />
```

Stesso schema dell'hook `useProducts` del modulo 1, `AbortController` compreso. Invece di rendere dinamico tutto il sito per un numero, rendiamo dinamico **il numero**.

client:visible, e il costo del JavaScript

Quanto JavaScript scarica un lettore

- Home ed elenchi per tag: **un solo** `<script>`, e contiene il JSON-LD. Zero JavaScript eseguibile.
- Pagina di un post: tre script, cioè il runtime delle isole e il componente del prezzo, *quando* serve.
- Si paga solo quello che si usa.

Le direttive

- `client:load`: subito.
- `client:idle`: quando il browser è libero.
- `client:visible`: quando entra nello schermo (la nostra).
- `client:only`: mai sul server.
- Nessuna direttiva: HTML puro, zero JS.

La prova da fare in aula

Cambiate un prezzo dal negozio e ricaricate il post *senza* rifare la build: sotto il prezzo compare "prezzo aggiornato adesso", con il vecchio sbarrato. Poi disattivate JavaScript: resta il prezzo pubblicato, con scritto che è quello. L'isola **aggiunge**, non sostituisce.

Feed, sitemap e robots: file che non si scrivono

```ts
// blog/src/pages/rss.xml.ts
export async function GET(context: APIContext) {
  const post = await getCollection('blog', ({ data }) => !data.bozza)
  return rss({
    title: 'Il blog di minicommerce',
    site: context.site!,            // da "site" in astro.config.mjs
    items: post.map((a) => ({
      title: a.data.title, description: a.data.description,
      pubDate: a.data.pubDate, link: `/blog/${a.id}/`,
      categories: a.data.tags,
    })),
    customData: '<language>it-it</language>',
  })
}
```

- Un file dentro `src/pages` che non finisce in `.astro` è una rotta che restituisce quello che vuole: XML qui, testo in `robots.txt.ts`.
- Sei post diventano **15 pagine** in un secondo e mezzo: sei articoli, sette archivi per tag, la home e la 404, più feed, robots e sitemap.
- Nessun elenco è scritto a mano: i tag nascono dai post, la sitemap dalle pagine costruite davvero.

E Astro, tutto in uno?

La domanda è legittima: se Next ha `singleton/`, perché il blog chiama ancora l'API?

- I pezzi ci sono tutti: Astro 7 ha le **azioni** (`astro:actions`) al posto delle rotte, la **sessione** in configurazione, le **isole server** (`server:defer`), e con `@astrojs/node` e `output: 'server'` diventa un processo come Next.
- Quindi sì, `singleton/` si potrebbe riscrivere in Astro: si otterrebbe una versione *meno comoda* di ciò che Next fa di mestiere, usando Astro contro la sua natura.
- Il "tutto in uno" che ha senso per il blog è un altro: `lib/shop.ts` legge SQLite con Drizzle *durante la build* invece di chiamare `/api/products`. Un file, e il backend non serve più per costruire il blog.

Il punto, e il suo prezzo

A un generatore statico la sorgente è indifferente: API, database, CMS, file. Conta **quando** legge, e legge al build. Ma il prezzo resta congelato come prima, e il blog si lega allo **schema del database** invece che al contratto HTTP: più comodo oggi, più fragile al primo `ALTER TABLE`.

Astro o Next? La tabella che serve davvero

Se dovete...	Scegliete
pubblicare contenuti scritti da persone	**Astro**
mandare al browser il meno JavaScript possibile	**Astro**
pubblicare su un hosting qualunque, senza Node	**Astro**
mostrare dati che cambiano da soli (prezzi, scorte)	**Next**
rigenerare una pagina alla volta quando un dato cambia	**Next**
gestire sessioni, form autenticati, azioni lato server	**Next**
fare tutto questo insieme	**tutti e due**, su pezzi diversi

- Entrambi sanno fare quasi tutto: la differenza è *cosa* rendono facile e cosa fanno pagare.
- Ed è esattamente ciò che fa minicommerce: quattro rami, quattro strumenti, un repository e un solo backend.

PHP nel 2026: il sito che il negozio ha già

- WordPress da solo serve oltre il 40% dei siti; PHP è il rendering lato server *originale*.
- Un negozio di informatica che vende online, oggi, molto probabilmente ha **WooCommerce** o **PrestaShop**: SSR puro, SEO già risolta, catalogo gestito dal cliente.
- **Laravel** è un framework moderno: routing, ORM, template Blade, code, autenticazione.
- Con **Inertia.js** Laravel serve pagine React: il server fa routing e dati, React disegna. Niente API REST da scrivere.
- Con **Livewire** si scrivono componenti interattivi in PHP, con pochissimo JS.

Laravel + Blade: la stessa scheda prodotto

```php
// routes/web.php
Route::get('/prodotti/{id}', function (int $id) {
    $articolo = Product::with('category')->findOrFail($id);
    return view('prodotti.show', ['articolo' => $articolo]);
});
```

```
{{-- resources/views/prodotti/show.blade.php --}}
@extends('layouts.app', ['title' => $articolo->name])
@section('content')
  <article>
    <h1>{{ $articolo->name }}</h1>
    <p>{{ $articolo->brand }} - {{ number_format($articolo->price, 2) }} EUR</p>
    <p>{{ $articolo->stock > 0 ? 'Disponibile' : 'Esaurito' }}</p>
  </article>
@endsection
```

- Ogni richiesta esegue PHP e produce HTML: SSR puro. Cache (Redis, CDN) per reggere il traffico.
- {{ }} fa l'escape: la protezione da XSS è il default, come in JSX.

CMS: quando il catalogo non è vostro

- Il cliente vuole cambiare testi, foto e promozioni da solo: un CMS è la risposta, non un file Markdown nel vostro repo.
- Due modi di usare WordPress:
  - **Classico**: tema PHP, plugin, hosting condiviso. Veloce da consegnare, lento e fragile nel tempo.
  - **Headless**: WordPress solo come backend (REST o GraphQL), il sito lo genera Astro/Next. Prestazioni del sito statico, editing familiare.
- Alternative headless nate per questo: **Strapi**, **Payload**, **Sanity**, **Directus**. Per i cataloghi: Shopify, Medusa, commercetools.
- Nel nostro caso il "CMS" è l'admin di minicommerce, e il webhook "articolo salvato" → `POST /api/revalidate` è lo stesso schema.

Il resto del panorama

- **Nuxt** (Vue), **SvelteKit** (Svelte), **React Router framework mode** (ex Remix): stessi concetti di Next con altri framework.
- **Hugo**, **Eleventy**: generatori statici senza framework JS. Velocissimi, ideali per documentazione.
- **HTMX**: HTML che chiede frammenti HTML al server. Con PHP, Go o Python permette interattività senza SPA.
- **Edge rendering**: Cloudflare Workers, Vercel Edge. SSR eseguito vicino all'utente.
- **Site builder** (Webflow, Framer, Shopify): per il cliente che non vuole un repo. Sappiate cosa sono e quando proporli.

Cosa c'è nei due rami

`web/nextjs` – la vetrina

- Cartella `web/`: 38 file aggiunti
- 16 schede prodotto + 7 categorie pregenerate
- sitemap, robots, 16 immagini di anteprima
- `POST /api/revalidate`
- Più `singleton/`: la stessa app senza API
- **Non tocca** né `frontend/` né `backend/`

`web/astro` – il blog

- Cartella `blog/`: 34 file aggiunti
- 6 post, 15 pagine, feed RSS
- Una sola isola React
- `lib/shop.ts` legge il catalogo al build
- Del negozio cambia **un solo file**: la voce "Blog" nell'header

Un repository, quattro rami

`master` + `web/nextjs` + `web/astro` + `desktop/electron` si fondono fra loro senza un conflitto: ogni ramo aggiunge una cartella e non tocca quelle degli altri. Gli unici due incompatibili sono Electron e Tauri *fra loro*, ed è giusto: sono due risposte alla stessa domanda.

I due rami di riferimento

```
git switch web/nextjs     # la vetrina in web/, e un secondo commit: singleton/
git switch web/astro      # il blog in blog/, e nient'altro di diverso

# oppure, da un clone pulito di master, la trasformazione completa:
node scripts/setup-nextjs.mjs    # create-next-app, i file, install, build
node scripts/setup-astro.mjs     # create astro, astro add, i file, build

git diff master..web/astro -- frontend/    # l'unico file toccato nel negozio
cd singleton && npm install && npm run db:push && npm run db:seed && npm run dev
                           # database suo, porta 3000: non pesta i piedi a nessuno
```

- Il messaggio di ogni commit di merge racconta tutto ciò che è stato fatto. Leggetelo, è la sintesi della lezione.
- I due script stampano ogni comando esterno prima di eseguirlo, e sono **idempotenti**: potete rilanciarli senza rompere niente.
- `setup-nextjs.mjs` controlla che l'API risponda prima di compilare: le pagine statiche nascono dai dati.
- Documentazione: `docs/WEB-NEXTJS.md` e `docs/WEB-ASTRO.md`, con gli esperimenti nell'ordine giusto e i problemi noti.

Laboratorio 4a

Laboratorio: La vetrina Next, e poi misurarla

1. Da un clone pulito: `node scripts/setup-nextjs.mjs` con il backend avviato. Leggere l'elenco di `next build` e dire, riga per riga, perché quella pagina è statica o dinamica.
2. `node scripts/confronto-seo.mjs`: confrontare SPA e vetrina sullo stesso articolo. Poi Lighthouse su entrambe, da telefono.
3. Cambiare un prezzo dall'admin, verificare che la vetrina mostri ancora quello vecchio, invalidare con `POST /api/revalidate` e ricaricare.
4. **Chiudere il cerchio**: far chiamare la rotta di invalidazione dal backend dentro `PUT /api/products/:id`, con il segreto in `.env`.
5. Aggiungere la **paginazione** al catalogo e una pagina `/cerca` con il suo `noindex`.
6. Validare il JSON-LD di una scheda col Rich Results Test di Google.
7. Avviare `singleton/`: dov'è finita la validazione dell'API, dov'è la sessione, e cosa dovrebbe cambiare nell'Electron del modulo 2 per parlare con *questa* versione?

Laboratorio 4b

Laboratorio: Il blog Astro, e il confine fra statico e dinamico

1. `node scripts/setup-astro.mjs`, poi `npm run build`: servire `dist/` senza Node (`npx serve dist`).
2. Scrivere un settimo post che cita due articoli veri. Prima rompere il frontmatter di proposito, per vedere la build fermarsi.
3. Cambiare un prezzo dal negozio e ricaricare il post *senza* rifare la build: osservare l'isola. Poi disattivare JavaScript.
4. Contare gli `<script>` nella home del blog e in una pagina della SPA, e spiegare la differenza.
5. Una copertina ai post con `astro:assets`.
6. **Togliere l'API al blog**: far leggere a `lib/shop.ts` il database con Drizzle durante la build. Poi dieci righe: quale strumento per quale pezzo di questo negozio, e quale delle due versioni si rompe prima?

Domande di verifica

Domande di verifica

- Cos'è l'idratazione, e perché un Server Component non può usare `useState`?
- Cosa cambia fra `client:load` e `client:visible`?
- Cosa fa `'use cache'`, e cosa fa `revalidateTag`?
- Perché una pagina filtrata dichiara `noindex` ma non `nofollow`?
- Come si deduce `offers.availability` dai dati di minicommerce?
- Perché il carrello resta nella SPA e non entra nella vetrina?
- Perché un'immagine base64 non viene ottimizzata da `next/image`?
- Perché il prezzo in un post del blog può essere vecchio, e cosa lo corregge?
- `singleton/` non ha API REST: chi, nel repository, se ne accorgerebbe per primo?

Sommario del modulo

6. App native: Windows, Linux, macOS, Android, iOS
6.1 Perché tornare al nativo
6.2 Architettura comune
6.3 Apple: iOS e macOS con SwiftUI
6.4 Android con Kotlin e Jetpack Compose
6.5 Windows
6.6 Linux
6.7 Cross-platform nativo
6.8 Guidare un agente su una piattaforma nativa

Che cosa vuol dire “nativo”

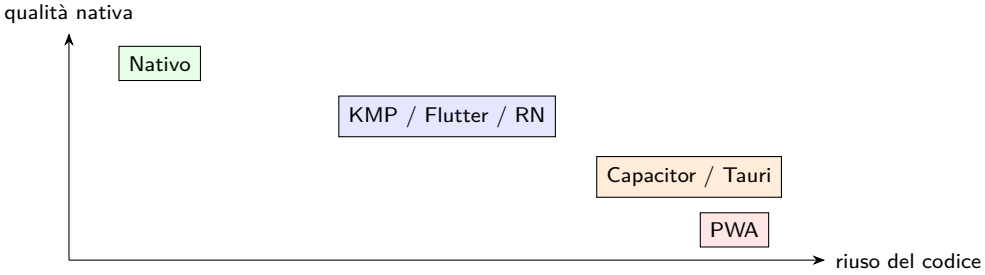
- Un’app scritta con il **linguaggio e il toolkit UI della piattaforma**: Swift/SwiftUI su Apple, Kotlin/Compose su Android, C#/WinUI su Windows, GTK o Qt su Linux.
- Niente browser, niente webview: i widget sono quelli del sistema, disegnati dal sistema.
- L’app è compilata per il processore: parte in decimi di secondo, usa poca memoria.
- Ha accesso a *tutto*: sensori, background, widget, Siri/Assistant, Watch, Auto, accessibilità di sistema.

I vantaggi, concretamente

- **Prestazioni**: liste da 10.000 articoli fluide, animazioni a 120 Hz, avvio istantaneo.
- **Batteria**: niente motore JS che gira; il sistema ottimizza per voi.
- **Integrazione**: widget, notifiche ricche, condivisione, scorciatoie, drag and drop tra app, menu nativi.
- **Accessibilità**: VoiceOver/TalkBack/Narrator funzionano subito e bene.
- **Feeling**: gesti, transizioni, tastiera, selezione testo: tutto "giusto" senza sforzo.
- **Offline e background**: sincronizzazione, geofencing, Bluetooth in background.
- **Hardware**: lettori di codici a barre, stampanti di etichette, bilance, cassetti portasoldi.
- **Sicurezza**: sandbox del sistema, keychain, biometria integrate.

Il prezzo che si pagava (fino a ieri)

- **Un codice per piattaforma**: 3–5 codebase da scrivere, testare, mantenere in sincronia.
- **Un linguaggio per piattaforma**: Swift, Kotlin, C#, e persone che li conoscono.
- Ogni funzionalità nuova costa N volte; ogni bug di logica va corretto N volte.
- Le aziende piccole non potevano permetterselo: da qui Electron, Capacitor, React Native, Flutter.



Cosa è cambiato nel 2026: gli agenti di programmazione

- Un agente (Claude Code, Codex, Copilot agent, Cursor, Gemini CLI) scrive, compila, esegue i test e corregge **da solo**, in un linguaggio che voi *non* conoscete a fondo.
- Riprodurre la pagina del catalogo di minicommerce in tre toolkit diversi è un lavoro **meccanico e ben specificato**: esattamente ciò in cui gli agenti eccellono.
- Il costo marginale della "N-esima piattaforma" è crollato. Il costo che resta è **specificare bene** e **verificare** su dispositivo.
- Conseguenza: il compromesso "web dentro una webview per risparmiare" non è più scontato. Per molte app, il nativo torna a essere la scelta economica.

Attenzione

"Economico" non vuol dire "gratis": serve qualcuno che sappia *giudicare* un'app nativa. È il vostro ruolo.

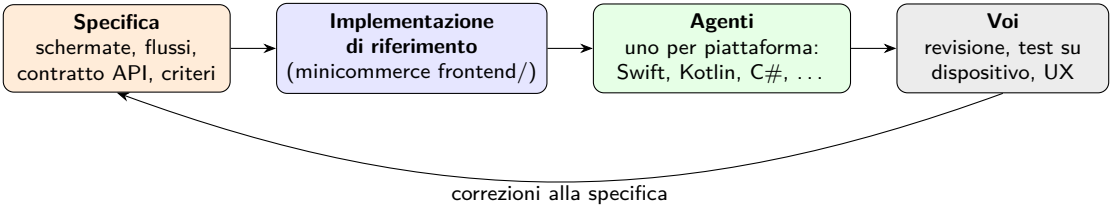
Quattro app native per minicommerce

- **Android, magazzino**: lettore di codici a barre per trovare l'articolo, rettifica di `stock`, foto dell'articolo con la fotocamera. Usata in piedi, con una mano, su un telefono economico.
- **iOS, agente di vendita**: catalogo e scheda articolo, prezzi e disponibilità, carrello del cliente per preparare l'ordine, funziona anche senza rete in capannone.
- **Windows, cassa**: lettore USB (che "digita" il codice), stampante di etichette, scontrino, tastiera senza mouse.
- **Linux, chiosco in negozio**: schermo fisso che mostra il catalogo a schermo intero, nessuna tastiera, riavvio automatico, aggiornamento via Flatpak.

Il filo comune

Tutte e quattro parlano con lo **stesso backend Express sulla porta 3001**. Il server non cambia: cambia solo il client. Stesso campo `isAdmin`, quindi stessi due ruoli: l'admin gestisce il catalogo, il cliente usa carrello e ordine.

Il nuovo flusso di lavoro

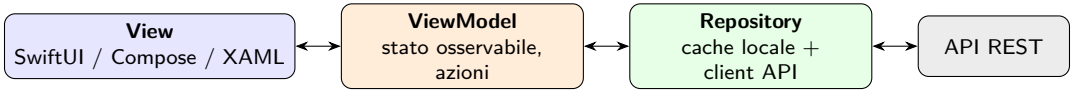


- La **specifica** è il vero codice sorgente: ciò che non c'è scritto, l'agente lo inventa.
- `frontend/src/pages/CatalogPage.tsx` resta l'implementazione di riferimento: è quella che capite meglio e da cui l'agente "traduce".
- Cosa condividere davvero: il riferimento completo delle rotte in `docs/API.md`, le convenzioni di `AGENTS.md`, testi, casi di prova. *Non* il codice UI.

Cosa resta difficile (e vostro)

- **Giudicare la UX**: un'app Android che si comporta come iOS è sbagliata anche se compila.
- **Testare su dispositivi reali**: il lettore di codici a barre e la stampante di etichette non esistono nell'emulatore.
- **Le linee guida degli store**: revisione Apple, privacy, permessi (fotocamera!). Un agente non sa cosa vi rifiuteranno.
- **Verificare le API usate**: gli agenti a volte usano API deprecate o inventate. Compilare è il primo test, non l'ultimo.
- **La parità tra piattaforme**: una checklist per funzionalità, aggiornata a ogni release.

Lo stesso disegno su tutte le piattaforme



- Il pattern **MVVM** è la lingua franca: SwiftUI, Compose e WinUI lo assumono.
- È lo stesso schema del frontend web: pagina $\leftrightarrow$ query/stato $\leftrightarrow$ `lib/api.ts`.
- Il `Repository` nativo è il gemello di `lib/api.ts`: un solo punto che conosce gli URL, il `Bearer` e i tipi.
- I **design token** (colori, spaziature) si esportano in un JSON e ogni piattaforma li legge.

Il contratto genera i client (esercizio)

- – Nel repository di minicommerce **non c'è un** `openapi.yaml`: il contratto è scritto in prosa in `docs/API.md` (e in forma ridotta nella tabella del `README.md`).
- \+ Tradurlo in OpenAPI è il primo esercizio: una ventina di rotte, cinque schemi (`Product`, `Category`, `AuthUser`, `Cart`, ordine finto), i parametri di `GET /api/products`. Poi:

```
# openapi.yaml nella root del repository; nessuna installazione globale, solo npx
OGEN="npx @openapitools/openapi-generator-cli generate -i openapi.yaml"
$OGEN -g swift6           -o clients/swift
$OGEN -g kotlin           -o clients/kotlin
$OGEN -g csharp           -o clients/csharp
$OGEN -g typescript-axios -o clients/ts     # anche per il web!
```

- Modelli tipizzati, un metodo per ogni endpoint, autenticazione gestita. Se il contratto cambia, si rigenera e **il compilatore** vi dice cosa si è rotto in ogni app.
- Alternativa più leggera (quella delle prossime slide): scrivere a mano i modelli `Codable`/`@Serializable` e usare il client HTTP standard.

Il mondo Apple in una slide

- Linguaggio **Swift**, UI **SwiftUI** (dichiarativa, molto simile a React nel modo di pensare), IDE **Xcode** (solo macOS).
- Un solo progetto può compilare per iOS, iPadOS, macOS, watchOS, visionOS: **multipiattaforma Apple** con lo stesso codice UI.
- Concorrenza moderna con `async/await` e `actor`; rete con `URLSession`; JSON con `Codable`.
- Anteprime live in Xcode: si vede la UI cambiare mentre si scrive.
- Distribuzione: TestFlight $\rightarrow$ App Store; su Mac anche fuori dallo store con notarizzazione.

SwiftUI: i tipi di minicommerce e il login

```swift
struct Product: Codable, Identifiable {
    let id: Int
    var name: String; var category: String; var brand: String
    var price: Double; var stock: Int
}
struct AuthUser: Codable { let id: Int; let email: String; let name: String; let isAdmin: Bool }
struct LoginResponse: Codable { let token: String; let user: AuthUser }

actor ApiClient {
    let base = URL(string: "http://192.168.1.10:3001/api")!   // configurabile, non cablato
    var token: String?

    func login(email: String, password: String) async throws -> AuthUser {
        var req = URLRequest(url: base.appending(path: "auth/login"))
        req.httpMethod = "POST"
        req.setValue("application/json", forHTTPHeaderField: "Content-Type")
        req.httpBody = try JSONEncoder().encode(["email": email, "password": password])
        let (data, _) = try await URLSession.shared.data(for: req)
        let res = try JSONDecoder().decode(LoginResponse.self, from: data)
        token = res.token            // in produzione: Keychain, non UserDefaults
        return res.user
    }
}
```

SwiftUI: la chiamata al catalogo

```swift
extension ApiClient {
    // GET /api/products?search=&category=&sort=name&order=asc
    func products(search: String, category: String?) async throws -> [Product] {
        var c = URLComponents(url: base.appending(path: "products"), resolvingAgainstBaseURL: false)!
        var q = [URLQueryItem(name: "sort", value: "name"), URLQueryItem(name: "order", value: "asc")]
        if !search.isEmpty { q.append(URLQueryItem(name: "search", value: search)) }
        if let category { q.append(URLQueryItem(name: "category", value: category)) }
        c.queryItems = q
        var req = URLRequest(url: c.url!)
        if let token { req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization") }
        let (data, resp) = try await URLSession.shared.data(for: req)
        guard (resp as? HTTPURLResponse)?.statusCode == 200 else { throw ApiError.http }
        return try JSONDecoder().decode([Product].self, from: data)
    }
}
```

Notate: tipi, `async/await`, decodifica JSON, gli stessi parametri di query del web. Se leggete TypeScript, leggete Swift.

SwiftUI: ViewModel e lista articoli

```swift
@Observable final class CatalogViewModel {
    var products: [Product] = []; var loading = false; var error: String?
    var search = ""; var category: String?
    let api: ApiClient                          // init omesso

    func load() async {
        loading = true; defer { loading = false }
        do { products = try await api.products(search: search, category: category) }
        catch { error = "Errore di rete" }
    }
}

struct CatalogView: View {
    @State var vm: CatalogViewModel
    var body: some View {
        List(vm.products) { p in
            VStack(alignment: .leading) {
                Text(p.name)
                Text("\(p.brand) - \(p.category)").font(.caption).foregroundStyle(.secondary)
            }
            .badge(p.stock > 0 ? "\(p.stock) pz" : "Esaurito")
        }
        .searchable(text: $vm.search)               // ricerca di sistema, gratis
        .task(id: vm.search) { await vm.load() }    // come useEffect, ma cancellabile
        .refreshable { await vm.load() }            // pull-to-refresh gratis
        .navigationTitle("Catalogo")
    }
}
```

Cosa ottenete gratis su Apple

- `List`: scorrimento, selezione, swipe actions, ricerca (`.searchable`), pull-to-refresh.
- `NavigationStack`: catalogo $\rightarrow$ dettaglio articolo con gesto indietro e stato ripristinabile.
- Dark mode, Dynamic Type (font grandi), VoiceOver, formattazione valuta locale, iPad multi-finestra.
- Keychain per il token JWT, Face ID con due righe, notifiche push via APNs, widget con WidgetKit.
- Lo stesso codice su Mac: barra dei menu, scorciatoie, finestre multiple.

Per l'agente di vendita

Login o registrazione cliente, catalogo con ricerca e filtro categoria, scheda articolo, carrello lato server, cache locale per il capannone senza rete: un pomeriggio con un agente, un Mac e un iPhone.

Il mondo Android in una slide

- Linguaggio **Kotlin**, UI **Jetpack Compose** (dichiarativa, funzioni `@Composable`), IDE **Android Studio** (Windows, Linux, macOS).
- Architettura raccomandata da Google: ViewModel + `StateFlow` + Repository; **Hilt** per la dependency injection.
- Rete: **Retrofit** o **Ktor**; JSON con `kotlinx.serialization`; immagini con Coil; DB locale con Room.
- **Material 3**: componenti pronti, tema dinamico dai colori del wallpaper.
- Distribuzione: Google Play (AAB), ma anche APK diretto o MDM aziendale: per l'app di magazzino è la via normale.

Compose: modello, API e ViewModel

```
@Serializable data class Product(val id: Int, val name: String, val category: String,
                                 val brand: String, val price: Double, val stock: Int)

interface MinicommerceApi {
    @POST("auth/login") suspend fun login(@Body body: LoginRequest): LoginResponse
    @GET("products") suspend fun products(@Query("search") search: String?,
                                          @Query("category") category: String?,
                                          @Query("sort") sort: String = "name"): List<Product>
    @PUT("products/{id}") suspend fun update(@Path("id") id: Int, @Body body: ProductInput): Product
}

@HiltViewModel
class CatalogViewModel @Inject constructor(private val api: MinicommerceApi) : ViewModel() {
    private val _state = MutableStateFlow(CatalogState())
    val state: StateFlow<CatalogState> = _state.asStateFlow()

    fun load(search: String? = null, category: String? = null) = viewModelScope.launch {
        _state.update { it.copy(loading = true, error = null) }
        runCatching { api.products(search, category) }
            .onSuccess { p -> _state.update { it.copy(products = p, loading = false) } }
            .onFailure { _state.update { it.copy(error = "Errore di rete", loading = false) } }
    }
}
```

Compose: la lista del catalogo

```
@Composable
fun CatalogScreen(vm: CatalogViewModel = hiltViewModel()) {
    val state by vm.state.collectAsStateWithLifecycle()
    LaunchedEffect(Unit) { vm.load() }

    Scaffold(topBar = { TopAppBar(title = { Text("Catalogo") }) },
             floatingActionButton = {
                 FloatingActionButton(onClick = { scanBarcode() }) {
                     Icon(Icons.Default.QrCodeScanner, contentDescription = "Scansiona codice")
                 }
             }
    ) { padding ->
        if (state.loading) LinearProgressIndicator(Modifier.fillMaxWidth())
        LazyColumn(Modifier.padding(padding)) {
            items(state.products, key = { it.id }) { p ->
                ListItem(headlineContent = { Text(p.name) },
                         supportingContent = { Text("${p.brand} - ${p.category}") },
                         trailingContent = { Text(if (p.stock > 0) "${p.stock} pz" else "Esaurito") })
            }
        }
    }
}
```

`LazyColumn` = lista virtualizzata; `items(key=)` = la `key` di React; `LaunchedEffect` = `useEffect`. Stesse idee, sintassi diversa: è per questo che un agente le traduce bene.

Cosa ottenete gratis su Android

- Material 3 con tema dinamico, dark mode, TalkBack, dimensioni carattere di sistema.
- **ML Kit Barcode Scanning** e CameraX: lettore di codici a barre e foto dell'articolo in poche righe.
- `WorkManager` per rimandare al server le rettifiche di `stock` fatte offline; `Room` per il catalogo locale.
- Permessi granulari (fotocamera), biometria (`BiometricPrompt`), `EncryptedSharedPreferences` per il token JWT.
- Compose gira anche su desktop e iOS con Compose Multiplatform (prossima sezione).

Confronto con il modulo 3

La stessa foto che lì scattavamo con Capacitor qui si scatta con CameraX: identico il risultato, diverso il costo di ingresso. Oggi l'immagine finisce nel database come data URL base64; l'endpoint di upload multipart resta un esercizio, qui come sul web.

Il mondo Windows in una slide

- Linguaggio **C#** (.NET), UI **WinUI 3** con XAML (Windows App SDK): l'aspetto di Windows 11 (Fluent), Mica, dark mode.
- Alternative vive: **WPF** (enorme base installata nelle aziende), **.NET MAUI** (Windows + macOS + mobile), **Avalonia** (cross-platform, anche Linux), **Uno Platform**.
- IDE: Visual Studio (Community gratuito) o VS Code con C# Dev Kit.
- Pattern: MVVM con `CommunityToolkit.Mvvm` (`[ObservableProperty]`, `[RelayCommand]`).
- Distribuzione: **MSIX** (Microsoft Store o sideload), oppure installer classico; `winget`.

WinUI 3: ViewModel con il toolkit MVVM

```
public record ProductDto(int Id, string Name, string Category, string Brand,
                         double Price, int Stock);

public partial class CatalogViewModel : ObservableObject
{
    private readonly HttpClient _http;          // BaseAddress = http://192.168.1.10:3001/api/
    public ObservableCollection<ProductDto> Products { get; } = new();
    [ObservableProperty] private bool _loading;
    [ObservableProperty] private string? _error;
    [ObservableProperty] private string _search = "";

    public CatalogViewModel(HttpClient http) => _http = http;

    [RelayCommand]
    public async Task LoadAsync()
    {
        Loading = true; Error = null;
        try {
            var url = $"products?sort=name&order=asc&search={Uri.EscapeDataString(Search)}";
            var items = await _http.GetFromJsonAsync<List<ProductDto>>(url) ?? new();
            Products.Clear(); foreach (var p in items) Products.Add(p);
        } catch (HttpRequestException) { Error = "Errore di rete"; }
        finally { Loading = false; }
    }
}
```

WinUI 3: la vista in XAML

```
<Page x:Class="Minicommerce.CatalogPage" xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation">
  <Grid RowDefinitions="Auto,Auto,*">
    <CommandBar Grid.Row="0">
      <AppBarButton Icon="Add" Label="Nuovo articolo" />
      <AppBarButton Icon="Print" Label="Stampa etichetta" />
      <AppBarButton Icon="Refresh" Label="Aggiorna" Command="{x:Bind ViewModel.LoadCommand}" />
    </CommandBar>
    <!-- il lettore USB "digita" il codice qui dentro e poi invia Enter -->
    <TextBox Grid.Row="1" PlaceholderText="Codice o nome articolo"
             Text="{x:Bind ViewModel.Search, Mode=TwoWay}" />
    <ProgressBar Grid.Row="1" IsIndeterminate="True" Visibility="{x:Bind ViewModel.Loading, Mode=OneWay}" />
    <ListView Grid.Row="2" ItemsSource="{x:Bind ViewModel.Products}">
      <ListView.ItemTemplate>
        <DataTemplate x:DataType="local:ProductDto">
          <StackPanel Orientation="Horizontal" Spacing="12">
            <TextBlock Text="{x:Bind Name}" Width="260" />
            <TextBlock Text="{x:Bind Category}" Opacity="0.6" Width="120" />
            <TextBlock Text="{x:Bind Price}" />
            <TextBlock Text="{x:Bind Stock}" Opacity="0.6" />
          </StackPanel>
        </DataTemplate>
      </ListView.ItemTemplate>
    </ListView>
  </Grid>
</Page>
```

Cosa ottenete gratis su Windows

- `x:Bind` = binding compilato e tipizzato: l'equivalente delle props, verificato dal compilatore.
- Fluent Design (Mica, acrilico), dark mode, alto contrasto, Narrator, navigazione da tastiera completa: alla cassa il mouse non si usa.
- Stampa (anche su stampanti di etichette) e lettori USB HID: il sistema li espone, non serve un driver vostro.
- `Windows.Security.Credentials.PasswordVault` per il token; Windows Hello per la biometria.
- Aggiornamenti automatici via MSIX senza scrivere un updater; notifiche toast con azioni.

Il mondo Linux in una slide

- Due toolkit principali: **GTK 4 + libadwaita** (GNOME) e **Qt 6 / QML** (KDE, industria, embedded).
- Linguaggi: C, ma in pratica **Rust** (gtk-rs, Relm4, Slint), **Python** (PyGObject, PySide6), C++ (Qt), Vala.
- Distribuzione universale: **Flatpak** (Flathub) e Snap; pacchetti `.deb`/`.rpm` per distro specifiche; AppImage.
- Linee guida GNOME HIG e KDE HIG: un'app Linux "giusta" segue una delle due.
- Mercato più piccolo, ma: chioschi in negozio, postazioni industriali, PA, sviluppatori, Steam Deck.

GTK 4 con Python: il chiosco del catalogo

```
import gi; gi.require_version('Gtk', '4.0'); gi.require_version('Adw', '1')
from gi.repository import Gtk, Adw, GLib
import threading, requests

API = 'http://192.168.1.10:3001/api'

class CatalogWindow(Adw.ApplicationWindow):
    def __init__(self, app):
        super().__init__(application=app, title='minicommerce', fullscreened=True)
        self.list = Gtk.ListBox(css_classes=['boxed-list'])
        box = Gtk.Box(orientation=Gtk.Orientation.VERTICAL)
        box.append(Adw.HeaderBar()); box.append(Gtk.ScrolledWindow(child=self.list, vexpand=True))
        self.set_content(box)
        threading.Thread(target=self.load, daemon=True).start()   # rete fuori dal thread UI

    def load(self):
        products = requests.get(API + '/products', params={'sort': 'name'}, timeout=5).json()
        GLib.idle_add(self.show, products)                        # torna sul thread UI

    def show(self, products):
        for p in products:
            self.list.append(Adw.ActionRow(title=p['name'],
                                           subtitle=f"{p['brand']} - {p['category']}"))
```

La stessa idea in Rust con Relm4 (estratto)

```rust
#[derive(Debug)] enum Msg { Load, Loaded(Vec<Product>) }

#[relm4::component]
impl SimpleComponent for Catalog {
    view! {
        adw::ApplicationWindow { set_title: Some("minicommerce"),
            gtk::Box { set_orientation: gtk::Orientation::Vertical,
                adw::HeaderBar {},
                #[local_ref] product_list -> gtk::ListBox { add_css_class: "boxed-list" }
            }
        }
    }
    fn update(&mut self, msg: Msg, sender: ComponentSender<Self>) {
        match msg {
            Msg::Load => { let s = sender.clone();
                tokio::spawn(async move { s.input(Msg::Loaded(api::products(None).await)); }); }
            Msg::Loaded(items) => self.products = items,   // la view si aggiorna da sola
        }
    }
}
```

Rust + GTK produce binari da 3 MB senza runtime: ideale per Flatpak e per un chiosco che deve solo accendersi e funzionare. Lo stesso Rust del core di Tauri (modulo 2).

Flatpak: pubblicare il chiosco

File `it.its.Minicommerce.json` (manifest Flatpak, estratto):

```
{ "app-id": "it.its.Minicommerce", "runtime": "org.gnome.Platform", "runtime-version": "48",
  "sdk": "org.gnome.Sdk", "command": "minicommerce",
  "finish-args": ["--share=network", "--socket=wayland", "--socket=fallback-x11"],
  "modules": [{ "name": "minicommerce", "buildsystem": "meson",
                "sources": [{ "type": "dir", "path": "." }] }] }
```

- Sandbox con **permessi dichiarati** (come Tauri e le app mobili): rete, display, notifiche.
- Un unico pacchetto per tutte le distribuzioni; aggiornamenti dal centro software, utile su un chiosco che nessuno tocca.
- Servono anche: file `.desktop`, icona, metadata AppStream (descrizione, screenshot).

Le vie di mezzo: un codice, widget nativi (o quasi)

Tecnologia	Linguaggio	Rendering	Note
React Native / Expo	TypeScript	widget nativi	**la via naturale per chi sa React**; riusate tipi e query di minicommerce; anche web
Flutter	Dart	proprio (Impeller)	pixel identici ovunque, incluso desktop; ottime prestazioni
Kotlin Multiplatform	Kotlin	Compose Multiplatform o UI nativa	logica condivisa, UI Compose anche su iOS; spinto da Google e JetBrains
.NET MAUI	C#	widget nativi	per team .NET; Windows, macOS, iOS, Android

- Tutte producono **vere app native** (niente webview), con un solo codice.
- Il prezzo: uno strato in mezzo che va aggiornato e che a volte non espone l'ultima API della piattaforma.

React Native con Expo: il catalogo

```
npx create-expo-app@latest minicommerce-mobile --template blank-typescript
cd minicommerce-mobile && npm install && npx expo start   # QR code -> Expo Go sul telefono
```

```
import { FlatList, Text, Pressable } from 'react-native';
import { useProducts } from './lib/useProducts';   // lo STESSO hook del modulo 1
import { fetchProducts, formatPrice, type Product } from './lib/api';

export function CatalogScreen({ search }: { search: string }) {
  const { data, loading, reload } = useProducts({ search, sort: 'name', order: 'asc' });
  return (
    <FlatList data={data} keyExtractor={(p: Product) => String(p.id)} onRefresh={reload}
      refreshing={loading} renderItem={({ item }) => (
        <Pressable style={{ padding: 16, borderBottomWidth: 1 }} onPress={() => openProduct(item.id)}>
          <Text style={{ fontSize: 16 }}>{item.name}</Text>
          <Text style={{ opacity: 0.6 }}>{item.brand} - {item.category}</Text>
          <Text>{formatPrice(item.price)} - {item.stock > 0 ? item.stock + ' pz' : 'Esaurito'}</Text>
        </Pressable>
      )} />
  );
}
```

Niente HTML: `Text` e `FlatList` diventano widget nativi. Tipi, query e client API: **gli stessi file** del web.

Flutter: la stessa schermata

```
class CatalogScreen extends StatelessWidget {
  final List<Product> products;
  const CatalogScreen({super.key, required this.products});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catalogo')),
      body: ListView.builder(
        itemCount: products.length,
        itemBuilder: (context, i) => ListTile(
          title: Text(products[i].name),
          subtitle: Text('${products[i].brand} - ${products[i].category}'),
          trailing: Text(formatPrice(products[i].price)),
          onTap: () => context.push('/products/${products[i].id}'),
        ),
      ),
    );
  }
}
```

Flutter disegna *tutto* da solo: perfetto per UI molto personalizzate e per un chiosco identico su ogni schermo, meno per chi vuole il look esatto di ogni OS.

Come scegliere nel 2026

1. **Team React, app mobile "normale"**: React Native + Expo. Riusate query, tipi, logica; UI nativa.
2. **Serve il massimo su una piattaforma** (magazzino con lettore di codici a barre, cassa con stampante): nativo puro, con l'agente per le altre piattaforme.
3. **UI molto grafica e identica ovunque** (chiosco, brand forte): Flutter.
4. **Team Kotlin/Android che deve fare iOS**: Kotlin Multiplatform.
5. **Gestionale interno con budget minimo**: PWA/Capacitor (modulo 3) restano validi.

La domanda da fare sempre

Chi mantiene questo codice tra tre anni, e con quali strumenti? Gli agenti spostano la risposta verso il nativo; il team la decide.

La specifica per l'agente

```
# SPEC-native.md (estratto)
## Obiettivo
App Android nativa (Kotlin, Jetpack Compose, Material 3) per il magazzino di minicommerce.
## Riferimenti nel repository
- docs/API.md       -> contratto completo: rotte, parametri, codici di errore
- README.md         -> funzionalita', modello dati, credenziali demo
- AGENTS.md         -> convenzioni del progetto (stile, naming, comandi)
- frontend/src/pages/CatalogPage.tsx -> implementazione di riferimento del catalogo
- frontend/src/lib/api.ts            -> tipi Product/Cart e parametri dei filtri
## Contratto API
Base URL configurabile (BuildConfig), default http://10.0.2.2:3001/api. Auth: Bearer da
POST /api/auth/login oppure /api/auth/register. Errori: JSON { "error": "..." }.
GET /api/products restituisce l'elenco completo: nessuna paginazione lato server.
## Schermate (parita' con il web)
1. Login e registrazione cliente (401 "Credenziali non valide" visibile nella UI)
2. Catalogo (GET /api/products: ricerca, filtro categoria, ordinamento nome|price|stock)
3. Scheda articolo (nome, marca, categoria, prezzo, disponibilita', descrizione)
4. Carrello cliente (/api/cart e /api/cart/items) e ordine finto (POST /api/orders)
5. Admin: crea/modifica/elimina articolo, foto da fotocamera, conferma prima di eliminare
## Requisiti non funzionali
- ViewModel + StateFlow + Repository; Retrofit + kotlinx.serialization
- Token in EncryptedSharedPreferences; nessun segreto nel repo; ./gradlew assembleDebug
## Cosa NON fare
- Niente webview. Niente endpoint inventati. Niente API deprecate (verificare con lint).
```

Il ciclo di lavoro con l'agente

1. **Specifica** (slide precedente) + accesso al repository di minicommerce come riferimento.
2. Chiedere un **piano** prima del codice: struttura del progetto, librerie, schermate in ordine.
3. Una schermata alla volta: *implementa, compila, esegui i test, fai uno screenshot*.
4. Voi: aprite l'app sull'emulatore/dispositivo, confrontate con la checklist di parità, annotate le differenze.
5. Le correzioni tornano nella **specifica**, non solo nel prompt: la specifica serve anche per iOS e Windows.
6. Revisione finale del codice: architettura rispettata? segreti? permessi minimi? endpoint realmente esistenti?

Checklist di parità tra piattaforme

Funzionale

- ☐ Login, registrazione cliente, logout, 401
- ☐ Catalogo: ricerca e filtro categoria
- ☐ Ordinamento nome/prezzo/disponibilità
- ☐ Scheda articolo completa
- ☐ Carrello cliente: aggiunta, quantità, svuota
- ☐ Ordine dimostrativo e carrello svuotato
- ☐ Admin: crea, modifica, elimina, foto
- ☐ Stati: caricamento, errore, vuoto

Piattaforma

- ☐ Segue le linee guida (HIG / Material / Fluent / GNOME)
- ☐ Dark mode e font di sistema
- ☐ Screen reader: ogni pulsante etichettato
- ☐ Prezzi formattati in euro, locale it-IT
- ☐ Token in storage sicuro
- ☐ Permessi minimi, motivati
- ☐ Build riproducibile da riga di comando

Il carrello è solo del cliente: con un token admin `GET /api/cart` risponde 403. Anche questo fa parte della parità.

Laboratorio 5

Laboratorio: minicommerce nativa su una piattaforma a scelta

1. Scegliere: Android (Kotlin/Compose, consigliata: gira su Windows), Windows (C#/WinUI 3), Linux (GTK4), iOS/macOS (solo con Mac) o React Native/Expo.
2. Scrivere `SPEC-native.md` di una pagina citando `docs/API.md` e `CatalogPage.tsx`; con un agente: login + catalogo con ricerca e filtro + scheda articolo, provati su emulatore o dispositivo. Chi arriva in fondo aggiunge il carrello del cliente.
3. Checklist di parità; correggere almeno tre difformità trovate *leggendo il codice*.
4. Relazione di una pagina: cosa l'agente ha fatto bene, cosa avete corretto, tempo impiegato rispetto al modulo 1.

Domande di verifica

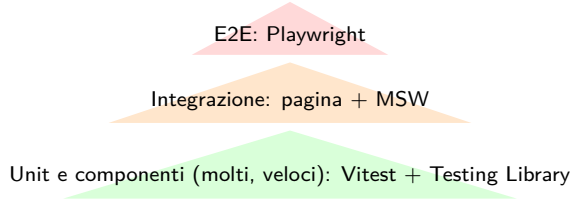
Tre vantaggi concreti di un'app nativa rispetto a una webview? Cosa condividiamo tra piattaforme e cosa no? Quando *non* sceglieresti il nativo per minicommerce?

Sommario del modulo

7. Qualità: testing, accessibilità, prestazioni, sicurezza
7.1 Testing
7.2 Accessibilità
7.3 Formattazione e lint
7.4 Prestazioni
7.5 Sicurezza del frontend

Perché testare il frontend

- Oggi **minicommerce non ha nemmeno un test**, né un test runner configurato: partiamo da zero. È la condizione normale di un progetto giovane.
- Il frontend cambia **più spesso** del backend: ogni modifica alla UI può rompere un flusso.
- Gli agenti di programmazione scrivono codice velocissimo: i test sono il modo per **fidarsi** senza rileggere tutto.
- Regola: testate il **comportamento** che l'utente vede, non l'implementazione.



Installare Vitest nel progetto

```
cd frontend
npm install -D vitest jsdom
npm install -D @testing-library/react @testing-library/user-event @testing-library/jest-dom
```

```
// vite.config.ts: si aggiunge una sezione test a quella che c'e' gia'
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: { alias: { '@': path.resolve(import.meta.dirname, './src') } },
  test: { environment: 'jsdom', setupFiles: ['./src/test/setup.ts'], globals: true },
});

// package.json
"scripts": { "test": "vitest", "test:run": "vitest run" }
```

`npm test` resta in ascolto e riesegue solo i test toccati. L'alias `@` è già configurato: i test lo usano come il resto del codice.

Primo test: la validazione dell'articolo

```
// validateForm oggi vive DENTRO ProductFormDialog e non e' esportata: il primo
// esercizio e' estrarla in src/lib/validazione.ts, cosi' si puo' testare da sola.
// (Dopo il passaggio a react-hook-form + zod del modulo 1, l'unita' da testare
//  diventa lo schema zod: gli stessi casi, un'altra implementazione.)
import { describe, expect, it } from 'vitest';
import { EMPTY_FORM, validateForm } from './validazione';

const valido = { ...EMPTY_FORM, name: 'Mouse MX Master', categoryId: '3', price: '49.90' };

describe('validateForm', () => {
  it('accetta un articolo valido', () => {
    expect(validateForm(valido)).toEqual({});
  });
  it('rifiuta il nome vuoto', () => {
    expect(validateForm({ ...valido, name: '   ' }).name).toMatch(/obbligatorio/i);
  });
  it('rifiuta un nome oltre 32 caratteri', () => {
    expect(validateForm({ ...valido, name: 'x'.repeat(33) }).name).toMatch(/32/);
  });
  it('rifiuta un prezzo negativo', () => {
    expect(validateForm({ ...valido, price: '-1' }).price).toBeDefined();
  });
  it('vuole la categoria', () => {
    expect(validateForm({ ...valido, categoryId: '' }).categoryId).toBeDefined();
  });
});
```

Il primo test trova già un difetto

- La regola dice **massimo 32 caratteri** per il nome, ma l'input ha `maxLength={64}`: l'utente può scriverne 64 e scopre l'errore solo dopo.
- Nessuna delle due è sbagliata in assoluto: il punto è che **erano in disaccordo** e nessuno lo aveva notato.
- Scrivere il test ha costretto a leggere la regola e a decidere. Capita quasi sempre.
- Altre cose da estrarre e testare subito: `getErrorMessage` e `formatPrice` in `lib/api.ts`, pure e già isolate.
- `formatPrice(49.9)` su locale `it-IT` dà `49,90 €`: un test lo blocca prima che qualcuno "semplifichi" in dollari.

Testing Library: il form come lo usa un utente

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { ProductFormDialog } from './ProductFormDialog';

it('segnala il nome troppo lungo e non salva', async () => {
  const onSaved = vi.fn();
  render(<ProductFormDialog open product={null} onOpenChange={vi.fn()} onSaved={onSaved} />);

  await userEvent.type(screen.getByLabelText(/nome/i), 'T'.repeat(33));
  // qui solo il campo Nome e' "touched": c'e' un solo alert nella pagina
  expect(await screen.findByRole('alert')).toHaveTextContent(/32 caratteri/);

  await userEvent.click(screen.getByRole('button', { name: /salva/i }));
  expect(onSaved).not.toHaveBeenCalled();
});
```

- Si cerca per **ruolo e testo** (`getByRole`, `getByLabelText`), mai per classe CSS.
- Il dialog chiama `fetchCategories()` all'apertura: senza una finta API il test fallisce con un errore di rete. Serve MSW.

MSW: una finta API che parla come la vera

```ts
// src/mocks/handlers.ts
import { http, HttpResponse } from 'msw';

const CATEGORIE = [{ id: 1, name: 'Mouse', description: '', productCount: 1, createdAt: '2026-01-01' }];
const MOUSE = {
  id: 7, name: 'Mouse MX Master', description: 'Wireless', categoryId: 1, category: 'Mouse',
  brand: 'Logitech', price: 49.9, stock: 3, imageUrl: null, createdAt: '2026-01-01',
};

export const handlers = [
  http.get('/api/categories', () => HttpResponse.json(CATEGORIE)),
  http.get('/api/products', ({ request }) => {
    const search = new URL(request.url).searchParams.get('search') ?? '';
    const righe = [MOUSE].filter((p) => p.name.toLowerCase().includes(search.toLowerCase()));
    return HttpResponse.json(righe);   // la vera API risponde con un array, non un oggetto
  }),
];

// src/test/setup.ts
import '@testing-library/jest-dom/vitest';
import { server } from '../mocks/server';
beforeAll(() => server.listen()); afterEach(() => server.resetHandlers()); afterAll(() => server.close());
```

Gli stati di CatalogPage

```tsx
// src/test/render.tsx -- router in memoria + AuthProvider, nuovi per ogni test
export function renderApp(path: string) {
  return render(
    <MemoryRouter initialEntries={[path]}><AuthProvider><App /></AuthProvider></MemoryRouter>,
  );
}

it('caricamento, poi dati', async () => {
  renderApp('/');
  expect(screen.getByText(/caricamento/i)).toBeInTheDocument();
  expect(await screen.findByText('Mouse MX Master')).toBeInTheDocument();
});

it('stato vuoto quando il filtro non trova niente', async () => {
  renderApp('/');
  await userEvent.type(screen.getByPlaceholderText(/cerca/i), 'zzz');
  expect(await screen.findByText(/nessun articolo trovato/i)).toBeInTheDocument();
});
```

Il quarto stato, l'**errore**, oggi non è testabile: `CatalogPage` lo mostra solo con un toast e la lista cade nello stato vuoto, indistinguibile da "nessun risultato". È il difetto che il modulo 1 corregge con uno stato di errore vero.

Playwright: il flusso completo nel browser vero

```
npm init playwright@latest   # installa i browser, crea playwright.config.ts e tests/
```

```
import { expect, test } from '@playwright/test';

test('l amministratore crea un articolo e lo trova in catalogo', async ({ page }) => {
  await page.goto('/login');
  await page.getByLabel('Email').fill('admin@minicommerce.dev');
  await page.getByLabel('Password').fill('admin123');
  await page.getByRole('button', { name: 'Accedi' }).click();
  await expect(page).toHaveURL('/admin');

  await page.getByRole('button', { name: 'Nuovo articolo' }).click();
  await page.getByLabel('Nome *').fill('Mouse di prova');
  await page.getByLabel('Categoria *').click();
  await page.getByRole('option', { name: 'Mouse' }).click();
  await page.getByLabel(/Prezzo/).fill('19.90');
  await page.getByRole('button', { name: 'Salva' }).click();

  await page.getByRole('link', { name: 'Catalogo' }).click();
  await expect(page.getByText('Mouse di prova')).toBeVisible();
});
```

E2E: l'eliminazione, e un test verde che inganna

```
test('e poi lo elimina', async ({ page }) => {
  // window.confirm non e' un elemento della pagina: Playwright lo gestisce a parte
  page.on('dialog', (d) => d.accept());
  await page.goto('/admin');

  await page.getByRole('row', { name: /Mouse di prova/ })
    .getByRole('button', { name: 'Elimina' }).click();

  await expect(page.getByText('Mouse di prova')).toHaveCount(0);
});
```

- Questo test **passa**, e proprio per questo è istruttivo: i pulsanti della tabella hanno solo `title="Elimina"`, e `title` entra nel calcolo del nome accessibile come **ultima risorsa**. `getByRole` lo trova.
- Ma quel nome è un incidente, non una scelta: `title` non compare mai su telefono e diversi screen reader sono configurati per ignorarlo.
- Morale: – **un test verde non certifica l'accessibilità**, e nemmeno axe segnala questo caso. Si vede solo usando l'app, ed è la prossima sezione.

E2E: il cliente, e il carrello vietato agli amministratori

```
test('un cliente si registra e aggiunge al carrello', async ({ page }) => {
  await page.goto('/login');
  await page.getByRole('button', { name: 'Crea account' }).click();   // il tasto che cambia modo
  await page.getByLabel('Nome').fill('Cliente Prova');
  await page.getByLabel('Email').fill(`c${Date.now()}@test.dev`);  // il seed ha solo l admin
  await page.getByLabel('Password').fill('password123');
  await page.getByRole('button', { name: 'Crea account cliente' }).click();
  await page.getByRole('button', { name: 'Aggiungi al carrello' }).first().click();
  await page.getByRole('link', { name: /Carrello/ }).click();
  await expect(page.getByText('Totale')).toBeVisible();
});

test('a un amministratore il carrello e vietato', async ({ request }) => {
  const ADMIN = { email: 'admin@minicommerce.dev', password: 'admin123' };  // dal seed
  const { token } = await (await request.post('/api/auth/login', { data: ADMIN })).json();
  const r = await request.get('/api/cart', { headers: { Authorization: `Bearer ${token}` } });
  expect(r.status()).toBe(403);   // non 401: autenticato, ma non autorizzato
});
```

- Il carrello vive **sul server**, legato all'utente: questo test passa dal database vero, non da MSW. Ogni esecuzione crea un cliente nuovo, perché il seed inserisce solo l'amministratore.
- La UI rimanda l'admin su `/admin`, ma il divieto vero è `requireCustomer` nel backend: **autenticazione e autorizzazione sono due cose diverse**, e il test lo verifica dove conta.

Cosa testare (e cosa no)

Sì

- Logica pura: `validateForm`, `formatPrice`, `getErrorMessage`
- Componenti con comportamento: il form articolo, i filtri del catalogo
- Pagine: i 4 stati (caricamento, errore, vuoto, dati)
- I flussi veri end-to-end: login, CRUD articolo, registrazione, carrello e ordine
- Le regole di autorizzazione: il carrello risponde 403 a un admin
- Ogni bug corretto: prima il test che fallisce, poi la fix

No (o con cautela)

- Snapshot di interi componenti: si rompono a ogni ritocco e nessuno li legge
- Dettagli di implementazione (stato interno, chiamate a hook)
- I componenti di `ui/`: vengono da shadcn e Radix, già testati a monte
- Stile e layout, se non con test visivi dedicati

Accessibilità: non è opzionale

- Il 15–20% delle persone ha una disabilità permanente; tutti ne hanno di temporanee (braccio rotto, sole sullo schermo).
- **European Accessibility Act**: in vigore da giugno 2025 per e-commerce, banche, trasporti, e-book, servizi digitali. minicommerce è un e-commerce: ci rientrerebbe.
- In Italia si aggiunge la **Legge Stanca** per PA e grandi aziende.
- Standard di riferimento: **WCAG 2.2**, livello AA. Quattro principi: Percepibile, Utilizzabile, Comprensibile, Robusto.
- HTML semantico fatto bene copre l'80% del lavoro; i `<div onClick>` lo distruggono.

minicommerce: cosa è già fatto bene

- + Ogni campo ha un `<Label htmlFor>` collegato al suo `id`, nel login e nel form articolo. È la cosa che più spesso manca, e qui c'è.
- + `aria-invalid` sui campi non validi, e sul nome anche `aria-describedby` con `role="alert"` sul messaggio: lo screen reader lo annuncia da solo.
- + L'anteprima immagine ha un `alt` descrittivo; l'input file è `sr-only` ma pilotato da un `<label>` visibile e raggiungibile con Tab.
- + `autoComplete="email"` e `"current-password"` nel login: i gestori di password funzionano.
- + `Dialog` e `Select` vengono da Radix: trappola del focus, Esc, ruoli ARIA e frecce sono già corretti. **Usare una libreria serie è una scelta di accessibilità.**

...e cosa va corretto

```
// 1. Icona sola: title e' un nome di ultima risorsa. AdminPage (modifica,
//    elimina) e CatalogPage (direzione ordinamento). Il carrello fa gia' bene.
<Button size="icon" title="Elimina"><Trash2 /></Button>                    // NO
<Button size="icon" aria-label={`Elimina ${product.name}`}>                // SI
  <Trash2 aria-hidden />
</Button>

// 2. aria-describedby c'e' solo sul campo Nome: gli altri errori non sono collegati
<Input id="p-price" aria-invalid={!!err} aria-describedby={err && 'p-price-error'} />
<p id="p-price-error" role="alert">{err}</p>

// 3. index.html dichiara lang="en" su un'app interamente in italiano
<html lang="it">

// 4. CatalogPage: la Card porta all'articolo con onClick su un <div>:
//    niente Tab, niente Invio, niente annuncio. Serve un vero link.
<Card onClick={() => navigate(`/article/${product.id}`)}>                  // NO
<Link to={`/article/${product.id}`}>{product.name}</Link>                  // SI
```

`title` compare come tooltip solo col mouse, dopo un'attesa, e non esiste su telefono: formalmente conta come nome, in pratica non lo è.

Tastiera e focus

- Tutto ciò che si fa con il mouse si deve poter fare con **Tab, Invio, Spazio, frecce, Esc**. Provate 10 minuti di minicommerce senza toccare il mouse.
- `window.confirm` in `AdminPage`: sposta il focus fuori dalla pagina, non si può stilare né annunciare. Sostituitelo con l'**AlertDialog** deciso nel modulo 1.
- L'anello di focus deve essere **visibile**: mai `outline: none` senza alternativa. I componenti shadcn lo gestiscono: non toglietelo.
- SPA: passando da `/` a `/admin` il focus resta dov'era e lo screen reader non dice niente. Spostatelo sull'`<h1>` della nuova pagina.
- Link "salta al contenuto" come primo elemento, per non ritabulare l'`Header` ogni volta.

Annunciare i risultati del filtro

```
// Nel catalogo il numero di risultati cambia mentre si digita: chi non vede
// lo schermo non lo sa. Un contatore in una regione live risolve.
<p aria-live="polite" role="status" className="text-sm text-muted-foreground">
  {loading ? 'Ricerca in corso...' : `${products.length} articoli trovati`}
</p>
```

- `aria-live="polite"` annuncia quando l'utente fa una pausa; `assertive` interrompe: solo per errori gravi.
- La regione deve **esistere già** nel DOM quando il testo cambia: se la create insieme al messaggio, spesso non viene annunciata.
- I toast di `sonner` sono già regioni live, ma sono **effimeri**: un errore che l'utente deve poter rileggere non va in un toast.

Colori, movimento, testo

- Contrasto minimo **4,5:1** per il testo normale, 3:1 per testo grande e componenti UI.
- Da misurare in minicommerce: il Badge "Esaurito" è `text-destructive` su `bg-destructive/10`, cioè rosso su rosa pallido. Con ogni probabilità non arriva a 4,5:1.
- Mai il colore come *unica* informazione. Qui va bene: "Esaurito" è anche una parola, non solo un badge rosso.
- `prefers-reduced-motion`: disattivare le animazioni non essenziali.
- Testo ridimensionabile al 200% senza rotture: la barra dei filtri ha larghezze fisse (`w-28`, `w-44`) da controllare.

Strumenti

```
npm install -D eslint-plugin-jsx-a11y  # errori a11y mentre scrivete (oxlint ha regole simili)
npm install -D vitest-axe              # regole axe nei test dei componenti
```

```
import { axe } from 'vitest-axe';
it('il form articolo non ha violazioni a11y', async () => {
  const { container } = render(<ProductFormDialog open product={null} ... />);
  expect(await axe(container)).toHaveNoViolations();
});
```

- Estensione **axe DevTools** e scheda Accessibility di Lighthouse: trovano il 30–40% dei problemi, fra cui subito `lang="en"` e i contrasti.
- Il resto si trova **usando** l'app: solo tastiera, poi con **NVDA** (Windows, gratuito) o VoiceOver.
- In E2E: `@axe-core/playwright` per l'audit di ogni pagina.

Un formattatore non è una questione di gusto

```
npm run lint                # oxlint: velocissimo, trova errori ma NON formatta
npm install -D prettier     # oppure: npm install -D --save-exact @biomejs/biome
npx prettier --write "src/**/*.{ts,tsx,css}"
```

- Il progetto usa **oxlint** e non ha Prettier. Risultato: `AGENTS.md` chiede virgolette doppie nel frontend, ma le seguono solo `CatalogPage`, `ArticlePage`, `ProductFormDialog` e `components/ui/`; `App.tsx`, `Header.tsx`, `AddToCartButton`, `lib/` e le altre pagine usano le singole.
- Il costo non è estetico: sono **righe che cambiano senza motivo** nei diff, e conflitti su codice identico.
- Scegliete *uno* strumento, eseguitelo una volta su tutto (un commit solo, "formattazione") e poi bloccatelo con un hook pre-commit: `husky` e `lint-staged`, oppure i git hooks nativi.
- Biome formatta *e* lint in un solo strumento; Prettier formatta e si affianca a oxlint. Entrambe difendibili: la scelta peggiore è non scegliere.

Il problema numero uno: le immagini in base64

```
// ProductFormDialog: la foto dell'articolo diventa una data URL...
const MAX_IMAGE_SIZE = 5 * 1024 * 1024;      // fino a 5 MB per articolo
reader.readAsDataURL(file);                  // "..."

// ...finisce in una colonna TEXT senza limiti...
imageUrl: text('image_url'),

// ...e torna indietro DENTRO OGNI RIGA di GET /api/products
const productSelection = { /* ... */ imageUrl: products.imageUrl };
```

- Base64 aggiunge un terzo: i 16 articoli del seed con una foto da 2 MB ciascuno fanno **oltre 40 MB di JSON** a ogni apertura del catalogo.
- E il catalogo **non mostra nemmeno l'immagine**: la paghiamo senza usarla. La mostrano `ArticlePage` e `CartPage`, che però hanno bisogno di un articolo solo.
- Nessuna cache possibile: il JSON cambia a ogni filtro, il browser riscarica tutto.

Lo stesso difetto è anche un bug

```
// backend/src/server.ts
app.use(express.json());   // limite predefinito del corpo: 100 KB
```

- Il client accetta 5 MB, il server ne accetta 100 KB: una foto normale fa fallire il salvataggio, e l'utente vede un errore generico perché il gestore centralizzato risponde 500.
- – Soluzione sbagliata: `express.json({ limit: '10mb' })`. Sposta il problema e peggiora quello di prestazioni.
- + Soluzione giusta (è il modulo 3): **upload multipart**, il file su disco o su storage, nel database solo il percorso. Poi `<img loading="lazy">` con `width` e `height` e immagini ridimensionate.
- Da aggiungere lato server: controllo del tipo reale del file e della dimensione. La validazione nel browser aiuta l'utente, non difende l'API.

Il secondo problema: nessuna paginazione

```
// backend: GET /api/products fa .all(), cioe' tutto il catalogo, sempre
const rows = selectProducts().where(...).orderBy(...).all();

// Da aggiungere all'API (esercizio): ?limit=24&offset=0
// risposta: { items: Product[], total: number }
// nel frontend bastano i filtri in piu': l'hook rifa' la richiesta da solo
const { data, loading } = useProducts({ ...filtri, limit: 24, offset: pagina * 24 });
```

- Con i 16 articoli del seed non si vede niente. Inserite 5.000 righe nel seed e riprovate: il server ordina tutto, la rete trasporta tutto, React monta 5.000 `Card`.
- Oltre le 200 righe visibili servono **virtualizzazione** (per esempio TanStack Virtual) o paginazione. Meglio la paginazione: risolve anche la rete.

Misurare prima di ottimizzare

- **Lighthouse** (in DevTools, con throttling mobile): punto di partenza, ripetibile.
- **Network** di DevTools: la prima cosa da guardare è il peso della risposta di `/api/products`.
- **Performance panel**: dove va il tempo (script, layout, paint); i *long task* sopra 50 ms.
- **React DevTools Profiler**: quali componenti si ri-renderizzano e perché.
- `npm run build` con `rollup-plugin-visualizer`: cosa c'è nel bundle e quanto pesa.

Budget di prestazioni per minicommerce

JS iniziale < 200 KB compressi; risposta di `/api/products` < 100 KB; LCP $< 2{,}5$ s su 4G; INP < 200 ms.

Le ottimizzazioni che contano davvero

1. **Meno dati**: paginazione, nessun base64 nelle risposte, solo i campi che servono.
2. **Meno JavaScript**: code splitting per rotta (il modulo 1 lo fa su `/admin`), dipendenze leggere, controllate su bundlephobia.
3. **Immagini**: formato moderno, dimensioni giuste, `loading="lazy"`, `width` e `height` sempre per non far saltare il layout (CLS).
4. **Font**: un solo file, `woff2`, `font-display: swap`, subset dei caratteri.
5. **Cache**: asset con hash e `immutable`; un hook che non rifa' la richiesta se i filtri non cambiano, al posto del debounce con `setTimeout` fatto a mano.
6. **Rete**: HTTP/2 o 3, CDN, compressione Brotli, preconnect all'API.

Quando il filtro diventa locale

```
// Col catalogo offline della PWA (modulo 3) il filtro gira nel browser:
// su liste grandi la digitazione comincia a scattare.
const [q, setQ] = useState('');
const qDiff = useDeferredValue(q);            // React aggiorna la lista "con calma"
const visibili = useMemo(() => filtra(articoli, qDiff), [articoli, qDiff]);

// Lavoro pesante davvero (l'esportazione CSV del modulo 2): fuori dal main thread
const worker = new Worker(new URL('./csv.worker.ts', import.meta.url), { type: 'module' });
worker.postMessage(articoli);
worker.onmessage = (e) => scarica(e.data);
```

- Vite gestisce i worker come moduli: TypeScript e import funzionano anche lì.
- Perdite di memoria tipiche: `setTimeout` e listener senza pulizia negli effetti. Il debounce di `CatalogPage` lo fa bene (`return () => clearTimeout(timer)`): copiate quel pattern.

Il modello mentale

- Il codice frontend gira **sulla macchina dell'utente**: chiunque può leggerlo, modificarlo, chiamare l'API a mano.
- Quindi: **nessun segreto nel client**, **nessun controllo di sicurezza solo nel client**. L'API valida e autorizza sempre.
- minicommerce su questo è corretto: `AdminPage` nasconde la UI, ma è `requireAdmin` nel backend che blocca davvero POST, PUT e DELETE; e il carrello è chiuso dietro `requireCustomer`, che risponde **403** a un amministratore.
- Il frontend protegge *l'utente* (da script malevoli, da siti che lo impersonano), non l'API.
- Le minacce principali: **XSS**, furto di sessione, CSRF, dipendenze compromesse, configurazioni sbagliate.

XSS: come nasce e come si evita

```
// React fa l'escape di tutto cio' che interpola: questo e' SICURO
<p>{product.description}</p>   // "<img onerror=...>" viene mostrato come testo

// Questo NO: HTML grezzo dal server o dall'utente
<div dangerouslySetInnerHTML={{ __html: product.descriptionHtml }} />

// Se dovete davvero renderizzare HTML (descrizioni in markdown), sanificate
import DOMPurify from 'dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(html) }} />

// Altro ingresso da sorvegliare: imageUrl arriva dal database, non da voi
<img src={product.imageUrl} />   // "javascript:..." o un URL di tracciamento esterno
```

- Oggi minicommerce non usa `dangerouslySetInnerHTML`: +. Mantenete l'abitudine.
- Un XSS riuscito legge `localStorage` (il token!) e fa richieste come l'utente. Difesa in profondità: **Content Security Policy**.

Content Security Policy

```
# header HTTP inviato dal server che serve la SPA (o <meta http-equiv> in index.html)
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data:;
  connect-src 'self' https://api.minicommerce.example;
  frame-ancestors 'none';
  base-uri 'self'; form-action 'self'
```

- minicommerce oggi **non ha nessuna CSP**: né header, né meta tag in `index.html`.
- Anche se un attaccante inietta `<script>`, con la CSP il browser **non lo esegue**.
- `connect-src` limita a chi il JS può parlare: un token rubato non può essere spedito altrove.
- `img-src data:` serve finché le immagini sono base64: togliendo il base64 la CSP diventa più stretta. Partite in `Report-Only` e guardate cosa rompe.

Sessione: la versione da produzione

- Oggi: JWT da 8 ore in `localStorage`, aggiunto dall'interceptor di richiesta di axios. Semplice, e vulnerabile a qualunque XSS.
- Per il **web puro**: token in cookie `httpOnly; Secure; SameSite=Lax` impostato dall'API. Il JS non lo vede: un XSS non può rubarlo.
- Con i cookie serve difendersi da **CSRF**: `SameSite` già aiuta; per le POST da altri siti l'API richiede un header custom o un token anti-CSRF.
- Il frontend cambia poco: `withCredentials: true` su axios e via l'interceptor del token.
- Per **desktop e mobile**: storage sicuro del sistema (`safeStorage`, Keychain, Preferences), come nei moduli 2 e 3.
- Token brevi (15 min) con refresh: un furto vale poco e per poco. Otto ore sono tante.

CORS, spiegato una volta per tutte

- CORS è una **protezione del browser per l'utente**: impedisce a `sito-cattivo.com` di leggere risposte della vostra API usando i cookie dell'utente.
- **Non** protegge l'API da chiamate dirette: curl, Postman e un'app nativa non hanno CORS.
- L'API dichiara chi può leggerla: `Access-Control-Allow-Origin: https://minicommerce.example`. Mai `*` insieme a credenziali.
- Le richieste "non semplici" (JSON, header custom) fanno prima una `OPTIONS` di preflight: l'API deve risponderle.
- In sviluppo il problema non si vede, perché Vite fa da **proxy**: `/api` è la stessa origine. In produzione i domini sono diversi e CORS torna a contare.

I dati che non volete nel vostro sistema

```
// CartPage: il form del pagamento dimostrativo chiede numero, scadenza e codice...
await placeMockOrder({
  paymentCardholder: cardholder,
  paymentLastFour: cardNumber.slice(-4),     // ...ma al server parte solo questo
});
// backend/src/routes/orders.ts: l'ordine registra solo cio' che e' arrivato
paymentCardholder: paymentCardholder.trim(),
paymentLastFour: paymentLastFour.trim().slice(-4),
```

- `POST /api/orders` registra l'ordine con stato `accepted` e svuota il carrello: nessun circuito di pagamento, nessuna carta validata, nessuna scorta scalata. È una **simulazione**, e la pagina lo dice.
- Numero completo, scadenza e codice di sicurezza **non lasciano mai il browser**: nessuna richiesta li contiene, nessun log li registra, nessuna colonna li conserva. +
- Chi tratta davvero i dati di una carta entra nel perimetro **PCI DSS**: audit, segmentazione di rete, responsabilità legale. Un e-commerce vero ci resta fuori: il pagamento lo gestisce un fornitore (Stripe, PayPal), voi tenete un identificativo.
- **Il dato più sicuro è quello che non raccogliete.**

I difetti reali, nel repository

```
// backend/src/middleware/auth.ts
export const JWT_SECRET = process.env.JWT_SECRET ?? 'dev-secret-change-me';
// Se in produzione la variabile manca, l'app parte comunque: chiunque conosca
// questa riga si firma un token con isAdmin true. Nessun errore, nessun log.

// backend/src/server.ts
app.use(cors());          // Access-Control-Allow-Origin: * per qualunque sito
app.use(express.json());  // nessun helmet, nessuna CSP, nessun header di sicurezza

// backend/src/routes/auth.ts: login e register senza alcun limite ai tentativi.
// "admin123" si trova con qualche migliaio di richieste in un minuto, e la
// registrazione lascia creare account a volonta', senza verifica dell'email.
```

Non sono ipotesi: sono righe che potete aprire e leggere. **Un audit comincia così**, non da uno strumento.

Le correzioni

```
// 1. Nessun segreto con valore di default
if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET mancante');
export const JWT_SECRET = process.env.JWT_SECRET;

// 2. CORS con origini esplicite, prese dalla configurazione
app.use(cors({ origin: process.env.CORS_ORIGIN?.split(',') ?? [], credentials: true }));

// 3. Header di sicurezza e corpo limitato
app.use(helmet());
app.use(express.json({ limit: '1mb' }));

// 4. Limite ai tentativi su login e registrazione
import rateLimit from 'express-rate-limit';
app.use(['/api/auth/login', '/api/auth/register'], rateLimit({ windowMs: 60000, limit: 10 }));
```

E nel seed: la password dell'amministratore da variabile d'ambiente, non `admin123` scritto nel codice.

Dati e script distruttivi

- + Il database è già fuori dal repository: `*.db`, `*.db-shm`, `*.db-wal` e `.env` sono nel `.gitignore`. Verificatelo con `git check-ignore`, non a memoria: un database committato è una fuga di dati permanente.
- − `db:push` usa `-force`: applica anche le modifiche che perdono dati, senza chiedere. Comodo in sviluppo, **inaccettabile** su un database vero.
- − `db:seed` comincia svuotando *tutte* le tabelle, utenti, carrelli e ordini compresi; `db:reset` cancella il file. Nomi innocui, effetti irreversibili.
- In produzione servono **migrazioni** versionate (`drizzle-kit generate` e `migrate`), non `push`. E backup.
- Regola pratica: uno script che distrugge dati va chiuso dietro una variabile d'ambiente, o rinominato in modo che faccia paura.

Dipendenze e catena di fornitura

```
npm audit                          # vulnerabilita' note nelle dipendenze
npm outdated                       # cosa e' vecchio
npm ci    # in CI: esattamente le versioni del lockfile
```

- Ogni pacchetto npm è codice che gira nel vostro build e nel browser dell'utente. Nel 2024–2025 diversi pacchetti popolari sono stati compromessi.
- Guardate le dipendenze di minicommerce: `shadcn` è uno strumento da riga di comando e sta fra le *dependencies* del frontend. Meno pacchetti, meno superficie.
- Regole: **lockfile committato**, aggiornamenti automatici con revisione (Renovate o Dependabot), pacchetti mantenuti e usati da molti.
- `npm install` esegue gli script di installazione dei pacchetti (`postinstall` e simili): è codice arbitrario che gira sul vostro PC. `npm install -ignore-scripts`, o `ignore-scripts=true` nel file `.npmrc` del progetto, lo impedisce.

Checklist di sicurezza per minicommerce

- ☐ `JWT_SECRET` obbligatorio, nessun default
- ☐ CORS con origini esplicite da configurazione
- ☐ `helmet()` e CSP almeno `script-src 'self'`
- ☐ Limite ai tentativi su login e registrazione
- ☐ Corpo limitato, upload validato lato server
- ☐ Autorizzazione per ruolo sul server (`requireAdmin`, `requireCustomer`), non solo UI nascosta
- ☐ Token in cookie httpOnly sul web; storage sicuro su desktop e mobile
- ☐ Token brevi con refresh; logout che invalida
- ☐ Password admin da variabile d'ambiente
- ☐ Migrazioni al posto di `push --force`
- ☐ Nessun dato di carta nel sistema: solo intestatario e ultime quattro cifre
- ☐ `npm audit` in CI, lockfile bloccato, HTTPS ovunque

Riferimento: OWASP Top 10 e i Cheat Sheet OWASP "HTML5 Security" e "Content Security Policy".

Laboratorio 6

Laboratorio: Portare minicommerce a una qualità professionale

1. Test: Vitest; estraete e coprite `validateForm`; form articolo e `CatalogPage` con MSW; un E2E lato admin e uno lato cliente (registrazione, carrello, ordine), più il 403 del carrello.
2. Accessibilità: zero violazioni axe su tutte le pagine; `aria-label` su ogni pulsante con sola icona; `lang="it"`; contrasto di "Esaurito" misurato; 10 minuti di sola tastiera.
3. Prestazioni: peso di `/api/products` prima e dopo aver tolto il base64; Lighthouse mobile ≥ 90; bundle iniziale sotto 200 KB.
4. Sicurezza: chiudete almeno **quattro** voci della checklist, un commit per voce.
5. Formattazione: Prettier o Biome, un commit solo di riformattazione, un hook pre-commit.

Domande di verifica

Perché `title` non basta su un pulsante con sola icona? Cosa blocca una CSP con `script-src 'self'`? CORS protegge l'API o l'utente? Perché il base64 peggiora *ogni* richiesta al catalogo? Perché il carrello risponde 403 e non 401 a un admin? Perché il numero della carta non arriva al backend?

Sommario del modulo

8. Consegna: monorepo, CI/CD, deploy, monitoraggio
8.1 Da due cartelle a un monorepo
8.2 Integrazione continua
8.3 Distribuzione
8.4 Osservare l'app in produzione

Dove siamo: due progetti nello stesso repository

```
minicommerce/
  backend/
    package.json          # minicommerce-backend, typescript ~7.0.2
    package-lock.json
    src/db/schema.ts      # users, categories, products, cart_items, orders
  frontend/
    package.json          # frontend, typescript ~6.0.2
    package-lock.json
    src/lib/api.ts        # Product, Category, AuthUser, Cart, CartItem, MockOrder
```

- Due `npm install`, due lockfile, nessun comando che controlli tutto il progetto.
- Il tipo `Product` esiste **due volte**; `Cart` e `MockOrder` esistono solo nel frontend, perché il backend costruisce quelle risposte a mano. Nessuno garantisce che coincidano.
- Perfino TypeScript è a due versioni diverse: nessuno se ne accorge finché non rompe.
- E stanno arrivando desktop (modulo 2), mobile (modulo 3) e il sito vetrina (modulo 4).

Un workspace npm

```
// package.json -- nuovo, nella root del repository
{ "name": "minicommerce", "private": true,
  "workspaces": ["backend", "frontend", "desktop", "packages/*"] }
```

```
git rm backend/package-lock.json frontend/package-lock.json
npm install                  # una sola node_modules e un solo lockfile, nella root
npm run dev -w frontend      # lanciare uno script di un singolo progetto
npm run build --workspaces   # lo stesso script su tutti i progetti
```

La conversione è un vostro lavoro: oggi il repository ha ancora due `npm install` separati. Le dipendenze comuni (TypeScript, oxlint, Vitest) si dichiarano **una volta** nella root, così la versione è una sola.

packages/shared: i tipi una volta sola

```ts
// packages/shared/src/index.ts -- il contratto HTTP, non le righe del database
export interface Product {
  id: number; name: string; description: string; brand: string;
  categoryId: number; category: string;      // il nome, dalla join
  price: number; stock: number; imageUrl: string | null; createdAt: string;
}
export interface Category { id: number; name: string; description: string;
  productCount: number; createdAt: string }  // productCount lo calcola il backend
export interface AuthUser { id: number; email: string; name: string; isAdmin: boolean }
export interface CartItem { id: number; quantity: number; product: Product; subtotal: number }
export interface Cart { items: CartItem[]; total: number }
export interface MockOrder { id: number; status: 'accepted'; total: number;
  createdAt: string; message: string; items: Array<{ productId: number;
  productName: string; unitPrice: number; quantity: number }>;
  payment: { method: 'mock-card'; cardholder: string; lastFour: string } }
```

```jsonc
{ "name": "@minicommerce/shared", "private": true, "exports": { ".": "./src/index.ts" } }
// in backend e frontend: "@minicommerce/shared": "*"  -- npm lo risolve al workspace locale
```

Chi usa il pacchetto condiviso

```
// frontend/src/lib/api.ts -- le interfacce spariscono da qui
import type { Product, Category, AuthUser, Cart, MockOrder } from '@minicommerce/shared';
export type { Product, Category, AuthUser, Cart, MockOrder };

// backend/src/routes/cart.ts -- oggi { items, total } e' un oggetto costruito a mano
import type { Cart } from '@minicommerce/shared';
const carrello: Cart = { items, total };   // da qui in poi la forma la controlla tsc
res.json(carrello);
```

- Aggiungete un campo a `schema.ts` e lo dimenticate in `shared`? `tsc` fallisce in CI, non in produzione.
- In `shared` va solo ciò che viaggia su HTTP: **niente** Drizzle, niente React, niente Express. Altrimenti il frontend si tira dietro il backend.
- `passwordHash` non entra in `AuthUser`, e in `MockOrder` ci sono solo intestatario e ultime quattro cifre: il tipo condiviso è anche una difesa.

Il monorepo completo: un repository, cinque app

```
minicommerce/
  package.json            # "workspaces": [...]
  turbo.json
  openapi.yaml            # il contratto dell'API (da scrivere: e' un esercizio)
  packages/
    shared/               # @minicommerce/shared: i tipi del contratto
    config/               # tsconfig e configurazione oxlint condivisi
  backend/                # Express 5 + SQLite (il processo che scrive i dati)
  frontend/               # Vite SPA (modulo 1)        -> dist/ statico
    src-tauri/            # Tauri (modulo 2)           -> impacchetta la stessa dist/
    android/ ios/         # Capacitor (modulo 3)       -> impacchetta la stessa dist/
  desktop/                # Electron (modulo 2): guscio accanto al frontend
  web/                    # Next, vetrina (modulo 4)   -> pagine pregenerate
  blog/                   # Astro, blog  (modulo 4)    -> HTML statico
  singleton/              # Next tutto-in-uno (mod. 4) -> alternativa a backend+frontend
```

`singleton/` non è una sesta app: è l'*altra strada*, nel repository per confronto. Chi la sceglie cancella `backend/` e `frontend/`, e con loro il contratto HTTP su cui vivono desktop, mobile e native. Desktop e mobile non sono copie del frontend: ne riusano la `dist/`. L'app nativa del modulo 5 condivide il **contratto HTTP**, non il codice: può stare in un altro repository e generare il suo client da `openapi.yaml`. Una correzione in `shared` o in `lib/piattaforma.ts` arriva a web, desktop e mobile con un solo commit.

Turborepo: un comando, le dipendenze giuste

```
// turbo.json: cosa dipende da cosa, cosa si puo' cachare
{ "tasks": {
    "build":     { "dependsOn": ["^build"], "outputs": ["dist/**"] },
    "typecheck": { "dependsOn": ["^build"] },
    "test":      { "dependsOn": ["^build"] },
    "lint":      {},
    "dev":       { "cache": false, "persistent": true } } }
```

```
// package.json della root
{ "name": "minicommerce", "private": true,
  "scripts": { "dev": "turbo run dev", "lint": "turbo run lint",
    "typecheck": "turbo run typecheck", "build": "turbo run build",
    "test": "turbo run test" } }
```

`turbo run build --filter=frontend...` costruisce `frontend` e ciò da cui dipende. La cache (locale e remota) salta ciò che non è cambiato: toccate solo `blog/` e il resto non si ricompila.

Gli script che la CI deve poter chiamare

```
// frontend/package.json -- dev, build e lint ci sono: typecheck e test no
{ "scripts": { "dev": "vite", "build": "tsc -b && vite build",
    "typecheck": "tsc -b", "lint": "oxlint",
    "test": "vitest run", "test:e2e": "playwright test" } }

// backend/package.json -- qui lint e typecheck MANCANO: vanno aggiunti
{ "scripts": { "dev": "node --watch src/server.ts", "start": "node src/server.ts",
    "typecheck": "tsc --noEmit", "lint": "oxlint", "test": "vitest run",
    "db:push": "drizzle-kit push --force", "db:seed": "node src/seed.ts" } }
```

- Il backend **non ha un build**: Node 24 esegue TypeScript direttamente. Ma senza `tsc --noEmit` nessuno controlla i suoi tipi: oggi un errore si scopre solo quando la rotta va in errore.
- Regola: ogni progetto espone gli stessi nomi di script. Turborepo e la CI non devono conoscere i casi particolari.

GitHub Actions: la pipeline di base

```yaml
# .github/workflows/ci.yml
name: CI
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: npm }
      - run: npm ci
      - run: npx turbo run lint typecheck test build
      - run: npm audit --audit-level=high
      - uses: actions/upload-artifact@v4
        with: { name: frontend-dist, path: frontend/dist }
```

Ogni PR: oxlint, tipi, i test del modulo 6, build e audit delle dipendenze. Se è rosso, non si fa merge. È la rete di sicurezza che rende sostenibile il lavoro con gli agenti (modulo 8).

E2E in CI: serve anche il backend

```yaml
e2e:
  needs: check
  runs-on: ubuntu-latest
  env: { JWT_SECRET: test-secret, DATABASE_URL: /tmp/e2e.db }
  # perche' funzioni, drizzle.config.ts deve leggere DATABASE_URL:
  # oggi ha il percorso scritto a mano ('./minicommerce.db')
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: 24, cache: npm }
    - run: npm ci
    - run: npm run db:push -w backend && npm run db:seed -w backend
    - run: npm run start -w backend &
    - run: npx playwright install --with-deps chromium
    - run: npm run test:e2e -w frontend
    - uses: actions/upload-artifact@v4
      if: failure()
      with: { name: playwright-report, path: frontend/playwright-report }
```

Il database di test è un file usa e getta in `/tmp`: `db:push` e `db:seed` lo ricreano a ogni esecuzione, quindi i test partono sempre dagli stessi dati demo.

Anteprime, segreti, protezione del branch

- **Preview deployment**: Netlify, Vercel e Cloudflare Pages pubblicano il `dist/` di ogni PR a un URL temporaneo, puntato all'API di staging. Il revisore clicca invece di immaginare.
- Segreti del repository (*Settings* → *Secrets*): `JWT_SECRET` di produzione, token di deploy, chiavi di firma, e le chiavi del circuito di pagamento il giorno in cui `POST /api/orders` smetterà di essere finto. Mai nel codice, mai nei log, mai in un file `.env` committato.
- `npm ci` (non `npm install`): la CI installa **esattamente** il lockfile e fallisce se `package.json` e lockfile non concordano. Nessuna sorpresa di versione fra la vostra macchina e il server.
- **Branch protection** su `main`: CI verde obbligatoria, una revisione, niente push diretti.

Il frontend: file statici

```
npm run build -w frontend       # -> frontend/dist: index.html, assets/*.js, *.css

# nella CI, su push in main
- uses: cloudflare/wrangler-action@v3
  with:
    apiToken: ${{ secrets.CF_TOKEN }}
    command: pages deploy frontend/dist --project-name minicommerce
```

- `VITE_API_URL` si decide **al build**: una build per ambiente, oppure un `config.json` letto a runtime se volete un solo artefatto.
- **Fondamentale con BrowserRouter**: ogni percorso sconosciuto deve servire `index.html`, altrimenti `/cart`, `/about` o `/article/7` ricaricati a mano danno 404. Su Pages e Netlify è una riga di configurazione; con nginx è `try_files $uri /index.html`.
- Il blog Astro del modulo 4 segue la stessa strada: `blog/dist` è una cartella di file, si pubblica così. La vetrina Next invece è un **processo Node** da ospitare come il backend, perché le serve un server per rigenerare le pagine su richiesta: con `output: 'export'` tornerebbe statica, ma perderebbe `revalidateTag`.

Il backend: un processo e un file accanto

```
# backend/Dockerfile -- build context: la root del monorepo
FROM node:24-slim                # non alpine: better-sqlite3 e' un modulo nativo
WORKDIR /app
COPY package.json package-lock.json ./
COPY packages ./packages
COPY backend ./backend
RUN npm ci --omit=dev -w backend --include-workspace-root
ENV NODE_ENV=production DATABASE_URL=/data/minicommerce.db
EXPOSE 3001
CMD ["node", "backend/src/server.ts"]
```

- Niente passo di compilazione: Node 24 esegue i `.ts` come sono.
- `/data` deve essere un **volume persistente**. Senza volume, il container riparte vuoto: catalogo, utenti, carrelli e ordini svaniti a ogni deploy.
- Al primo avvio su un volume nuovo servono `db:push` e `db:seed`: un passo del deploy, non magia. Fate leggere `DATABASE_URL` anche a `drizzle.config.ts`, o `drizzle-kit` crea il database fuori dal volume.

SQLite in produzione: cosa comporta

- **Un solo processo scrive**. Niente due repliche, niente autoscaling: condividere il file via rete corrompe il database. Si scala *verticalmente*, non in orizzontale.
- Quindi **niente rolling deploy**: per un istante vecchio e nuovo container toccherebbero lo stesso file. Si ferma e si riavvia, con qualche secondo di interruzione.
- **Niente serverless** (funzioni Vercel/Netlify, Workers): filesystem effimero, molte istanze in parallelo, nessun volume. SQLite lì non regge senza cambiare archivio.
- **Backup**: il file va copiato *a caldo* con `sqlite3 db .backup` o `VACUUM INTO`. Un `cp` mentre l'app scrive, con il WAL attivo, produce una copia inconsistente.
- Le immagini salvate come data URL base64 finiscono **dentro** il file: fino a 5 MB per articolo, dentro ogni backup. Spostarle su disco con un upload multipart resta un esercizio; quei file vorranno lo stesso volume.

Quando passare a Postgres

```
# drizzle.config.ts: dialect 'sqlite' -> 'postgresql', url -> DATABASE_URL
# db/index.ts: better-sqlite3 -> postgres-js (o node-postgres) + pool
# db/schema.ts: sqliteTable -> pgTable, real -> doublePrecision,
#               integer({mode:'boolean'}) -> boolean, text created_at -> timestamp
npm exec -w backend -- drizzle-kit generate   # SQL di migrazione versionato
npm exec -w backend -- drizzle-kit migrate    # applicato dal deploy
```

- Drizzle rende il passaggio un **cambio di driver più una migrazione**: le query restano quelle, i tipi delle colonne cambiano.
- Conviene quando serve più di una replica, quando il backup a caldo diventa un problema, o quando i dati non stanno più comodi su un disco solo.
- In produzione si abbandona `db:push --force` (riscrive lo schema e può perdere dati) per le migrazioni generate e committate.
- Via di mezzo: SQLite gestito (Turso/libSQL), stesso dialetto, replica inclusa.

Variabili d'ambiente: le cinque che contano

Variabile	Dove	Perché
`JWT_SECRET`	backend	oggi il default è `'dev-secret-change-me'`: con quello in produzione **chiunque** si firma un token con `isAdmin: true`
`DATABASE_URL`	backend	percorso del file sul volume persistente
`PORT`	backend	la piattaforma decide la porta, il codice la legge
`NODE_ENV`	backend	fuori da `production` il gestore errori rispedisce `detail` con il messaggio interno: informazione regalata a un attaccante
`VITE_API_URL`	frontend	una per ambiente, letta *al build*

- All'avvio, il backend dovrebbe **rifiutarsi di partire** se `NODE_ENV=production` e `JWT_SECRET` è ancora il default. Dieci righe che evitano un disastro.

Desktop: build su tre sistemi e release

```yaml
# .github/workflows/release-desktop.yml (Tauri)
on: { push: { tags: ["v*"] } }
jobs:
  build:
    strategy:
      matrix: { os: [ubuntu-22.04, windows-latest, macos-latest] }
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - uses: actions/setup-node@v4
        with: { node-version: 24, cache: npm }
      - run: npm ci
      - uses: tauri-apps/tauri-action@v0
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          TAURI_SIGNING_PRIVATE_KEY: ${{ secrets.TAURI_KEY }}
          VITE_API_URL: https://api.minicommerce.example/api
        with: { projectPath: frontend, tagName: ${{ github.ref_name }}, releaseDraft: true }
```

`projectPath` è `frontend`: lì vive `src-tauri/`, creato da `scripts/setup-tauri.mjs` (branch `desktop/tauri`). Un tag `v1.2.0` produce una release con MSI, DMG, AppImage e il manifest per l'auto-update; con Electron il passo finale è `npm run package` in `desktop/`. Senza il proxy di Vite, `VITE_API_URL` assoluto è obbligatorio.

Mobile: automatizzare gli store

- **Fastlane**: lo standard per firmare e caricare su Play Console e App Store Connect da riga di comando.
- **EAS** (Expo) e **Ionic Appflow**: build in cloud, anche iOS senza Mac. Costo mensile, tempo risparmiato.
- GitHub Actions con runner `macos-latest`: Xcode incluso, ma i minuti macOS costano 10 volte quelli Linux.
- Keystore Android e certificati Apple: nei *secrets*, codificati in base64, mai nel repo.
- Flusso tipico: tag $\rightarrow$ build $\rightarrow$ traccia interna / TestFlight $\rightarrow$ promozione manuale in produzione.
- La revisione dello store dura giorni: le correzioni urgenti vanno nel frontend web, non nell'app.

Ambienti e feature flag

Ambiente	Scopo	API	Chi lo usa
development	la vostra macchina	`localhost:3001`	voi
preview	una PR	staging	revisori
staging	come produzione, dati del seed	staging	QA, cliente
production	clienti veri	produzione	tutti

- Staging ha il suo file `.db` popolato da `db:seed`: si può svuotare senza paura, ed è lì che si provano le migrazioni **prima** della produzione.
- **Feature flag**: funzionalità nascoste dietro un interruttore (Unleash, LaunchDarkly, o un campo nella risposta dell'API). Si rilascia il codice senza rilasciare la funzionalità.
- Versione visibile nell'interfaccia: `import.meta.env.VITE_APP_VERSION`, iniettata dalla CI dal tag git. Senza, nessuno sa quale build ha davanti.

Errori: Sentry (o simili)

```
npm install @sentry/react -w frontend
```

```
import * as Sentry from '@sentry/react';
Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  release: `minicommerce-web@${import.meta.env.VITE_APP_VERSION}`,
  environment: import.meta.env.MODE,
  integrations: [Sentry.browserTracingIntegration(), Sentry.replayIntegration()],
  tracesSampleRate: 0.1, replaysOnErrorSampleRate: 1.0,
});
// nell'Error Boundary del modulo 1: onError={(e) => Sentry.captureException(e)}
```

- Con le **source map** caricate in CI (e non pubblicate accanto al bundle), lo stack trace mostra il vostro TypeScript, non il codice minificato.
- Oggi un errore si vede solo nel toast dell'utente: nessuno, lato vostro, sa che è accaduto.
- Vale anche per il backend (`@sentry/node`): meglio di `console.error` in un log che nessuno legge.

Prestazioni reali, analytics, uptime

```
import { onLCP, onINP, onCLS, type Metric } from 'web-vitals';
// POST /api/vitals non esiste ancora nel backend: aggiungerlo e' parte del laboratorio
const invia = (m: Metric) => navigator.sendBeacon('/api/vitals',
  JSON.stringify({ name: m.name, value: m.value, page: location.pathname }));
onLCP(invia); onINP(invia); onCLS(invia);
```

- Le metriche di laboratorio (Lighthouse) non bastano: i dati reali mostrano i telefoni lenti e le reti pessime. Il catalogo con le immagini base64 si misura **lì**, non sul vostro portatile.
- Analytics rispettosi della privacy (**Plausible**, **Umami**, Fathom): niente cookie, niente banner.
- Uptime: un ping ogni minuto su `GET /api/health` e sul sito, con avviso su Telegram o Slack. L'endpoint esiste già: usatelo.

Versioni e changelog

```
npm install -D @changesets/cli && npx changeset init   # nella root del monorepo
npx changeset              # "cosa e' cambiato?" -> un file in .changeset/
npx changeset version      # aggiorna le versioni e i CHANGELOG.md
git tag v1.3.0 && git push --tags     # -> parte la release desktop
```

- **Semantic versioning**: MAJOR rompe, MINOR aggiunge, PATCH corregge. L'auto-update desktop e mobile si basa su questo.
- In un monorepo i changeset dicono *quali* pacchetti cambiano: toccate `shared` e vi ricordano che backend, frontend, desktop e mobile ne dipendono.
- Il changelog lo leggono gli utenti (note di rilascio) e i colleghi (cosa è successo fra due tag).
- Convenzione dei commit (`feat:`, `fix:`) $\rightarrow$ changelog generabile automaticamente.

Laboratorio 7

Laboratorio: Dal commit all'utente senza toccare nulla a mano

1. Monorepo con npm workspaces + Turborepo: `backend`, `frontend`, `packages/shared` con i tipi del contratto (`Product`, `Category`, `AuthUser`, `Cart`, `CartItem`, `MockOrder`).
2. Aggiungere gli script mancanti (`typecheck`, `lint` al backend) e una CI che su ogni PR esegue lint, tipi, test, build e `npm audit`.
3. Deploy automatico su push in `main`: `frontend/dist` su hosting statico, backend in container con volume persistente e segreti dall'ambiente.
4. Una release desktop su tag con `tauri-action` (almeno Windows e Linux).
5. Sentry con source map; `web-vitals` inviate a `POST /api/vitals`, da aggiungere.

Domande di verifica

Perché `npm ci` e non `npm install` in CI? Cosa cacha Turborepo? Perché il backend non può girare in due istanze? Cosa accade se `NODE_ENV` non è `production`?

Sommario del modulo

9. Lavorare con gli agenti di programmazione
9.1 Cosa sono e cosa fanno
9.2 Il file di contesto del progetto
9.3 Metodo
9.4 Cosa resta vostro

Dal completamento automatico all'agente

Livello	Cosa fa	Esempi
Completamento	suggerisce la riga successiva	Copilot classico
Chat	risponde e propone codice da incollare	ChatGPT, Claude nel browser
Agente	legge il repo, pianifica, modifica file, *esegue comandi e controlli*, corregge e ripete finché il compito è fatto	Claude Code, Codex, Copilot coding agent, Cursor, Gemini CLI

- L'agente lavora nel **vostro** terminale: apre `frontend/src/pages/CatalogPage.tsx`, lancia `npm run build`, legge gli errori di TypeScript.
- Cambia il vostro ruolo: da chi scrive ogni riga a chi **specifica, indirizza e verifica**.

Dove rende di più

- **Codice meccanico**: le funzioni axios per un gruppo di endpoint (come quelle del carrello in `lib/api.ts`), le colonne di una tabella admin, un form con le stesse regole di validazione delle altre.
- **Traduzioni e migrazioni**: da `useEffect + fetch` sparsi a un hook condiviso in tutte le pagine, da `window.confirm` a `AlertDialog`.
- **Esplorazione**: "come viene aggiunto il Bearer alle richieste in questo repo?", "chi usa `formatPrice`?".
- **Debug**: dato un 401 inatteso dopo otto ore, trovare il punto in cui il token scaduto non viene gestito.
- **Rifiniture**: `aria-label` sui pulsanti con sola icona, messaggi di errore, stati vuoti, documentazione.
- **Lavoro che non avreste fatto**: scrivere `openapi.yaml` per le API di minicommerce partendo dalle rotte vere.

Dove sbaglia (e come)

- **API inventate o vecchie**: "uso `GET /api/brands`" — non esiste. Oppure Tailwind v3 in un progetto con Tailwind v4.
- **Troppo codice**: un `BrandFilterProvider` con context quando serviva un `<Select>`. Chiedete il minimo.
- **Allargamento silenzioso**: "già che c'ero ho riformattato tutto `api.ts`". Il diff va letto tutto.
- **Dipendenze non richieste**: installa un pacchetto per i select quando `components/ui/select.tsx` c'è già.
- **Sicurezza**: segreti in chiaro, CORS aperto lasciato com'è, token spostato in un posto peggiore. La checklist del modulo 6 vale doppio.
- **Sicurezza operativa**: un agente che esegue comandi può lanciare `db:reset` e cancellare il vostro database locale.

minicommerce ha già il suo: AGENTS.md

```
# Repository Guidelines

## Project Structure
backend/src/ holds the Express API, route handlers, auth middleware,
SQLite/Drizzle schema and seed data. frontend/src/ contains the React app,
pages, shared components, API/auth helpers, styles and assets.
The project currently has no dedicated test directory or test runner.

## Build, Test, and Development
Use Node.js 24 or newer. Install dependencies separately in backend/ and
frontend/ with npm install. Backend: npm run dev starts the watch-mode API
on port 3001; npm run db:push applies the Drizzle schema and npm run db:seed
loads demo data and the admin account.
Frontend: npm run dev starts Vite on port 5173
(with /api proxied to the backend). npm run build type-checks and creates
the production bundle; npm run lint runs Oxlint.
npm run db:reset in backend/ deletes and rebuilds the local database,
including reseeding it. Use only when you intend to discard local data.

## Coding Style & Naming
Two-space indentation, semicolons, single quotes in backend files and double
quotes in frontend files. PascalCase for React components, camelCase for
variables/functions. Keep API access in frontend/src/lib/api.ts and database
definitions in backend/src/db/schema.ts.
```

Perché questo file funziona

- **Dice dove stanno le cose**: due cartelle, un ruolo ciascuna. L'agente non va a cercare le rotte dentro `frontend/`.
- **Dà i comandi esatti**: avvio, `db:push`, `db:seed`, build, lint. Senza, l'agente li indovina e sbaglia.
- **Ammette quello che manca**: "no dedicated test runner". Così non vi promette test che non può eseguire.
- **Fissa convenzioni verificabili**: indentazione, punto e virgola, `PascalCase` per i componenti.
- **Indica i confini**: le chiamate HTTP stanno in `lib/api.ts`, lo schema in `db/schema.ts`, non sparsi nelle pagine.
- **Avverte sui comandi distruttivi**: `db:reset` cancella il database. Una riga che vi salva un pomeriggio di dati.

Cosa manca all'`AGENTS.md` di oggi

- – Niente sui **quattro stati** di ogni schermata (caricamento, errore, vuoto, dati): `CatalogPage` ne gestisce tre, l'errore finisce solo in un toast. Un agente non sa che la regola esiste e ne salta un altro.
- – Niente **regole di accessibilità**: nessuno dice "i pulsanti con sola icona vogliono `aria-label`, non `title`" — e nel codice ce ne sono tre fatti così.
- – La regola delle **virgolette** non corrisponde al codice vero: dice apici singoli nel backend e doppi nel frontend, ma `frontend/src/lib/api.ts` usa gli apici singoli.
- – Non dice **cosa non fare**: non committare, non aggiungere dipendenze senza chiedere, non modificare i file copiati in `components/ui/`.
- – Non nomina le **scelte del corso**: un hook per i dati, filtri nell'URL, AlertDialog al posto di window.confirm.
- – Non dice che il progetto è **cresciuto**: due ruoli (`isAdmin`), carrello lato server solo per i clienti, e `POST /api/orders` che è un **ordine finto**. Un agente che vede `paymentLastFour` può mettersi a integrare un pagamento vero.

+ Tutto questo lo sistemate voi: è il laboratorio di questo modulo.

Il ciclo: specifica, piano, esecuzione, verifica

1. **Compito piccolo e chiaro**: “aggiungi al catalogo il filtro per marca”. Non “migliora il catalogo”.
2. **Chiedere il piano** prima: quali file toccherà, quali tipi cambierà, come verificherà. Correggere il piano costa niente; correggere il codice costa.
3. **Lasciarlo lavorare** con `npm run lint` e `npm run build` come vincolo: deve farli passare lui, non voi.
4. **Leggere il diff** come se fosse la PR di un collega junior brillante e un po’ frettoloso.
5. **Provare a mano**: aprite il catalogo, filtrate, provate con la tastiera. L’agente non ha occhi sull’interfaccia.
6. Commit con messaggio vostro. Il prossimo compito parte da un repo pulito.

Un prompt ben fatto: il filtro per marca

```
Aggiungi al catalogo il filtro per marca.

Contesto: GET /api/products accetta GIA' il parametro brand
(backend/src/routes/products.ts, filtro eq(products.brand, brand),
confronto esatto). Il frontend non lo usa: l'interfaccia ProductFilters in
frontend/src/lib/api.ts non ha il campo brand, e CatalogPage.tsx tiene i
filtri in useState. Non esiste un endpoint che elenca le marche.

Requisiti:
- aggiungi brand?: string a ProductFilters e passalo a fetchProducts
- in CatalogPage un Select "Marca" con "Tutte le marche" come default,
  costruito con components/ui/select.tsx (nessuna nuova dipendenza)
- le marche si ricavano dai prodotti ricevuti, ordinate e senza duplicati
- etichette in italiano, coerenti con il resto della pagina
- non toccare gli altri filtri, il backend o components/ui/

Prima dimmi il piano in 5 righe, poi procedi.
Alla fine esegui: cd frontend && npm run lint && npm run build
```

Perché quel prompt funziona

- **Contesto vero**: dice cosa esiste già (il parametro `brand` nel backend) e cosa no (l'endpoint delle marche). Senza, l'agente inventa `GET /api/brands`.
- **Requisiti verificabili**: un campo nel tipo, un componente, un comportamento di default. Si controllano uno per uno nel diff.
- **Vincoli espliciti**: nessuna dipendenza, componenti esistenti, backend intoccato.
- **Criterio di "fatto"**: `npm run lint` e `npm run build` verdi. Farli passare non è lavoro vostro.
- **Il piano prima**: cinque righe da leggere invece di trecento righe da rileggere.
- È la specifica del modulo 5, in piccolo. Lo stesso mestiere.

Secondo esempio: via `window.confirm`

```
In frontend/src/pages/AdminPage.tsx l'eliminazione di un articolo usa
window.confirm. Sostituiscilo con AlertDialog di shadcn/ui.

Il componente non c'e' ancora in src/components/ui/: aggiungilo con
  npx shadcn@latest add alert-dialog
(stile radix-nova, vedi frontend/components.json) e non modificare gli
altri file in components/ui/.

Requisiti:
- titolo e testo in italiano, con il nome dell'articolo da eliminare
- azioni "Annulla" ed "Elimina" (variante distruttiva), chiusura con Esc
- la chiamata deleteProduct e il toast di errore restano come sono
- il dialogo di modifica (ProductFormDialog) non va toccato

Piano in 5 righe, poi procedi. Verifica con npm run lint && npm run build.
```

Un compito con un confine netto: un file, un componente aggiunto, nessun cambio di logica.

Revisione del diff: cosa cercare su questo repo

- File toccati che **non c'entrano** con il compito
- `any`, `@ts-ignore`, regole di lint disattivate
- Tipi aggiornati in `lib/api.ts`? Un filtro nuovo è un campo nuovo in `ProductFilters`
- **Dipendenze** nuove in `package.json`: perché?
- Chiamate HTTP scritte nella pagina invece che in `lib/api.ts`
- Accessibilità dei controlli nuovi: label, `aria-label`, tastiera
- Segreti, URL assoluti al posto del proxy, `console.log`
- Messaggi in italiano coerenti con il resto dell'interfaccia
- I quattro stati della schermata ancora tutti gestiti
- Lo capite abbastanza da spiegarlo a un collega?

Se no

Chiedete all'agente di spiegare, semplificare o rifare. "Non capisco" è un motivo sufficiente per rifiutare un diff.

Strumenti collegati: MCP e automazione del browser

- **MCP** (Model Context Protocol): standard per collegare all'agente strumenti esterni: il browser, il database, Figma, l'issue tracker, la documentazione di una libreria.
- Con un server MCP per il browser (Playwright) l'agente **apre minicommerce, filtra il catalogo, fa screenshot** e verifica da solo che il filtro marca funzioni.
- Con un MCP su SQLite legge le categorie e gli articoli del seed invece di indovinare i nomi.
- Con la documentazione ufficiale (Context7 e simili): meno API inventate, meno Tailwind v3 in un progetto v4.
- Regola: ogni strumento in più è potere in più. Lettura prima della scrittura, e mai le credenziali di produzione.

Le competenze che valgono di più nel 2026

- **Leggere codice** velocemente e con giudizio: è l'attività principale.
- **Specificare**: trasformare "il catalogo è lento" in "paginare `GET /api/products` e smettere di rispedire le immagini base64 in ogni risposta".
- **Architettura e confini**: decidere dove stanno le cose (lo avete fatto in ogni modulo).
- **Giudizio di prodotto e UX**: l'agente non sa se un filtro in più aiuta o confonde.
- **Debug sistematico**: ipotesi, esperimento, conferma. L'agente accelera, voi guidate.
- **Fondamenti**: HTTP, browser, rendering, tipi. Senza, non riconoscete i suoi errori.

Detto altrimenti

L'agente moltiplica ciò che sapete. Moltiplicare zero dà zero.

Laboratorio 8

Laboratorio: Migliorare l'`AGENTS.md` del vostro repository

1. Riscrivete `AGENTS.md`: le scelte dei moduli 1–3, i quattro stati, le regole di accessibilità, la regola delle virgolette allineata al codice vero, le parti nuove del progetto (ruoli, carrello, ordine finto) e una sezione "cosa non fare".
2. Dategli poi un compito vero: il filtro per marca, oppure `AlertDialog` al posto di `window.confirm`.
3. Tenete un **diario**: prompt, piano proposto, cosa avete corretto e perché, tempo impiegato.
4. Revisione incrociata: un compagno rivede il vostro diff senza sapere cosa ha scritto l'agente e cosa voi.
5. Rifate lo stesso compito **senza** il file di contesto e confrontate i due diff.

Domande di verifica

Perché il piano prima del codice? Tre segnali di un diff da rifiutare? Quale riga dell'`AGENTS.md` attuale vi ha salvato il database locale?

Grazie

Buon lavoro!

Domande, dubbi e proposte: durante il laboratorio o via e-mail.

Il codice di minicommerce e le soluzioni dei laboratori sono nel repository condiviso della classe.
Il progetto finale e la valutazione: dispensa `progetto-finale.pdf`.